LLM INTERVIEW HANDBOOK 2026

大模型面试手册

大模型算法岗 · 基础模型 · Post-Training · 推理优化

数学推导 · PyTorch 手写 · 原理图解 · 高频追问 · 工程估算

2026 Edition · Version 2.1

从公式、张量形状与系统瓶颈出发，回答面试官真正会追问的“为什么”

版本与使用说明

版本： 2026 Edition · v2.1

适用岗位：

- LLM 算法工程师 / Foundation Model
- Post-Training / Alignment
- 推理优化与模型部署
- 大模型应用算法与 Agent

本资料围绕公开技术资料、数学推导与工程实践组织，重点服务个人学习与面试准备。技术结论应结合具体模型实现、硬件环境和业务约束判断。

版权说明

本资料仅供购买者个人学习与面试准备使用。未经许可不得批量转载、二次销售或公开传播。

目录

阅读方法 ix

术语与符号约定 x

第一部分　基础 1

第一章　分词与词元化（Tokenization） 2

1.1　常见方案 2

1.2　中文分词的注意点 3

1.2.1　面试追问链 4

面试高频问题：分词与词元化 4

第二章　Transformer 基础 5

2.1　词元嵌入与位置 5

2.2　从词元嵌入到 LM Head 6

2.3　自注意力（Self-Attention）推导 7

2.3.1　为什么 Q、K、V 使用不同投影？ 10

2.3.2　对应实现：Scaled Dot-Product Attention 11

2.4　因果遮罩、残差与归一化 12

2.4.1　残差流为什么保持固定宽度？ 13

2.5　前馈网络（FFN）与 SwiGLU 14

2.5.1　为什么先升维再降维？ 14

2.5.2　SwiGLU 的两条上投影 16

2.5.3　对应实现：SwiGLU 17

2.5.4　对应实现：最小前置归一化 Transformer 块 18

面试高频问题：Transformer 基础 19

第二部分　预训练 **20**

第三章　自回归语言建模 **21**

3.1 PyTorch 中的预测分数与目标标签 23

3.2 预训练流水线的面试拆解 25

第四章　数据与优化 **26**

4.1 数据混合、去重与序列拼接 26

4.2 优化细节 26

4.3 Scaling Laws 与 Compute-Optimal Training 28

4.3.1 Compute-optimal：在约束下平衡容量与数据 29

4.3.2 Chinchilla-style 直觉：不要只放大参数 30

面试高频问题：预训练 31

第三部分　现代 Transformer 改进 **32**

第五章　位置、归一化与前馈网络 **33**

5.1 RMSNorm 33

5.1.1 对应实现：RMSNorm 34

5.2 RoPE 35

5.2.1 对应实现：RoPE 二维旋转 36

5.3 ALiBi 与滑动窗口 38

第六章　多头、共享键值与分组查询注意力 **39**

6.1 多头潜在注意力（MLA） 41

6.2 查询/键归一化（QK normalization） 42

面试高频问题：现代结构 42

第四部分　高效注意力与推理 **44**

第七章　键值缓存（KV Cache） **45**

7.1 面试追问链 47

7.2 对应实现：单步逐词元解码与键值缓存 48

第八章 FlashAttention 50

8.1 面试追问链 52

第九章 量化、采样与动态批处理 53

9.1 量化 53

9.2 温度、Top-k 与 Top-p 采样 54

9.2.1 对应实现：Temperature、Top-k 与 Top-p 56

9.3 束搜索（Beam Search） 57

9.3.1 对应实现：单步 Beam 扩展 58

9.4 推测式解码（Speculative Decoding） 59

9.5 连续批处理与分页注意力 60

面试高频问题：推理 61

第五部分 推理模型与测试时计算 62

第十章 Thinking 与 Non-Thinking 模式 63

10.1 六个容易混淆的概念 63

10.2 推理词元、延迟与适用任务 64

第十一章 Test-Time Scaling 与验证 66

11.1 多次采样、自洽性与 Best-of-N 66

11.1.1 对应实现：Best-of-N 的系统边界 67

第十二章 RLVR：可验证奖励强化学习 69

12.1 Outcome、Process 与稀疏奖励 69

第十三章 推理蒸馏 72

第六部分 后训练 74

第十四章 监督微调与奖励模型 75

14.1 监督微调（SFT） 75
14.2 奖励建模（Reward Modeling） 77
14.2.1 对应实现：奖励模型成对损失 78

第十五章 PPO、DPO 与 GRPO **80**
15.1 近端策略优化（PPO）与人类反馈强化学习 80
15.1.1 对应实现：PPO 裁剪策略损失 81
15.1.2 旧策略与参考策略不是同一个概念 82
15.2 直接偏好优化（DPO） 83
15.2.1 对应实现：DPO 成对偏好损失 84
15.3 组相对策略优化（GRPO） 85
15.3.1 对应实现：GRPO 组内优势 87
面试高频问题：后训练 88

第七部分 参数高效微调与蒸馏 **89**

第十六章 LoRA 与 QLoRA **90**
16.1 对应实现：带缩放与零更新初始化的 LoRA 线性层 91
16.2 LoRA 到底省什么？ 92
16.3 QLoRA 93

第十七章 知识蒸馏 **94**
17.1 Logits、回答与推理蒸馏 94

第八部分 混合专家模型 **96**

第十八章 稀疏专家模型 **97**
18.1 为什么专家通常替换 FFN？ 98
18.2 对应实现：朴素 Top-k MoE 99
18.3 容量与负载均衡 100
18.4 并行与工程影响 101
面试高频问题：MoE 102

第九部分　分布式训练 **103**

第十九章　并行策略 **104**

19.1 为什么需要不同的并行维度 104
19.2 分布式数据并行与全归约 106
19.3 ZeRO 与 FSDP 的内存推理 107
19.3.1 训练显存的完整核算框架 108
19.4 激活检查点（Activation Checkpointing） 109
面试高频问题：分布式 110

第十部分　模型评测 **111**

第二十章　LLM Evaluation **112**

20.1 指标分别回答什么问题 112
20.2 污染、评审偏差与线上闭环 113

第十一部分　检索增强生成与智能体 **116**

第二十一章　检索增强生成（RAG） **117**

21.1 为什么需要切块、召回与重排 117

第二十二章　智能体（Agent） **120**

22.1 为什么需要参数规范、状态与安全边界 120
面试高频问题：RAG 与 Agents 122

第十二部分　高频面试问题与速查 **123**

第二十三章　高频公式速查 I **124**

第二十四章　高频公式速查 II **125**

第二十五章　Interview Last-Day Checklist **126**

25.1 不同岗位怎么刷这本书？ 126

第二十六章 面试问答索引：210 题 **128**

26.1 Tokenization（8 题） 128
26.2 Transformer Fundamentals（27 题） 128
26.3 Pretraining + Scaling Laws（25 题） 130
26.4 Modern Architecture（20 题） 132
26.5 Inference / KV / Flash（27 题） 134
26.6 Reasoning / Thinking（23 题） 136
26.7 Post-Training / PEFT / Distillation（32 题） 137
26.8 MoE（11 题） 140
26.9 Distributed Training（16 题） 141
26.10 Evaluation（9 题） 142
26.11 RAG + Agents（12 题） 142

参考文献与进一步阅读 **144**

阅读方法

本书不把技术名词当作孤立答案。几乎每一种 LLM 方法都来自上一种方案在规模、数据、数值或系统上的具体失败。阅读每节时，建议始终追问五件事：

1. **原始方案是什么：**没有这个技巧时，系统如何工作？
2. **瓶颈在哪里：**问题来自计算量、显存、带宽、优化稳定性、数据质量，还是目标与行为不一致？
3. **方法改变了什么：**它修改的是目标函数、张量布局、信息可见范围、参数共享方式，还是执行调度？
4. **为什么有效：**公式中的哪一项或系统中的哪一次重复工作被改变？
5. **代价和边界是什么：**它新增了什么误差、通信、超参数或质量风险，又在哪些场景下不适用？

后文的“提出动机”框先回答前两项，“具体例子”框把抽象机制落到可追踪的输入和中间结果上，公式与代码解释方法细节，章节结尾再交代代价与下一种方法为何出现。面试时也可以沿用这条链路，先说问题再说方案，通常比直接背定义更有说服力。

术语与符号约定

专业名词在第一次出现时采用“中文含义（英文原词或缩写）”的写法，后文再使用业界常见简称。代码中的变量名和 API 保留英文，以便与 PyTorch 文档对应。

常见张量符号

符号	含义
B	批次大小，即一次并行处理的样本数
T	序列长度，即每个样本包含的词元数量
V	词表大小，即模型可输出的词元类别数
C 或 d_{model}	模型隐藏维度，即每个位置的向量宽度
H	查询注意力头数量
D 或 d_h	单个注意力头的维度，常有 $C = HD$
L	Transformer 层数
n_{kv}	键/值（key/value）的头数量；在 MHA 中通常等于 H，在 GQA/MQA 中更少
P	模型参数总数
i, j, t	位置或时间步下标；具体范围由当前公式说明
θ, ϕ	可学习参数集合；θ 常指主模型或策略参数，ϕ 常指奖励模型参数

因此 shape $[B, T, C]$ 表示“B 个样本，每个样本 T 个词元，每个词元用 C 维向量表示”；$[B, H, T, D]$ 则表示把隐藏维拆成 H 个注意力头，每头宽度为 D。

常见中英文术语

术语	本书中的含义
词元（token）	分词器输出的最小处理单元，可能是字、子词、标点或字节
词元编号（token id）	词元在有限词表中的整数下标
批次（batch）	一次并行送入模型的一组样本
嵌入向量（embedding）	由词元编号查表得到的连续向量
隐藏状态（hidden state）	Transformer 中间层对当前位置的向量表示
未归一化分数（logits）	softmax 之前的实数分数，不要求和为 1
目标标签（target/label）	希望模型预测出的正确词元编号
提示词（prompt）	用户输入或生成开始前已经给定的上下文
回答（response）	模型在提示词之后生成的内容
遮罩（mask）	用布尔值或极小分数禁止某些位置参与计算
缓存（cache）	保存可复用的中间结果，避免重复计算
注意力头（head）	一组独立的查询/键/值（query/key/value）投影与注意力关系
损失（loss）	训练中需要最小化的标量目标

推理的两个基本阶段

阶段	做什么	主要特点
提示词预填充（prefill）	一次并行处理完整提示词，并为各层建立键值缓存	矩阵规模大，通常更偏计算密集
逐词元解码（decode）	每一步只生成一个新词元，并读取此前的键值缓存	串行步骤多，通常更受显存带宽限制

例如提示词是“我喜欢”，提示词预填充会同时处理“我”和“喜欢”；随后模型逐步生成“机器”“学习”，这两个生成步骤分别属于两次逐词元解码。后文再使用 prefill 和 decode 时，都指这里定义的两个阶段。

第一部分

基础

第一章　分词与词元化（Tokenization）

提出动机：从原始文本到有限词表

神经网络只能接收数值张量，而原始文本是长度不定的 Unicode 字符串。若按整词建表，训练语料中没有出现过的词会成为未登录词（out-of-vocabulary，OOV），词表也会迅速膨胀；若逐字符或逐字节处理，虽然覆盖完整，序列却明显变长。分词器（tokenizer）的提出，就是为了在“有限词表、完整覆盖、较短序列”之间取得可训练的折中。

分词器的任务是把文本切成词元（token），再把每个词元映射为词元编号（token id）。LLM 训练和推理真正处理的是这些整数编号；分词器的质量会影响上下文利用率、未登录词处理、跨语言鲁棒性和生成边界。

1.1　常见方案

字节对编码（Byte Pair Encoding，BPE）是为了解决词级词表存在未登录词、字符级序列又过长的矛盾。它从字符或字节开始，反复合并高频相邻片段：常见词可被压成较少词元，罕见词仍能拆回更小单元。优点是简单、可控、压缩率高；缺点是合并规则依赖训练语料分布。

字节级 BPE（Byte-level BPE）针对字符表仍可能遗漏罕见文字、表情符号或控制符的问题，把最底层单位退回到字节。256 种字节值即可表示任何输入，因此理论上不再需要未知词元（unknown token）。代价是一个非英文字符通常由多个 UTF-8 字节组成，可能被切得更碎。

SentencePiece 针对传统分词器依赖英文空格和语言专用预分词的问题，直接在原始文本上学习子词单元，常见一元语言模型（unigram language model）与 BPE 两类方案。它把空格也视为普通符号，不依赖“先分词再建词表”，因此更适合中日韩和多语言混合文本。

具体例子：“机器学习”如何变成词元编号

假设训练语料只有“机器学习”和“机器翻译”。最初按字符切开：

$$[\text{机}, \text{器}, \text{学}, \text{习}], \qquad [\text{机}, \text{器}, \text{翻}, \text{译}].$$

相邻片段“机器”出现两次，频率最高，于是 BPE 把它合并为一个新词元：

$$[\text{机器}, \text{学}, \text{习}], \qquad [\text{机器}, \text{翻}, \text{译}].$$

若之后“学习”也被合并，输入“我爱机器学习”可能被切成

$$[\text{我}, \text{爱}, \text{机器}, \text{学习}].$$

再假设词表映射为“我”$\mapsto 17$、“爱”$\mapsto 42$、“机器”$\mapsto 310$、“学习”$\mapsto 908$，模型真正收到的就是整数序列 $[17, 42, 310, 908]$。词元编号只是词表下标，本身没有大小或距离含义；后续的嵌入（embedding）查表才把它变成向量。若新词“机器视觉”从未整体出现，只需退回已有子词 [机器, 视, 觉]，而不是把整词变成未知词元。字节级方案还能把词表未覆盖的罕见字符退回 UTF-8 字节，因此任何输入最终都能编码。

维度	小词表	大词表	面试要点
序列长度	更长	更短	长序列增加 attention 与键值缓存成本
稀有词	更可组合	可能有专用词元	大词表需要更多嵌入参数
多语言	更稳健	依赖语料覆盖	中文常需平衡字、词、标点与英文混合

表 1.1 词表大小的主要权衡。

面试一句话回答

分词器不是预处理小细节；它直接改变序列长度、词表维度、嵌入参数量、键值缓存成本和多语言泛化。

1.2 中文分词的注意点

中文没有天然空格边界，字符粒度虽然稳健但序列长；词粒度压缩率好但受分词词典影响。现代 LLM 常使用字节级或 SentencePiece 类方案，使中文、英文、代码、表情符号和罕见字符都能统一编码。

1.2.1 面试追问链

面试深挖：为什么分词方案会改变模型的系统成本？

20 秒回答：分词器决定同一段文本会变成多少个词元 T。训练和提示词预填充中的稠密 attention 位置对随 T^2 增长，键值缓存则随 T 线性增长；词表变大虽然可能缩短序列，却会扩大嵌入层和 LM Head。因此它同时改变计算、显存和参数量。

深入解释：设平均每个字符对应 r 个词元，一段长度为 N 的文本约有 $T = rN$ 个词元。把 r 增大 25%，attention 分数位置对近似增至 $1.25^2 = 1.5625$ 倍，键值缓存增至 1.25 倍。反过来，大词表使嵌入矩阵参数量 Vd_{model} 增长，而且低频词元可能得不到充分训练。

面试官继续追问：

1. 为什么不能只比较词表大小，而要比较同一评测集上的词元数？
2. 不同 tokenizer 的困惑度为什么不能直接横向比较？
3. 字节回退解决了什么问题，又付出了什么代价？

面试高频问题

Q: 为什么不全部按字符切分？

20 秒回答：字符切分简单且没有未登录词，但序列会变长，attention 计算和键值缓存显存都会上升，而且语义组合要由模型完全学习。

深入解释：自注意力在提示词预填充（prefill）阶段近似需要处理 $O(T^2)$ 个位置对，逐词元解码（decode）的键值缓存需要 $O(T)$ 存储。这里 T 表示序列中的词元数。字符级分词器把同样文本变成更大的 T，会同时放大训练和推理成本。

Q: 词表变大一定更好吗？

不是。词表变大会缩短序列，但嵌入层和语言模型输出头（LM head）的参数增加，低频词元学得差，还可能降低跨域泛化。

第二章　Transformer 基础

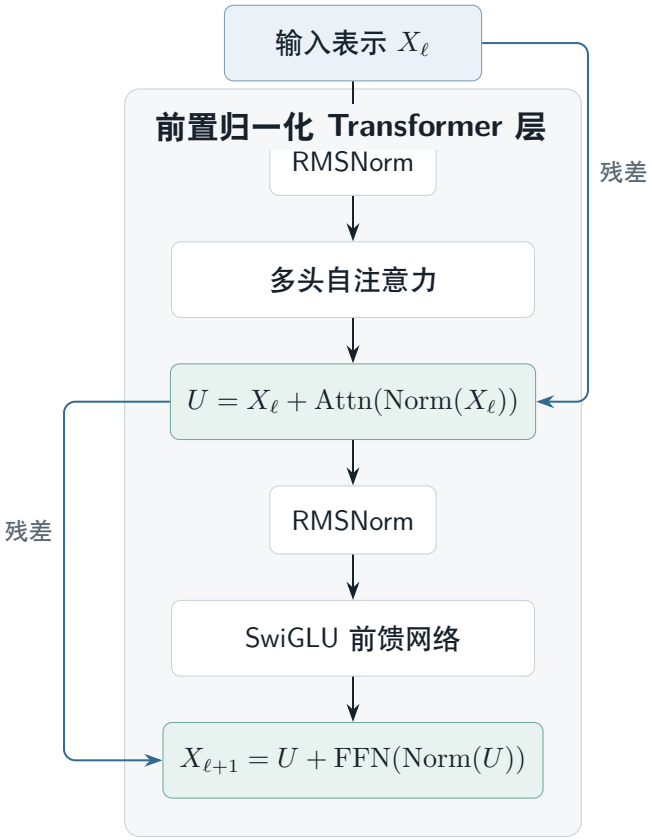


图 2.1　现代仅解码器 Transformer 的单层主路径。蓝色旁路表示恒等残差，绿色节点给出两次更新后的张量。

2.1　词元嵌入与位置

提出动机：为什么内容表示之外还需要位置？

自注意力只根据向量之间的相似度工作。若交换两个词元，同时交换对应的 Q, K, V 行，注意力结果也只会跟着交换，模型本身无法区分“我喜欢你”和“你喜欢我”的顺序。位置编码因此不是装饰信息，而是打破这种置换对称性的必要条件。

输入词元编号序列 $x_{1:T} = (x_1, \ldots, x_T)$ 先查嵌入表，得到 $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$。其中，$x_t$ 是第 t 个位置的词元编号，T 是序列长度，B 是批次中的样本数，d_{model} 是每个词元向量的维度。由于自注意力本身对位置置换不敏感，模型必须注入位置信息：绝对位置编码、ALiBi 或 RoPE 都是在解决“顺序从哪里来”的问题。

有了“内容是什么”和“位于哪里”的输入表示后，下一个问题是如何让任意位置直接交换信息。RNN 需要沿时间步串行传递，卷积的感受野又受核宽和层数限制，这正是自注意力出现的背景。

2.2 从词元嵌入到 LM Head

设词表大小为 V，模型隐藏维度为 d_{model}。词元嵌入矩阵

$$E \in \mathbb{R}^{V \times d_{\text{model}}}$$

的第 i 行就是词元编号 i 的初始向量。经过多层 Transformer 后，位置 t 得到隐藏状态 $h_t \in \mathbb{R}^{d_{\text{model}}}$。语言模型输出头（LM Head）再把它投影回词表维度：

$$z_t = h_t W_{\text{lm}} + b, \qquad W_{\text{lm}} \in \mathbb{R}^{d_{\text{model}} \times V}, \quad z_t \in \mathbb{R}^{V}.$$

z_t 是未归一化词表分数而不是概率。只有沿词表维执行 softmax 后，才得到 V 个候选词元的概率分布。隐藏状态也不是概率：它是可含正负值、没有归一化约束的内部特征向量。

嵌入层和 LM Head 可以使用权重绑定（weight tying），即令 $W_{\text{lm}} = E^{\top}$。这样输入侧“词元如何表示”和输出侧“隐藏状态与哪个词元相似”共享同一组几何基，并节省 Vd_{model} 个参数。代价是输入表示与输出分类器不再完全独立，是否绑定仍取决于模型设计。

计算题：大词表为什么昂贵？

设 $V = 128000$、$d_{\text{model}} = 4096$。单个嵌入矩阵有

$$Vd_{\text{model}} = 128000 \times 4096 = 524{,}288{,}000$$

个参数，BF16 权重约占 1.05 GB。若嵌入层与 LM Head 不绑定，两者合计约 10.49 亿参数；绑定后可省去一份约 5.24 亿参数的矩阵。

面试深挖：为什么 LM Head 必须投影到词表维？

20 秒回答： Transformer 输出的是 d_{model} 维隐藏特征，而下一个词元预测是一个 V 分类问题。LM Head 用 $W_{\text{lm}} \in \mathbb{R}^{d_{\text{model}} \times V}$ 为每个词元生成一个分数，再沿 V 维 softmax，才能得到词表上的条件分布。

深入解释： 若批量隐藏状态为 $H \in \mathbb{R}^{B \times T \times d_{\text{model}}}$，矩阵乘法后 logits 形状为 $[B, T, V]$。softmax 必须沿最后的词表维做，因为每个样本、每个位置都要在 V 个互斥候选中归一化；若沿时间维 softmax，得到的会是“不同位置互相竞争”，不再对应下一个词元分布。

面试官继续追问：

1. 为什么 logits 可以是任意实数?
2. weight tying 节省多少参数?
3. 词表很大时，LM Head 的参数量和计算量如何变化?
4. temperature 为什么更自然地作用在 logits 上?

2.3 自注意力（Self-Attention）推导

提出动机：并行建模长距离依赖

序列模型既要让远距离词元交互，又希望训练时能并行。RNN 的信息必须逐步传递，长路径不利于梯度和吞吐；卷积要堆叠多层才能覆盖远距离。自注意力让每个查询向量（query）直接与所有键向量（key）比较，把任意两点之间的交互路径缩短为一步，代价则是长度 T 上的二次计算和显存。

给定一层输入 $X \in \mathbb{R}^{B\times T\times d_{\text{model}}}$，线性投影得到：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

这里 W_Q, W_K, W_V 是三组可学习投影矩阵；Q 是查询矩阵（query），表示当前位置“想找什么”；K 是键矩阵（key），表示每个位置“可用什么特征被匹配”；V 是值矩阵（value），表示匹配后真正取回的内容。三组投影不是多余复制：把匹配空间与内容空间分开后，模型可以用一种特征判断相关性，再传递另一种特征；不同注意力头又能在不同子空间学习语法、指代、位置或主题关系。

单头时 $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}}\times d_k}$，因此 $Q, K, V \in \mathbb{R}^{B\times T\times d_k}$，其中 d_k 是单头查询和键的维度。多头时通常重排为 $[B, H, T, d_h]$：H 是注意力头数，d_h 是每个头的维度，常见设置满足 $d_{\text{model}} = Hd_h$。

对批次中的某个样本、某个注意力头，位置 i 到位置 j 的分数可逐元素写成

$$s_{ij} = \frac{1}{\sqrt{d_h}} \sum_{r=1}^{d_h} q_{ir}k_{jr} + M_{ij}, \qquad M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

这里 i 是发起查询的位置，j 是被读取的键和值的位置，r 枚举头内的第 r 个向量分量；q_{ir} 与 k_{jr} 分别是对应分量。M_{ij} 是因果遮罩：允许读取历史和当前位置时取 0，禁止读取未来时取 $-\infty$。因此 s_{ij} 表示缩放并遮罩后的相关性分数。这一定义也说明了因果遮罩为什么能令未来位置的 softmax 权重严格为零。

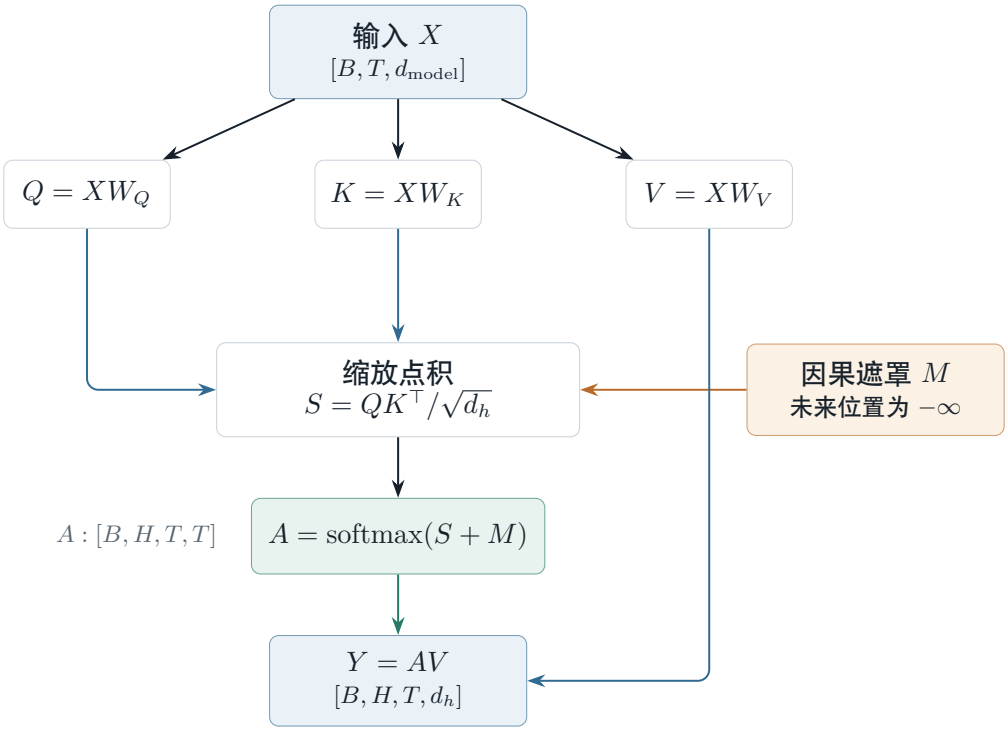


图 2.2 自注意力的完整数据流。分数矩阵沿最后一个维度归一化，随后用每一行权重汇聚对应的值向量。

不考虑遮罩时，每个查询与每个键的原始注意力分数是内积：

$$S_{ij} = q_i^\top k_j.$$

加入因果遮罩矩阵 M 后：

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \qquad Y = AV.$$

其中 S 是原始分数矩阵，$A \in \mathbb{R}^{B\times H\times T\times T}$ 是沿最后一维归一化后的注意力权重，$Y \in \mathbb{R}^{B\times H\times T\times d_h}$ 是对值矩阵 V 加权求和后的输出。

具体例子：一个查询如何读取历史词元

考虑三个位置“我 / 喜欢 / 机器”，当前计算位置 2“喜欢”的输出。假设除以 $\sqrt{d_h}$ 后，它与三个键的分数为

$$[2,\ 1,\ 4].$$

第三个位置“机器”属于未来，因果遮罩把它改成 $-\infty$：

$$[2,\ 1,\ -\infty].$$

Softmax 后约为 $[0.731, 0.269, 0]$。若 $v_1 = (1, 0)$、$v_2 = (0, 2)$，则

$$y_2 = 0.731v_1 + 0.269v_2 = (0.731, 0.538).$$

这就是 attention 的实际含义：先用查询和键决定“从每个可见位置取多少”，再对值向量做加权求和。未来位置即使原始分数最高，也会因遮罩得到零权重。

关键推导

若查询向量 q 和键向量 k 的每个分量独立、均值为 0、方差为 1，则

$$\mathrm{Var}(q^\top k) = \mathrm{Var}\left(\sum_{\ell=1}^{d_k} q_\ell k_\ell\right) = d_k.$$

求和下标 ℓ 枚举向量的 d_k 个分量。不缩放时，d_k 越大，softmax 输入方差越大，分布越尖，梯度容易饱和。除以 $\sqrt{d_k}$ 后方差约回到 1。

对第 i 个查询，softmax 权重和输出分别为

$$a_{ij} = \frac{e^{s_{ij}}}{\sum_{u \le i} e^{s_{iu}}}, \qquad y_i = \sum_{j \le i} a_{ij} v_j.$$

其中 a_{ij} 表示位置 i 分配给位置 j 的权重，y_i 是位置 i 的输出。因此 $a_{ij} \ge 0$ 且 $\sum_{j \le i} a_{ij} = 1$，$y_i$ 是可见值向量的凸组合。softmax 对输入分数的雅可比矩阵（Jacobian）元素为

$$\frac{\partial a_{ij}}{\partial s_{ik}} = a_{ij}(\mathbf{1}[j = k] - a_{ik}),$$

这里 k 是另一个分数位置的下标，$\mathbf{1}[j = k]$ 是指示函数：条件成立取 1，否则取 0。该式解释了当某个 a_{ij} 接近 1、其余权重接近 0 时，大部分梯度会变小。

单头 attention 只能在一套相似度空间中形成一张权重图。多头注意力（Multi-Head Attention，MHA）的提出，是为了让模型并行保留多种关系：每

个头使用独立投影和较小的 d_h，最后拼接回 d_{model}。它提高表示多样性，但也让键值缓存随键值头数增长，这一代价随后促成了 MQA 与 GQA。

2.3.1 为什么 Q、K、V 使用不同投影？

令 $Q = K = V = X$ 仍然可以计算注意力，但模型只能在原始残差空间中用同一组特征同时承担“发起查询、被匹配、传递内容”三种职责。独立的 W_Q, W_K, W_V 允许模型在匹配空间中强调一种特征，在值空间中传递另一种特征。例如，一个头可以用实体类型判断指代关系，却把实体的语义内容作为值返回。

查询/键维度 d_k 决定内积分数的尺度，因此缩放项使用 $\sqrt{d_k}$；值维度 d_v 决定每个头输出的宽度。数学上 d_k 不必等于 d_v，只是很多实现为了规则的张量布局和高效内核令二者都等于 d_h。

多头输出先得到

$$Y^{(h)} \in \mathbb{R}^{B\times T\times d_v}, \qquad h = 1, \dots, H,$$

再沿特征维拼接并经过输出投影：

$$Y = \text{Concat}(Y^{(1)}, \dots, Y^{(H)})W_O, \qquad W_O \in \mathbb{R}^{Hd_v\times d_{\text{model}}}.$$

W_O 不只是为“凑形状”：它让不同头的特征重新线性混合，并把结果投影回残差流宽度。没有 W_O 时，各头输出只能机械并排，下一条残差相加还要求恰好满足 $Hd_v = d_{\text{model}}$。

面试深挖：为什么不能直接令 $Q = K = V = X$？

20 秒回答：这样仍能形成注意力，但会把“用什么特征匹配”和“匹配后传递什么内容”绑在同一空间。独立投影让查询、键和值形成不同的可学习子空间，模型可以用一种特征决定相关性，再传递另一种特征。

深入解释：分数由 $(x_iW_Q)(x_jW_K)^\top$ 决定，输出内容由 x_jW_V 决定。三组矩阵使双线性匹配矩阵 $W_QW_K^\top$ 与内容变换 W_V 独立可学。若直接使用 X，匹配度固定为 $x_ix_j^\top$，模型失去按头改变相似度度量和内容编码的能力。

面试官继续追问：

1. d_k 必须等于 d_v 吗？
2. 为什么分数缩放看 d_k 而不是 d_v？
3. 为什么值向量不参与相似度计算？
4. RoPE 为什么通常只作用于 Q/K？

面试深挖：多头数量越多一定越好吗？

20 秒回答：不一定。固定 d_{model} 时，头数 H 增加会使单头维度 $d_h = d_{\text{model}}/H$ 变小；关系子空间更多，但每个头的表达宽度下降，内核效率、训练稳定性和键值缓存布局也会变化。

深入解释：多头提供多组独立的投影与权重图，不是重复计算同一注意力。但当 d_h 过小时，单头难以承载复杂特征；头数过多还会增加调度开销。输出投影 W_O 负责混合各头，否则头之间直到下一层前都没有显式的信息融合。

面试官继续追问：

1. $d_{\text{model}} = 4096, H = 32$ 时单头维度是多少?
2. 增加 Q 头数是否一定增加键值缓存?
3. GQA 为什么能保持 Q 头数而减少 KV 头数?

关键推导

忽略常数后，QKV 与输出投影的计算量为 $O(BTd_{\text{model}}^2)$；分数矩阵和加权求和各为

$$O(BHT^2d_h) = O(BT^2d_{\text{model}}).$$

标准实现还要保存 $A \in \mathbb{R}^{B\times H\times T\times T}$，其激活显存是 $O(BHT^2)$。这就是长序列下 FlashAttention、稀疏注意力和滑动窗口有价值的根源。

2.3.2 对应实现：Scaled Dot-Product Attention

下面的实现刻意返回注意力权重，便于在面试中检查张量形状（shape）。生产环境通常优先使用 PyTorch 的融合缩放点积注意力实现，以获得更好的速度和显存效率。

```
import math
import torch

def causal_mask(seq_len, device):
    # 先构造 [T, T] 上三角矩阵；对角线以上代表未来位置。
    # 这里约定 True 表示“禁止关注”，False 表示“可以关注”。
    mask = torch.ones(
        seq_len, seq_len, dtype=torch.bool, device=device
    ).triu(diagonal=1)

    # 增加 batch 和 head 维，得到 [1, 1, T, T]。
    # 后续会自动广播到每个样本和每个注意力头。
    return mask[None, None, :, :]
```

```python
def scaled_dot_product_attention(q, k, v, mask=None):
    # q, k, v: [B, H, T, D]
    # k.transpose(-2, -1): [B, H, D, T]
    scale = 1.0 / math.sqrt(q.size(-1))
    scores = torch.matmul(q, k.transpose(-2, -1)) * scale
    # scores: [B, H, T, T]

    if mask is not None:
        # 被 mask 的位置在 softmax 后应严格接近 0。
        scores = scores.masked_fill(mask, float("-inf"))

    # 半精度下先转 FP32 做 softmax，可降低溢出和下溢风险。
    weights = torch.softmax(scores.float(), dim=-1).to(q.dtype)

    # 每个 query 对所有可见 value 做加权求和。
    output = torch.matmul(weights, v)  # [B, H, T, D]
    return output, weights
```

最容易出错的是遮罩方向：第 i 行对应第 i 个查询，第 j 列对应第 j 个键，因此应屏蔽 $j > i$ 的右上三角，而不是左下三角。$[1, 1, T, T]$ 中的两个长度为 1 的维度利用广播机制（broadcasting），把同一个因果约束应用到全部批次样本和注意力头。

2.4 因果遮罩、残差与归一化

提出动机：让注意力既守因果约束，又能堆得足够深

裸 attention 有两个问题。第一，训练时整段序列同时输入，若不遮挡未来位置，模型会偷看答案；第二，几十到上百层子网络连续变换会让激活尺度和梯度路径越来越不稳定。因果遮罩（causal mask）解决信息泄漏，残差连接提供恒等梯度通路，归一化则控制每层输入尺度。

因果遮罩把未来词元的分数置为 $-\infty$，确保第 t 个词元 x_t 只能依赖当前位置及其左侧词元 $x_{\le t}$。它解决的是训练目标的合法性，而不是计算加速：普通实现即使遮掉右上三角，也可能仍构造完整 $T \times T$ 分数矩阵。

残差连接让每个子层只需学习对当前表示的增量，并让梯度有短路径。前置归一化（Pre-Norm）把归一化放在 attention 或前馈网络之前，使残差主干更接近恒等映射，深层训练通常比后置归一化（Post-Norm）稳定。均方根归一化（RMSNorm）则去掉均值中心化，只按均方根缩放，以更少统计量维持尺度稳定。

项目	LayerNorm	RMSNorm
公式	$(x-\mu)/\sqrt{\sigma^2+\epsilon}$	$x/\sqrt{\frac{1}{d}\sum_i x_i^2+\epsilon}$
参数	常有缩放与偏置参数	通常只有缩放参数
效果	通用稳定	更省计算，现代 LLM 常用

表 2.1 LayerNorm 与 RMSNorm 对比。

表中 $x=(x_1,\ldots,x_d)$ 是一个位置上的 d 维隐藏向量，μ 和 σ^2 分别是这些分量的均值与方差，ϵ 是防止分母为零的小常数。LayerNorm 先减去 μ，RMSNorm 则直接用原始分量的均方进行缩放。

2.4.1 残差流为什么保持固定宽度？

可以把 Transformer 看成一条贯穿所有层的残差流：

$$X_{\ell+1}=X_\ell+F_\ell(X_\ell),\qquad X_\ell\in\mathbb{R}^{B\times T\times d_{\text{model}}}.$$

每个子层可以在内部扩张、分头或门控，但最终必须返回 d_{model} 维，才能与 X_ℓ 逐元素相加。固定宽度让不同层共享统一接口，也避免整条网络永久工作在更昂贵的 d_{ff} 宽度。若子层输出宽度不同，必须额外使用投影或改变残差支路；标准 Transformer 选择在子层内部完成投影回收。

残差连接的作用也不只是“防止梯度消失”。对 $X_{\ell+1}=X_\ell+F_\ell(X_\ell)$ 求导：

$$\frac{\partial X_{\ell+1}}{\partial X_\ell}=I+J_{F_\ell}(X_\ell),$$

其中 I 是恒等映射，J_{F_ℓ} 是子层的雅可比矩阵。梯度有一条不依赖子层参数的直接路径；同时每层只需学习对已有表示的增量，使深层优化更接近连续的小步修正。它不能保证梯度永不爆炸或消失，归一化、初始化和残差缩放仍然重要。

面试深挖：为什么 attention 和 FFN 都必须回到 d_{model}？

20 秒回答： 它们都要把输出加回同一条残差流。子层内部可以使用多头或 d_{ff} 宽空间，但输出必须投影到 d_{model}，才能逐元素残差相加并保持各层接口与总体成本稳定。

深入解释： 固定残差宽度把“模型的长期状态容量”和“单个子层的临时计算容量”分开。attention 用 W_O 回收各头，FFN 用下投影回收中间特征。若全网永久保持 d_{ff}，attention 投影、激活和每层参数都会按更大宽度增长，成本远高于只在 FFN 内短暂扩张。

面试官继续追问：

1. 如果子层输出维度不同，能否给 shortcut 加投影？

2. Pre-Norm 为什么让恒等路径更干净?

3. 残差连接能否单独保证超深网络稳定?

2.5 前馈网络（FFN）与 SwiGLU

提出动机：注意力让词元交换信息，谁来做逐位置非线性变换？

Attention 的独特作用是跨序列位置通信：注意力权重让一个位置聚合其他可见位置的信息。但它并非只做词元混合，因为 W_Q, W_K, W_V, W_O 也会对特征通道做线性变换。前馈网络（feed-forward network，FFN）不交换不同词元的信息，而是为每个位置独立提供更强的高维非线性计算。SwiGLU 又在普通 FFN 上引入数据相关门控，让网络决定哪些中间特征应通过。

2.5.1 为什么先升维再降维？

标准 FFN 对每个词元独立执行

$$h = \phi(xW_1 + b_1), \qquad W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}},$$

$$y = hW_2 + b_2, \qquad W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \qquad d_{\text{ff}} > d_{\text{model}}.$$

常说“attention 做词元混合、FFN 做通道混合”，它适合作为第一层直觉，却不是严格的数学分解。更准确地说：attention 是 Transformer 中实现**跨词元通信**的组件，注意力权重决定不同位置如何聚合；与此同时，Q/K/V/O 投影也会混合特征通道。FFN 则是**逐位置**的，它不跨词元交换信息，而是在每个词元内部执行更强的非线性特征变换。上投影提供更多中间特征方向，激活函数或门控在这些方向上选择性地产生非线性响应，下投影再把丰富的临时特征压回固定残差宽度。

容易答错

错误说法： Attention 只负责 token mixing，FFN 只负责 channel mixing。
更准确： Attention 的独特能力是跨 token 通信，但 Q/K/V/O 线性投影本身也会混合特征维度；FFN 则不跨 token，而是在每个 token 内执行更强的非线性特征变换。

“高维一定更强”并不是完整答案。关键是扩张只发生在局部子层：模型获得较大的非线性特征基，却不要求 attention、残差流和所有层间接口永久承担 d_{ff} 的成本。若只做 $d_{\text{model}} \to d_{\text{model}}$，参数更省但中间可激活的特征基更少；若永远保持 d_{ff}，整网参数与矩阵乘法成本都会显著上升。

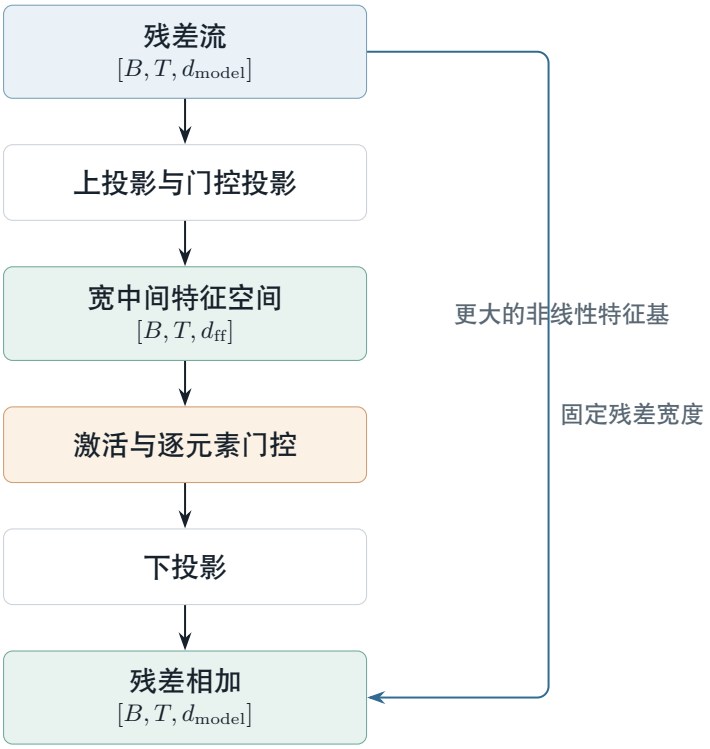


图 2.3 FFN 的扩张–变换–收缩路径。中间层提供较大的非线性容量，输出回到固定的残差流宽度。

若去掉非线性激活，两层线性层会塌缩成一层：

$$y = (xW_1)W_2 = x(W_1W_2).$$

W_1W_2 仍是一个线性变换，升维没有带来新的函数类别。真正让 FFN 能形成分段或平滑非线性特征的，是中间的激活函数与门控。

面试深挖：为什么 FFN 要先升维再降维？

20 秒回答： 残差流必须维持固定 d_{model}，而 FFN 需要更大的中间特征空间完成逐词元的非线性变换，因此先扩张到 d_{ff}，在宽空间中激活或门控，再投影回 d_{model}。这样提高局部非线性容量，又不让整个网络永久承担 d_{ff} 的成本。

深入解释： 扩张层学习大量候选特征，非线性使不同输入激活不同子集，下投影把它们重新组合成对残差流的增量。d_{ff} 增大通常会提升容量，但参数和 FLOPs 近似线性增长，也可能受数据规模、正则化和硬件效率限制，因此不是越大越好。

面试官继续追问：

1. 为什么不能只做 $d_{\text{model}} \to d_{\text{model}}$？
2. 如果没有激活函数，两层 linear 会发生什么？
3. 为什么升维后必须降回来？

4. 为什么 MoE 通常替换 FFN 而不是 attention?

2.5.2 SwiGLU 的两条上投影

SwiGLU 用门控改善表达能力:

$$\mathrm{SwiGLU}(x) = (\mathrm{SiLU}(xW_g) \odot xW_u)\, W_d.$$

设输入 x 的最后一维为 d_{model}，门控中间维度为 d_{ff}，则 $W_g, W_u \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$，$W_d \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$；$\odot$ 表示逐元素相乘。门控投影 W_g 决定哪些中间特征通过，上投影 W_u 提供被门控的内容，下投影 W_d 把结果送回残差流。

SiLU 定义为 $\mathrm{SiLU}(u) = u\sigma(u)$。它不是纯粹限制在 $[0,1]$ 的 sigmoid 门，而是平滑、可带小负值且幅度随输入变化的门控激活。SwiGLU 有三块大矩阵，若与普通两矩阵 FFN 使用同一 d_{ff}，参数会多约 50%。为保持相近参数预算，实践中常把 SwiGLU 的中间维调到约普通 $4d_{\text{model}}$ FFN 的 2/3，再对齐到硬件友好的整数倍。

计算题：估算一个 SwiGLU 层的参数量

设 $d_{\text{model}} = 4096$、$d_{\text{ff}} = 11008$，忽略偏置。三块矩阵共有

$$3d_{\text{model}}d_{\text{ff}} = 3 \times 4096 \times 11008 = 135{,}266{,}304$$

个参数，约 1.35 亿。普通两矩阵 FFN 在相同中间维下约 0.90 亿参数。SwiGLU 的中间维通常需要重新调整，不能只说“换激活函数而参数不变”。

面试深挖：SwiGLU 为什么需要两条上投影？

20 秒回答： W_g 产生经过 SiLU 的门控信号，W_u 产生内容特征，两者逐元素相乘后，模型可以按输入动态控制每个中间特征的通过量；W_d 再把门控后的宽特征投影回残差流。

深入解释： 普通 FFN 只对一组上投影特征应用固定形式的激活。SwiGLU 把“是否通过”和“传递什么”解耦成两组可学习投影。代价是三块权重矩阵，因此比较架构时必须同时比较中间维和总参数，而不能只比较激活名称。

面试官继续追问：

1. gate projection 与 up projection 分别做什么?
2. 为什么门不直接用 sigmoid?
3. 为什么最终仍需 down projection?
4. 如何调整 d_{ff} 才能与普通 FFN 参数量近似?

容易答错

错误说法： MoE 替换 FFN，只是因为 FFN 参数多。
更准确： 参数多是收益来源之一，更关键的是 FFN 对每个词元独立计算，天然适合先按词元路由、再在专家上批量执行；attention 需要跨词元构造匹配关系，若把它按专家稀疏路由，会同时改变信息可见性和通信结构，远比替换逐位置 FFN 复杂。

2.5.3 对应实现：SwiGLU

```python
import torch.nn as nn
import torch.nn.functional as F

class SwiGLU(nn.Module):
    def __init__(self, dim, hidden_dim):
        super().__init__()
        # gate_proj 产生门控分支，up_proj 产生内容分支。
        self.gate_proj = nn.Linear(dim, hidden_dim, bias=False)
        self.up_proj = nn.Linear(dim, hidden_dim, bias=False)

        # down_proj 把中间维度还原为模型维度，供残差相加。
        self.down_proj = nn.Linear(hidden_dim, dim, bias=False)

    def forward(self, x):
        # x: [B, T, C]
        gate = F.silu(self.gate_proj(x))  # [B, T, hidden_dim]
        value = self.up_proj(x)           # [B, T, hidden_dim]

        # 逐元素门控，而不是矩阵乘法。
        hidden = gate * value
        return self.down_proj(hidden)     # [B, T, C]
```

2.5.4 对应实现：最小前置归一化 Transformer 块

下面把前面的 attention、归一化、SwiGLU 和残差连接串起来。它省略了随机失活（dropout）、RoPE、GQA 与推理缓存，目的是把一层中的数据流说清楚。

```
class TransformerBlock(nn.Module):
    def __init__(self, dim, num_heads, hidden_dim):
        super().__init__()
        assert dim % num_heads == 0

        self.num_heads = num_heads
        self.head_dim = dim // num_heads
        self.attn_norm = nn.RMSNorm(dim)
        self.ffn_norm = nn.RMSNorm(dim)

        # 一次投影后切成 Q、K、V，减少代码重复。
        self.qkv_proj = nn.Linear(dim, 3 * dim, bias=False)
        self.out_proj = nn.Linear(dim, dim, bias=False)
        self.ffn = SwiGLU(dim, hidden_dim)

    def forward(self, x):
        # x: [B, T, C]
        batch, seq_len, dim = x.shape

        # Pre-Norm：先归一化，再进入注意力子层。
        q, k, v = self.qkv_proj(self.attn_norm(x)).chunk(3, dim=-1)

        # [B, T, C] -> [B, H, T, D]
        def split_heads(tensor):
            tensor = tensor.view(
                batch, seq_len, self.num_heads, self.head_dim
            )
            return tensor.transpose(1, 2)

        q, k, v = map(split_heads, (q, k, v))
        mask = causal_mask(seq_len, x.device)
        attn_out, _ = scaled_dot_product_attention(q, k, v, mask)

        # transpose 后内存通常不连续；contiguous 后才能安全 view。
        attn_out = attn_out.transpose(1, 2).contiguous()
        attn_out = attn_out.view(batch, seq_len, dim)

        # 第一条残差：原输入 + 注意力输出。
        x = x + self.out_proj(attn_out)

        # 第二条残差：注意力结果 + FFN 输出。
        return x + self.ffn(self.ffn_norm(x))
```

从梯度角度看，每个子层都写成 $x + F(\mathrm{Norm}(x))$。其中 F 表示注意力或前馈子层，Norm 表示归一化操作。即使 F 在训练早期不稳定，梯度仍可沿恒等残差路径传播；这正是前置归一化深层网络较容易训练的核心原因。

至此，基础 Transformer 已经回答了“如何表示顺序、如何跨位置交换信息、如何稳定堆叠和如何逐位置变换”四个问题。后续现代结构并不是推翻这些组件，而是继续处理它们在更大模型和更长上下文下暴露出的计算、显存与稳定性瓶颈。

常见误区

不要把多头注意力理解为“重复算多次一样的注意力”。不同头有不同投影矩阵，学习不同子空间；但键值缓存显存也随键值头数增长。

面试高频问题

Q: 为什么 attention 要除以 $\sqrt{d_k}$？

20 秒回答：内积方差随 d_k 增长，softmax 会过早饱和；除以 $\sqrt{d_k}$ 把分数尺度稳定在可训练范围。

Q: 前置归一化为什么更适合深层 Transformer？

20 秒回答：归一化在子层前，残差路径更接近恒等映射，梯度更容易穿过很多层，训练稳定性通常优于后置归一化。

第二部分

预训练

第三章　自回归语言建模

提出动机：如何从海量无标注文本获得监督信号？

人工为互联网规模文本逐条标注任务类别或答案不可行，但每段自然文本都天然提供了“前文”和“下一个词元”。自回归语言建模把序列向右移动一位就得到监督目标，使模型可以从原始文本自监督学习。它解决了监督数据规模问题，却也把模型能力绑定到数据分布和“预测下一个词元”的目标之上。

LLM 预训练的核心目标是最大化语料序列概率。对词元序列 $x_{1:T} = (x_1, \dots, x_T)$，$x_t$ 表示第 t 个词元，T 表示序列长度。链式法则给出：

$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

模型在位置 t 输出未归一化的词表分数（logits）$z_t \in \mathbb{R}^V$，其中 V 是词表大小，$z_{t,i}$ 是候选词元 i 的分数。softmax 后得到下一个词元的概率分布：

$$p_\theta(x_t = i \mid x_{<t}) = \frac{\exp z_{t,i}}{\sum_{j=1}^{V} \exp z_{t,j}}.$$

其中 θ 表示模型的全部可训练参数，$x_{<t}$ 表示位置 t 之前的词元。最大似然等价于最小化负对数似然：

$$\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

在一个批次中，若 $m_{b,t} \in \{0, 1\}$ 表示第 b 个样本的第 t 个位置是否参与监督，则实际使用的词元平均损失是

$$\mathcal{L}_{\text{token}} = -\frac{1}{N_{\text{valid}}} \sum_{b=1}^{B} \sum_{t=1}^{T-1} m_{b,t} \log p_\theta(x_{b,t+1} \mid x_{b,\le t}), \qquad N_{\text{valid}} = \sum_{b,t} m_{b,t}.$$

这里 B 是批次大小，N_{valid} 是整个批次中参与监督的有效位置总数。补齐位置（padding）、跨样本边界和监督微调中的提示词位置，都可以通过令 $m_{b,t} = 0$ 排除。

具体例子：一句话如何产生多条训练标签

句子“我喜欢机器学习”加上序列开始和结束词元后可写成

$$[\text{BOS},\ \text{我},\ \text{喜欢},\ \text{机器},\ \text{学习},\ \text{EOS}].$$

同一序列向右错开一位，就得到五个“预测下一个词元”的监督信号：

模型可见前缀	应预测的目标标签
[BOS]	“我”
[BOS, 我]	“喜欢”
[BOS, 我, 喜欢]	“机器”
[. . . , 机器]	“学习”
[. . . , 学习]	EOS

训练代码不需要真的运行五次模型。因果遮罩让一次并行前向中的每个位置只能看到左侧前缀，随后用 `logits[:, :-1]` 与 `input_ids[:, 1:]` 一次对齐全部五个预测任务。

关键推导

设真实数据分布为 p_{data}，参数为 θ 的模型分布为 p_θ。交叉熵

$$H(p_{\text{data}}, p_\theta) = H(p_{\text{data}}) + D_{\text{KL}}(p_{\text{data}} \| p_\theta).$$

其中 $H(p_{\text{data}})$ 是数据自身的熵，D_{KL} 是 Kullback–Leibler 散度。数据熵与模型参数无关，因此最小化交叉熵等价于最小化真实分布到模型分布的 KL 散度。

关键推导

对单个位置，令真实词元类别的编号为 y，候选类别 i 的模型分数为 z_i，交叉熵

$$\ell = -\log \frac{e^{z_y}}{\sum_j e^{z_j}} = -z_y + \log \sum_j e^{z_j}.$$

直接求导得到

$$\frac{\partial \ell}{\partial z_i} = -\mathbf{1}[i = y] + \frac{e^{z_i}}{\sum_j e^{z_j}} = p_i - \mathbf{1}[i = y].$$

其中 p_i 是 softmax 后候选类别 i 的概率，$\mathbf{1}[i = y]$ 是指示函数。所以梯度会降低真实词元的“概率缺口”，同时压低其他词元。困惑度（perplexity，PPL）定

义为 $\mathrm{PPL} = \exp(\mathcal{L}_{\text{token}})$；平均损失每下降 Δ，PPL 就乘以 $e^{-\Delta}$，这里 $\Delta > 0$ 表示下降量。

3.1 PyTorch 中的预测分数与目标标签

公式写成 $p(x_t \mid x_{<t})$，而模型输入和输出却具有相同的序列长度。实现中的错位对齐（shift）正是连接二者的步骤：位置 t 的隐藏状态（hidden state）表示前缀 $x_{\le t}$，它应预测 x_{t+1}，不能拿同一位置的输入词元当标签。

```
import torch.nn.functional as F

# input_ids: [B, T]，每个位置保存一个 token id。
input_ids = batch["input_ids"]

# 一次前向并行得到所有位置的预测，logits: [B, T, V]。
# causal mask 保证位置 t 的输出看不到位置 t 之后的 token。
logits = model(input_ids)

# 位置 0..T-2 的输出预测位置 1..T-1 的真实 token。
shift_logits = logits[:, :-1, :].contiguous()
shift_targets = input_ids[:, 1:].contiguous()

# cross_entropy 要求类别维在最后展平后变成 [N, V]。
loss = F.cross_entropy(
    shift_logits.view(-1, shift_logits.size(-1)),
    shift_targets.view(-1),
)
```

模型取 `logits[:, :-1]` 作为输入位置 $0 \ldots T-2$ 给出的预测分数，取 `input_ids[:, 1:]` 作为真实的下一个词元编号。两者各去掉一端，因此监督位置数是 $T-1$，不是 T。若批次含补齐位置，还应把这些位置的目标标签改成 `-100`，并向交叉熵传入 `ignore_index=-100`。

教师强制（teacher forcing）的“并行”容易被误解：模型不是先生成第一个预测再把预测送回去，而是一次看到整段真实序列，并靠因果遮罩保证每个位置只读取真实前缀。因此所有位置可同时训练；代价是推理时模型只能看到自己生成的前缀，错误可能逐步累积，这就是暴露偏差（exposure bias）。

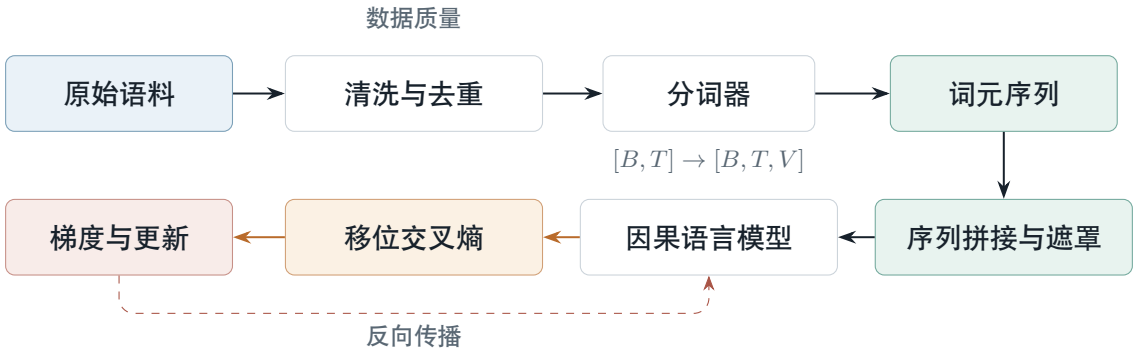


图 3.1 从语料到参数更新的预训练闭环。上行负责数据表示，下行完成批处理、前向计算和梯度更新。

面试深挖：为什么“预测下一个词元”能学到通用能力？

20 秒回答： 下一个词元看似是局部标签，但准确预测它往往需要压缩语法、事实、语义关系、代码结构和推理模式。目标函数没有逐项命名这些能力，它们是模型在大规模、多样数据上降低条件熵时形成的可迁移内部表示。

深入解释： 模型优化的是 $-\log p_\theta(x_t \mid x_{<t})$。当同一前缀允许多种合理续写时，模型需要估计的是整个条件分布，而不只是背诵一个答案。更低的训练损失只说明更贴合训练分布；能否迁移还取决于数据覆盖、容量、优化和评测分布，因此不能把“下一个词元目标”直接等同于“必然产生推理能力”。

面试官继续追问： 1. 这种自监督信号为什么可以扩展到互联网规模？2. 数据分布会怎样限制模型能力？3. 训练损失更低为什么不保证下游体验更好？4. 与掩码语言模型相比，因果语言模型的生成优势是什么？

面试深挖：教师强制为什么能并行训练，却产生暴露偏差？

20 秒回答： 训练时每个位置都读取真实前缀，并借助因果遮罩一次并行预测全部下一词元；推理时真实后续不可用，只能读取模型自己生成的前缀。训练与推理前缀分布不同，因此早期错误可能在解码中继续传播。

深入解释： 并行来自张量计算，不是来自取消自回归约束。对位置 t，注意力遮罩仍保证它只能读取 $x_{\le t}$，只是不同 t 的计算可以同时完成。若训练时改用模型逐步采样的前缀，成本显著上升且目标更难稳定；因此后训练和序列级优化通常用于补充，而不是简单替代教师强制。

面试官继续追问： 1. 为什么不能把同一位置的输入词元直接当标签？2. 暴露偏差一定会导致生成失败吗？3. 序列级强化学习与词元级交叉熵分别优化什么？

3.2 预训练流水线的面试拆解

提出动机：正确目标为什么仍可能训练出差模型？

损失函数只规定模型怎样从词元学习，却不保证输入词元本身干净、均衡且没有泄漏。重复网页会被过度记忆，低质量文本会消耗算力，错误的序列拼接会让样本互相泄漏。因此预训练流水线是在梯度进入模型前控制“模型究竟从什么分布学习”。

回答预训练流程题时，可以按“数据到梯度”的顺序展开：先说明原始数据需要清洗、去重和质量过滤，再说明分词器把文本变成词元编号，序列拼接（packing）把多个词元序列组织成高利用率批次，最后因果语言模型输出未归一化词表分数，并用错位后的目标标签计算下一个词元损失。

面试一句话回答

把预训练说清楚的关键是对齐三件事：输入词元的张量形状、目标词元的错位方式，以及损失如何反向更新所有 Transformer 参数。

第四章　数据与优化

4.1　数据混合、去重与序列拼接

提出动机：让有限训练预算花在有效词元上

模型不会判断某条数据是否值得学习，只会按出现频率更新参数。未经处理的网络语料常包含重复页面、模板垃圾、语言比例失衡和评测集（benchmark）泄漏。数据混合、去重和质量过滤的目标，是在固定词元与算力预算下提高每次梯度更新的信息密度。

数据配比决定模型能力边界。代码、数学、百科、对话、多语言数据应按目标岗位需求讨论。去重减少对同一内容的重复记忆，也降低训练集与评测集泄漏；质量过滤比单纯扩大词元数更重要，因为低质量词元同样会占用完整计算。

短样本若分别补齐（padding）到固定长度，大量算力会花在无意义的补齐位置上。序列拼接（sequence packing）因此把多个短样本拼入固定长度块以提高 GPU 利用率。它解决利用率问题，却引入样本边界问题：必须用注意力遮罩阻止不同样本之间互相读取，否则模型会学习不存在的跨样本上下文。

4.2　优化细节

提出动机：让大规模优化在数值、显存和训练阶段上都可控

大模型训练初期统计量尚未稳定，过大学习率容易直接发散；训练后期继续使用高学习率又会围绕较优区域震荡。同时，完整批次和 FP32 状态往往超出显存。学习率预热、衰减、混合精度、梯度裁剪与梯度累积，分别处理启动稳定性、后期收敛、数值成本、异常梯度和批次显存问题。

Adam 的自适应缩放会让“把 L2 正则直接加到梯度”也被逐坐标缩放，使实际衰减强度难以解释。AdamW 因此将权重衰减从自适应梯度中解耦，让参数收缩直接由 $\eta_t\lambda$ 控制。学习率预热（warmup）避免训练初期参数和动量统计尚不可靠时迈步过大，余弦衰减（cosine decay）则在后期逐步减小更新幅度，帮助模型稳定收敛。

混合精度（mixed precision）用低精度完成主要矩阵计算，以减少显存并提高张量计算核心（Tensor Core）的吞吐，但需要关注 FP16 溢出、损失缩放（loss scaling）与关键统计量精度。梯度裁剪（gradient clipping）在偶发梯度尖峰时限

制更新范数；梯度累积（gradient accumulation）则把一个逻辑大批次拆成多个可放入显存的微批次（micro-batch）。它们解决的问题不同，不能互相替代。

设 $g_t = \nabla_\theta \mathcal{L}_t$，AdamW 的核心更新为

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta_t \lambda \theta_t.$$

最后一项直接作用在参数上，不会被 $1/(\sqrt{\hat{v}_t}+\epsilon)$ 按坐标缩放，这就是 AdamW 与“把 L2 项加进 Adam 梯度”的关键区别。

式中，t 是优化步编号；θ_t 是第 t 步的模型参数；$g_t = \nabla_\theta \mathcal{L}_t$ 是当前损失对参数的梯度；m_t 和 v_t 分别是梯度的一阶动量与平方梯度的二阶动量。$\beta_1, \beta_2 \in [0, 1)$ 控制历史信息的衰减速度，$\hat{m}_t, \hat{v}_t$ 是经过初始偏差修正的动量，η_t 是当前学习率，λ 是权重衰减系数，ϵ 是防止除零的小常数。

```
# 累积开始前清空旧梯度；set_to_none=True 通常更省显存。
optimizer.zero_grad(set_to_none=True)

for micro_x in micro_batches:
    # micro_x: [b, T]，b 是能放入单卡的 micro-batch 大小。
    logits = model(micro_x)

    # 除以累积步数，使最终梯度近似大 batch 的平均梯度。
    # 若不除，梯度会被额外放大 grad_accum_steps 倍。
    loss = lm_loss(logits, micro_x) / grad_accum_steps
    loss.backward()

# 必须在所有 micro-batch 反向完成后再裁剪和更新。
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

梯度累积只减少单次前向需要保存的激活，不会减少模型参数、梯度和优化器状态本身的显存。它也不等于数据并行：累积是在同一组参数上串行处理多个微批次，数据并行则在多张卡上同时计算并通过通信聚合梯度。分布式训练中常配合 `no_sync()`，避免每个微步都执行全归约通信（AllReduce）。

容易答错

容易答错：“梯度累积把一个大批次拆开，所以训练显存都会按累积步数下降。”

更准确：它主要降低每个微步的激活峰值；参数、梯度与优化器状态仍需驻留。若吞吐、归一化方式和有效词元权重处理不当，累积后的更新也未必与一次性大批次完全相同。

面试深挖：AdamW、预热和梯度裁剪分别在解决什么？

20 秒回答： AdamW 让权重衰减不再被自适应预条件器扭曲；预热限制训练初期不可靠统计量下的步长；梯度裁剪只处理偶发的异常大更新。三者作用在不同环节，不能互相替代。

深入解释： AdamW 的衰减项是 $-\eta_t \lambda \theta_t$，而不是把 $\lambda\theta$ 混进梯度后再除以 $\sqrt{\hat{v}_t}$。预热改变一段时间内的 η_t；裁剪改变当前梯度的范数。模型发散时需要结合损失缩放、数据异常、学习率、梯度范数和通信错误定位，不能把所有问题都归因于“缺少 warmup”。

面试官继续追问： 1. AdamW 的 ϵ 是正则项吗？2. 混合精度下为什么还可能保留高精度主权重？3. 梯度累积是否会减少优化器状态？4. 有效词元数不同的微批次怎样正确加权？

具体例子：为什么累积时要除以步数

设两个微批次对某参数产生的梯度分别为 $g_1 = (2, 0)$、$g_2 = (0, 4)$。逻辑大批次的平均梯度应为

$$\frac{g_1 + g_2}{2} = (1, 2).$$

因此每个微批次的损失先除以 2 再反向传播，累积结果才是 $(1, 2)$。若不除，得到的是和 $(2, 4)$，等价于把学习率额外放大两倍。若各微批次的有效词元数不同，则更严谨的做法是按总有效词元数加权，而不是机械地平均每个微批次。

4.3 Scaling Laws 与 Compute-Optimal Training

提出动机：固定预算应该买更多参数，还是训练更多 token？

扩大模型并不等于只增加参数量。基础模型的最终质量同时受参数量、训练数据量、数据质量和计算预算约束：模型过大但训练不足，容量来不及被数据塑造；模型过小却反复吞入大量数据，又可能把计算花在一个容量受限的模型上。Scaling Laws 的价值，是把“越大越好”改写成可通过实验拟合和预算约束讨论的问题。

令 N 表示模型参数量，D 表示训练词元数，C 表示训练计算量，L 表示验证损失。一类常见的经验形式是

$$L(N, D) \approx L_\infty + AN^{-\alpha} + BD^{-\beta},$$

其中 L_∞ 是给定数据分布下难以继续降低的损失项，$A, B > 0$ 是尺度系数，$\alpha, \beta > 0$ 描述参数和数据增加时的收益递减速度。这个式子表达的是：固定其他条件时，增大 N 或 D 往往能按幂律趋势降低损失，但边际收益逐渐减小。

它是经验规律，不是数学定理。系数和指数来自特定模型族、数据分布、tokenizer、优化器、训练时长和评测口径下的拟合。换数据清洗流程、模型结构或训练目标后，曲线可能改变；因此不能把某组指数当作跨项目不变的自然常数。

对稠密 Transformer，粗略预算常写成

$$C_{\text{train}} \approx \kappa N D,$$

其中 κ 汇总一次 token 训练所需的前向、反向与参数相关计算常数。它适合做数量级推理，却会忽略序列长度带来的注意力项、词表投影、稀疏结构、硬件利用率和通信开销。固定预算 C_0 下，问题可以概念化为

$$\min_{N,D} \; L(N,D) \qquad \text{s.t.} \quad C_{\text{train}}(N,D) \le C_0.$$

4.3.1 Compute-optimal：在约束下平衡容量与数据

计算最优训练（compute-optimal training）不是寻找全世界唯一的最佳模型，而是在给定计算预算、模型族和数据条件下，协调模型规模 N 与训练词元数 D，使预算带来的损失下降尽可能大。

若 N 很大而 D 太少，模型是**欠训练的（under-trained）**：大量参数只看到有限训练信号，在同样 FLOPs 下，缩小模型并训练更多高质量 token 可能更有效。反过来，若 D 很大而 N 太小，模型容量可能成为瓶颈；继续增加 token 的边际收益很低，把一部分预算用于扩大模型可能更合理。这里的“数据更多”还必须区分新鲜、高质量、多样的数据与低质量重复数据，二者不能只按 token 数等价计算。

面试深挖：什么是 Scaling Law？它是数学定理吗？

20 秒回答：Scaling Law 是从一组受控训练实验中拟合出的经验关系，用来描述损失如何随参数量、训练 token 和计算量变化。它常呈幂律和收益递减趋势，但系数、指数依赖模型与数据条件，不是可以机械外推的普适定理。
深入解释：它的工程价值不在背某个指数，而在于用小规模试验拟合趋势，再比较等计算量方案。若训练分布、架构或数据质量改变，原曲线可能失效；超出拟合范围的外推也会放大误差。Scaling Law 通常预测的是某种损失或指标趋势，并不自动等价于指令遵循、事实可靠性或部署体验。
面试官继续追问： 1. 为什么幂律中的指数不能跨模型族照搬？2. 验证损失下降是否保证下游体验更好？3. 怎样用一组小模型实验规划大训练任务？

4.3.2 Chinchilla-style 直觉：不要只放大参数

Chinchilla-style 分析关注的问题是：在近似固定的训练计算量下，参数和训练 token 应怎样共同增长。Hoffmann 等人的受控实验表明，在其研究设置和范围内，许多大模型相对数据量过大、训练 token 不足；把计算分配给更小但训练更充分的模型，可以得到更低损失。

面试时不应把这段结论压缩成一个永远成立的“每参数固定多少 token”。后续模型的数据配比、重复训练、长上下文阶段、合成数据、稀疏结构和部署约束都可能改变最优点。真正应掌握的是：**固定 FLOPs 时，N 与 D 互相竞争预算；最优比例应由当前训练条件下的曲线与产品目标决定**。

面试深挖：什么叫 compute-optimal training？

20 秒回答：它是在给定训练计算预算下平衡模型参数量与训练 token 数，使模型既有足够容量，又不会因为数据不足而欠训练。目标不是单独最大化参数量或数据量，而是让每一部分计算产生更高的质量收益。

深入解释：在 $C \approx \kappa ND$ 的粗略约束下，增大 N 会压缩可承担的 D，反之亦然。实际决策还要加入数据质量、硬件效率、训练稳定性和推理成本：训练损失近似最优的方案，未必是延迟、显存或服务成本最优的产品方案。

面试官继续追问： 1. 什么现象说明模型可能 under-trained？2. 数据无限增加时，小模型能否无限获益？3. 训练最优与部署最优为什么可能不同？4. MoE 模型还能直接使用 $C \propto ND$ 吗？

具体例子：数值直觉：相近计算量的两种训练计划

考虑同一稠密模型族的两个假设方案：A 使用 $N = 13$B 参数、训练 $D = 100$B token；B 使用 $N = 7$B 参数、训练 $D = 200$B token。若只用 $C \propto ND$ 粗估，则两者分别约为 1.3×10^{21} 与 1.4×10^{21} 的参数–token 乘积，训练计算量处在相近数量级。

仅凭这两个乘积**不能断言谁更好**。还需要同模型族的等计算量实验或拟合曲线、数据质量与重复率、优化是否充分、目标任务，以及推理显存和延迟约束。这个练习考查的不是背诵固定比例，而是能否把模型容量、token 数、计算和数据质量放在同一决策框架中。

容易答错

容易答错：“Chinchilla 给出了固定的参数/token 比例，新模型都按这个比例训练就是 compute-optimal。”

更准确：论文结论来自特定模型族、数据和计算范围。它提供了平衡 N 与 D 的方法和证据，而不是跨架构、跨数据管线永久不变的配方；新项目仍需用自己的试验、质量目标和部署约束校准。

完成优化以后，下一类瓶颈不再是“能否把模型训练起来”，而是基础结构在长上下文和逐词元推理时是否足够高效。RMSNorm、RoPE、GQA、键值缓存与 FlashAttention 都可以从这一问题继续推导出来。

阶段	目标	主要瓶颈	典型技巧
训练	更新全部参数	计算、通信、显存	DP/TP/PP、ZeRO、混合精度
提示词预填充（prefill）	处理完整提示词	大矩阵计算	FlashAttention、批处理
逐词元解码（decode）	每步生成一个词元	键值读取与带宽	GQA/MQA、分页缓存、量化

表 4.1 训练、提示词预填充与逐词元解码的差异。

面试高频问题

Q: 教师强制的利弊是什么？

20 秒回答：它让训练可并行且监督稳定，但训练时总看真实前缀，推理时看模型生成前缀，两者存在暴露偏差。

Q: 为什么要做序列拼接？

20 秒回答：短样本直接补齐会浪费计算；序列拼接提高词元利用率，但必须正确构造跨样本遮罩。

第三部分

现代 Transformer 改进

第五章　位置、归一化与前馈网络

5.1　RMSNorm

提出动机：更低成本地稳定残差流尺度

层归一化（LayerNorm）同时计算均值和方差，在每层、每个词元上都要执行。现代仅解码器（decoder-only）模型层数深、调用归一化频繁，因此希望保留“控制向量尺度”这一主要收益，同时减少中心化计算和参数。RMSNorm 正是对这一需求的简化：只按均方根缩放，不再减均值。

RMSNorm 对向量 $x \in \mathbb{R}^d$ 做：

$$\mathrm{RMSNorm}(x) = g \odot \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}},$$

其中 $x = (x_1, \ldots, x_d)$ 是待归一化向量，d 是特征维数，i 是特征下标；$g \in \mathbb{R}^d$ 是逐维可学习缩放向量，$\odot$ 表示逐元素相乘，ϵ 是防止除零的小常数。它不减均值，计算更省，在仅解码器 LLM 中非常常见。

具体例子：手算一个二维 RMSNorm

取 $x = (3, 4)$、$g = (1, 1)$，暂时忽略很小的 ϵ。均方根为

$$\mathrm{RMS}(x) = \sqrt{\frac{3^2 + 4^2}{2}} = \sqrt{12.5} \approx 3.536.$$

因此归一化结果约为

$$\frac{x}{\mathrm{RMS}(x)} = (0.849, 1.131).$$

它没有先减均值，所以两个分量仍为正，方向比例 $3:4$ 也保持不变；它只把整体尺度拉回稳定范围。可学习的 g 随后允许模型重新放大或缩小每个维度。

“不减均值”意味着 RMSNorm 不强制特征以 0 为中心，只控制向量的整体能量。分母中的均方根可理解为向量长度的按维归一化版本；除以它以后，向量方向基本保留，而尺度被稳定到可控范围。ϵ 不是训练超参数意义上的正则项，它只是防止全零或极小输入导致除零和数值爆炸。

5.1.1 对应实现：RMSNorm

```python
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        # 每个 hidden 维度都有独立的可学习缩放系数。
        self.weight = nn.Parameter(torch.ones(dim))
        self.eps = eps

    def forward(self, x):
        # x: [..., C]，只沿最后一个 hidden 维归一化。
        input_dtype = x.dtype

        # FP16 平方可能溢出，因此统计量通常用 FP32 计算。
        x_float = x.float()
        mean_square = x_float.pow(2).mean(dim=-1, keepdim=True)

        # rsqrt(a) 等价于 1 / sqrt(a)，但表达更直接。
        inv_rms = torch.rsqrt(mean_square + self.eps)
        normalized = (x_float * inv_rms).to(input_dtype)

        # weight 的 shape 为 [C]，会广播到 batch 和序列维。
        return normalized * self.weight
```

RMSNorm 与 LayerNorm 最容易混淆之处是统计量：LayerNorm 使用 $x-\mu$ 的方差，RMSNorm 使用原始 x 的均方。两者都不会在 batch 维混合样本，因此不同于 BatchNorm。

对输入 $X \in \mathbb{R}^{B\times T\times d}$，LayerNorm 与 RMSNorm 通常都沿最后的隐藏维 d 独立计算统计量。不同批次样本、不同序列位置不会互相参与均值或均方计算。LayerNorm 的减均值使归一化结果显式零中心；RMSNorm 不保证均值为 0，它保留方向关系，只约束整体均方根尺度。

Transformer 不主要使用 BatchNorm，是因为 BatchNorm 依赖批次统计量：小批次、变长序列、补齐位置和训练/推理模式切换都会使统计量更难控制，自回归推理还常只有极小批次。按单个词元隐藏维归一化，不依赖其他样本，行为更稳定且自然适配变长输入。

若隐藏维为 d，RMSNorm 通常只有 d 个可学习缩放参数；LayerNorm 常有 d 个缩放和 d 个偏置参数。两者的参数量相对主干矩阵很小，但它们位于每层关键入口，会直接影响残差流尺度、softmax 分数和半精度数值范围，所以“小模块”不等于“不重要”。

面试深挖：RMSNorm 为什么可以不减均值？

20 秒回答： RMSNorm 的目标主要是控制残差流的尺度，而不是强制特征零中心。它用均方根归一化向量能量，保留方向并减少均值统计；经验上这种尺度控制足以支持许多仅解码器 LLM 的稳定训练。

深入解释： LayerNorm 同时消除平移和缩放，RMSNorm 只消除整体缩放。RMSNorm 输出的均方根接近由 g 决定的尺度，但均值可以非零。ϵ 只是数值稳定项，不是鼓励小权重或小激活的正则化系数。

面试官继续追问：

1. 归一化沿 batch、sequence 还是 hidden 维做?
2. RMSNorm 是否保证输出均值为 0?
3. 为什么 Transformer 不主要使用 BatchNorm?
4. Pre-Norm 与 Post-Norm 的梯度路径有何区别?

5.2 RoPE

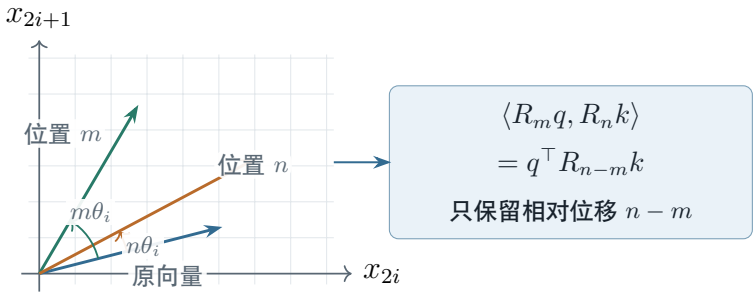


图 5.1 RoPE 的二维几何解释：旋转后内积只依赖相对位置。

提出动机：让位置关系直接进入注意力内积

绝对位置嵌入把“位置 m”作为附加向量写入隐藏状态，但 attention 真正需要的往往是两个词元相隔多远。固定长度的位置表也不自然支持超出训练长度的位置。旋转位置编码（Rotary Position Embedding，RoPE）把位置编码改写为对查询和键的旋转，使二者内积能够显式化为相对位移，同时不增加每个位置的可学习嵌入表。

RoPE 把每两个维度视为一个二维平面。对当前这一对坐标，位置 m 对应旋转角 θ_m，旋转矩阵为

$$R_m = \begin{bmatrix} \cos\theta_m & -\sin\theta_m \\ \sin\theta_m & \cos\theta_m \end{bmatrix}.$$

设 $q, k \in \mathbb{R}^2$ 分别是这一坐标对上的查询和键，m, n 是它们所在的位置。分别旋转后：

$$(R_m q)^\top (R_n k) = q^\top R_m^\top R_n k = q^\top R_{n-m} k.$$

其中 R_{n-m} 表示由位置差决定的相对旋转。因此注意力分数显式依赖相对位置 $n-m$，这解释了 RoPE 为什么适合自回归模型，也解释了长上下文外推会涉及频率缩放。

具体例子：旋转后为什么只剩相对距离

只看一个二维坐标对，并假设每前进一个位置旋转 $30°$。令原始 $q = k = (1, 0)$，位置 $m = 2$ 的查询旋转到 $60°$，位置 $n = 5$ 的键旋转到 $150°$。两者内积为

$$\cos(150° - 60°) = \cos 90° = 0.$$

若把两个词元同时向后移动十个位置，角度变成 $360°$ 与 $450°$，差仍是 $90°$，内积仍为 0。绝对位置变了但相对距离 $n-m = 3$ 未变，这就是 $R_m^\top R_n = R_{n-m}$ 的几何含义。

实际模型不会让所有二维平面使用同一个角速度。第 j 对维度通常使用角频率 $\theta_j = \text{base}^{-2j/d_h}$，位置 m 的旋转角为 $m\theta_j$。这里 $j = 0, 1, \ldots, d_h/2 - 1$ 是坐标对编号，d_h 是单个注意力头的维度，base 是控制频率跨度的常数。高频维度更敏感于相邻位置，低频维度变化更慢、覆盖距离更长。因为每次旋转必须取两个坐标，所以头维度必须为偶数。

5.2.1 对应实现：RoPE 二维旋转

```python
import torch

def apply_rope(x, cos, sin):
    # x: [B, H, T, D]，最后一维必须能两两配对。
    # cos/sin: [1, 1, T, D/2]，已包含各位置和频率。
    if x.size(-1) % 2 != 0:
        raise ValueError("RoPE requires an even head dimension")

    # 把 (x0, x1), (x2, x3), ... 拆成二维坐标对。
    x_even = x[..., 0::2]  # 每对的第一个坐标
    x_odd = x[..., 1::2]   # 每对的第二个坐标

    # [a, b] 乘旋转矩阵后得到
    # [a*cos - b*sin, a*sin + b*cos]。
    rotated_even = x_even * cos - x_odd * sin
    rotated_odd = x_even * sin + x_odd * cos
```

```
    # stack 后为 [..., D/2, 2]; flatten 恢复交错的 D 维。
    return torch.stack(
        (rotated_even, rotated_odd), dim=-1
    ).flatten(-2)
```

代码中的 cos 和 sin 只在位置与维度上变化，在批次与注意力头维度通过广播机制复用。通常只对查询和键应用 RoPE，不旋转值向量，因为位置关系需要进入注意力分数 $QK^\top$，而值向量负责承载被汇聚的内容。

RoPE 不需要为每个位置学习一行嵌入参数，也不改变 Q, K 的张量形状；它只对最后一维成对坐标做确定性旋转。值向量不参与 $QK^\top$ 的匹配，旋转 V 不会帮助分数显式获得相对位移，反而会把位置相位写入被聚合的内容。

长上下文外推的困难来自频率分布失配：训练时模型只见过有限位置范围与相位组合，超出范围后，某些高频坐标会经历更多旋转，注意力模式可能进入训练未覆盖区域。位置插值或频率缩放的基本思想，是把更长的实际位置映射到较温和的有效相位范围；它改善外推，但可能压缩近距离分辨率，因此仍需要校准或长上下文训练。

面试深挖：RoPE 为什么只作用于 Q/K？

20 秒回答： 位置信息需要进入注意力匹配分数 $QK^\top$。对位置 m, n 的 Q/K 做同形式旋转后，内积可化为相对旋转 R_{n-m}，从而让分数依赖相对距离；V 负责传递内容，不参与匹配，因此通常无需旋转。

深入解释： $R_m^\top R_n = R_{n-m}$ 是关键。RoPE 不增加可学习位置表，也不改变 shape，只在二维坐标对上施加正交变换。要求头维度成对处理，是因为每个旋转块至少需要两个坐标。

面试官继续追问：

1. RoPE 是否增加参数量？
2. 为什么绝对位置嵌入不能直接保证相对关系？
3. 长上下文外推为什么可能退化？
4. position interpolation 牺牲了什么？

5.3 ALiBi 与滑动窗口

提出动机：完整长上下文注意力仍然太贵

RoPE 解决“如何表达位置”，却没有改变完整 attention 的 $O(T^2)$ 计算，也不能单独保证长度外推。线性偏置注意力（Attention with Linear Biases，ALiBi）试图用简单的距离偏置提供更平滑的位置归纳偏置；滑动窗口则直接限制可见范围，把资源集中到更可能相关的邻近词元。

ALiBi 为分数加入随距离衰减的线性偏置；滑动窗口只看最近 W 个词元，其中 W 是窗口大小。这样可将位置对数量从 T^2 降为约 TW，代价是削弱远距离依赖。

二者解决问题的方式不同：ALiBi 改变“看多远时应付出多少分数惩罚”，但仍可计算完整注意力；滑动窗口直接把窗口外位置设为不可见，因此真正减少计算。工程上也可把局部窗口与少量全局词元结合，让模型保留跨段信息通路。

面试提醒：训练长度、位置外推、键值缓存和长距检索能力并不等同。

第六章　多头、共享键值与分组查询注意力

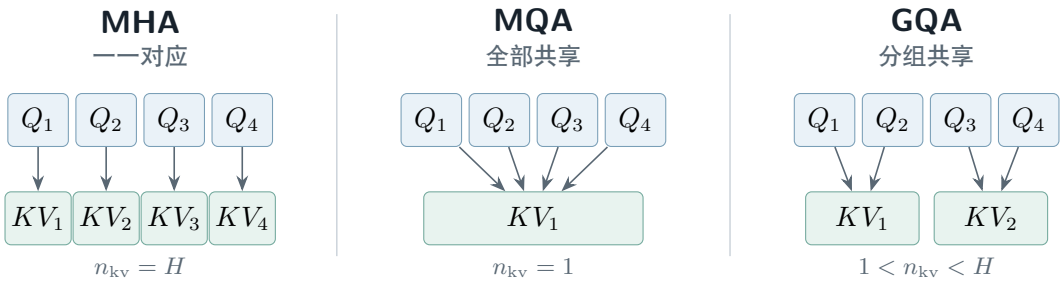


图 6.1　MHA、MQA 与 GQA 的键值共享方式。查询头数量保持不变，键值头越少，缓存与解码读取带宽越小。

提出动机：降低逐词元推理的键值读取成本

多头注意力（Multi-Head Attention，MHA）为每个查询头保存独立的键/值头，表达能力强，但逐词元解码（decode）每一步都要从显存读取所有历史键值。随着批次和上下文增长，瓶颈逐渐从矩阵计算转为显存容量与带宽。多查询注意力（Multi-Query Attention，MQA）通过共享一组键值把成本降到最低，分组查询注意力（Grouped-Query Attention，GQA）则在质量下降与缓存开销之间提供折中。

MHA 中查询头数 H 与键值头数通常相同。MQA 让所有查询头共享一组键值。GQA 设 $n_{\mathrm{kv}} < H$，每组查询头共享一个键值头。

若每层缓存都保存键矩阵 K 与值矩阵 V，显存与 n_{kv} 成正比：

$$M_{\mathrm{KV}} \propto 2 \cdot L \cdot B \cdot T \cdot n_{\mathrm{kv}} \cdot d_h.$$

式中，系数 2 表示同时保存键和值，L 是层数，B 是批次大小，T 是已缓存的序列长度，n_{kv} 是键值头数，d_h 是每头维度。因此从 MHA 的 $n_{\mathrm{kv}} = H$ 改到 GQA 的 $n_{\mathrm{kv}} = H/g$ 时，g 表示每个键值头服务的查询头数，键值缓存约减少 g 倍。

具体例子：8 个查询头如何共享 2 个键值头

设 $H = 8$、$n_{\mathrm{kv}} = 2$，则每组大小为 $g = H/n_{\mathrm{kv}} = 4$：

$$Q_1, Q_2, Q_3, Q_4 \to (KV)_1, \qquad Q_5, Q_6, Q_7, Q_8 \to (KV)_2.$$

八个查询头仍各自提出不同查询，因此输出仍有八个头；只是在读取历史信息时，每四个查询头共享同一份键和值。相较 $n_{\mathrm{kv}} = 8$ 的 MHA，每层、每个词元只需缓存两组而不是八组键值，显存和读取量都约降为四分之一。

机制	KV 头数	优点	风险
MHA	H	表达能力强	键值缓存大
MQA	1	逐词元解码显存最低	质量可能下降
GQA	$1 < n_{\mathrm{kv}} < H$	质量与效率折中	分组设计需调参

表 6.1 MHA、MQA 与 GQA 的工程对比。

GQA 的收益必须按阶段区分。在提示词预填充阶段，T 个查询可并行形成大矩阵乘法，系统常更偏计算密集；减少 K/V 投影和缓存写入有收益，但不会消除所有 QK 分数计算。逐词元解码时每步只有一个新查询，却要读取长度为 T 的历史键值，算术强度低，因此减少 n_{kv} 往往直接降低显存流量和延迟。

计算题：GQA 的分组与缓存缩减

给定 $H = 32$ 个查询头、$n_{\mathrm{kv}} = 8$ 个键值头，每个键值头服务

$$g = H/n_{\mathrm{kv}} = 32/8 = 4$$

个查询头。与 $n_{\mathrm{kv}} = 32$ 的 MHA 相比，键值缓存和理想键值读取量降为 $8/32 = 1/4$，即约减少 4 倍。查询头仍为 32，所以模型仍保留 32 组查询投影。

面试深挖：GQA 省的是 FLOPs、显存还是带宽？

20 秒回答： 三者都有局部收益，但最核心的是键值缓存容量和逐词元解码的显存带宽。K/V 投影 FLOPs 会下降；然而每个 Q 头仍要与历史键值形成注意力，稠密 attention 的主要位置对数量不会按相同比例消失。decode memory-bound 时带宽收益最明显。

深入解释： 缓存大小与 n_{kv} 成正比。decode 每步新计算的 Q 很小，主要成本常是从 HBM 读取历史 K/V；GQA 让多个 Q 头复用同一份 K/V。prefill 中大矩阵算术强度更高，收益更依赖具体内核与投影占比，因此不能笼统承诺同样的加速倍数。

面试官继续追问：

1. 为什么 Q 头数可以保持不变？
2. MQA 为什么压缩最大，但并非所有模型都使用？
3. $H = 32, n_{\text{kv}} = 8$ 时 group size 是多少？
4. 缓存减少 4 倍是否意味着端到端一定加速 4 倍？

6.1 多头潜在注意力（MLA）

提出动机：比减少键值头更进一步压缩缓存表示

GQA 通过让多个查询头共享较少的键值头来缩减缓存，但缓存中仍保存这些键值头的显式向量。多头潜在注意力（Multi-Head Latent Attention，MLA）的出发点，是把历史信息压缩成更低维的潜在表示，在需要计算注意力时再通过投影使用或重建相应表示，以进一步降低缓存容量与读取流量。

可以用概念式表示：

$$c_t = x_t W_c, \qquad c_t \in \mathbb{R}^{d_c}, \quad d_c \ll H d_h,$$

其中 c_t 是位置 t 的低维潜在缓存，W_c 是压缩投影。推理时，注意力计算通过额外投影从 $c_{1:t}$ 获得所需的键值相关表示。具体模型如何处理位置编码、投影吸收和解码内核各不相同，本节只强调稳定的架构思想：**缓存低维潜变量，而不是完整的逐头 K/V**。

机制	缓存思路	主要权衡
MHA	保存每个头的完整 K/V	表达直接，缓存最大
GQA/MQA	减少并共享 K/V 头	共享更强时质量风险上升
MLA	保存低维潜在表示	缓存更小，但引入压缩与投影复杂度

表 6.2 MHA、GQA 与 MLA 的概念差异。

面试深挖：GQA 和 MLA 降低键值缓存的本质区别是什么？

20 秒回答： GQA 减少显式 K/V 头数，让多组 Q 共享较少的键值；MLA 则把历史信息压缩到更低维潜在表示，缓存的是压缩表示。前者是“少存几组头”，后者是“换一种低维表示来存”。

深入解释： 若潜在维 d_c 显著小于显式键值宽度，存储和读取量都可下降；代价是压缩可能损失信息，并需要额外投影、位置处理与高效内核支持。压缩率不能脱离质量、计算和实现复杂度单独比较。

面试官继续追问：

1. latent compression 为什么能降低存储?
2. 压缩带来的质量与计算代价是什么?
3. MLA 是否等价于把 n_{kv} 设得更小?

6.2 查询/键归一化（QK normalization）

提出动机：防止注意力分数随训练失控

除以 $\sqrt{d_h}$ 只根据初始化时的独立同分布假设校准尺度，但训练会持续改变查询和键的范数。若范数越来越大，内积仍会把 softmax 推入近似独热（one-hot）的饱和区，导致梯度变小、注意力熵骤降。长上下文中极端分数的机会更多，因此问题更明显。

查询/键归一化在查询和键向量上做归一化或尺度控制，使内积主要反映方向相似度而不是无界增长的向量长度。它解决的是注意力分数与梯度稳定性，不直接降低 T^2 复杂度，也不替代因果遮罩。面试中可把它归为“控制 softmax 有效温度”的技巧。

具体例子：方向相同但范数失控时会发生什么

令 $q = (30, 40)$、$k = (6, 8)$，两者方向完全相同，但原始内积为 500，softmax 中的 e^{500} 会极端放大该位置。归一化后 $\hat{q} = (0.6, 0.8)$、$\hat{k} = (0.6, 0.8)$，内积变成 1。相关性排序仍表达“方向完全一致”，但不会因为训练把向量范数放大而无界增长。

这些改进分别作用在不同轴上：RMSNorm 控制残差流尺度，RoPE/ALiBi 注入位置，滑动窗口减少可见范围，GQA 减少键值头，查询/键归一化控制分数尺度。进入推理阶段后，还需要进一步消除历史 K, V 的重复计算，并优化 attention 的内存访问。

面试高频问题

Q: RoPE 为什么能表达相对位置？

20 秒回答：两个位置的旋转内积可化为基向量之间乘以相对旋转矩阵，score 依赖 $n - m$ 而不是单独的 m, n。

Q: GQA 为什么能加速逐词元解码？

20 秒回答：逐词元解码每步要从显存读取所有历史键值；GQA 减少键值头数，直接降低缓存读取量和显存占用。

Q: 查询/键归一化什么时候值得提？

当讨论长上下文稳定性、注意力分数过大或训练后期不稳定时值得提。它的核心是控制查询和键的尺度，让 softmax 温度更可控。

第四部分

高效注意力与推理

第七章　键值缓存（KV Cache）

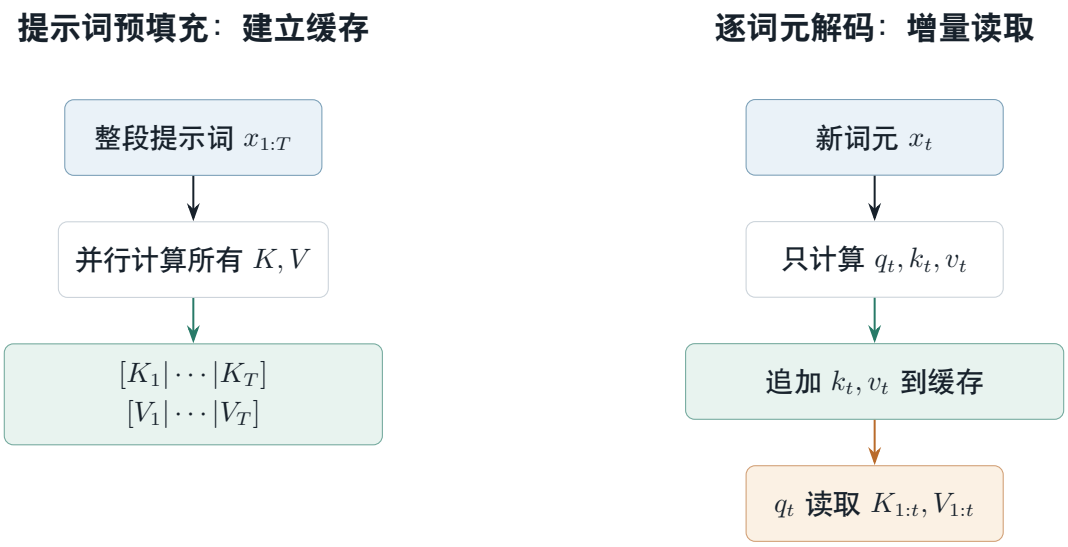


图 7.1　键值缓存的两阶段视角。提示词预填充并行写入整段上下文；逐词元解码只追加当前位置，并读取全部历史缓存。

提出动机：避免每生成一个词元都重算整段历史

自回归模型第 t 步的历史词元与上一步几乎相同。若每次都重新投影全部历史 K, V，生成 T 个词元会重复执行大量相同计算。键值缓存用显存换计算：历史位置的键和值只计算一次，后续步骤直接复用。它显著降低逐词元解码的计算，却把长上下文推理变成显存容量和带宽问题。

模型推理通常分成两个阶段。第一阶段是**提示词预填充（prefill）**：把用户给出的整段提示词一次并行送入模型，并为其中每个位置建立键值缓存。第二阶段是**逐词元解码（decode）**：第 t 步只产生一个新词元，并把它的新键和值追加到缓存。若不缓存，每一步都要重新计算全部历史词元的 K, V。通常不缓存查询向量，因为它只服务当前一步，用完即可。

具体例子：生成“我喜欢机器学习”时缓存了什么

提示词预填充阶段输入“我 / 喜欢”后，每一层保存 (k_1, v_1) 与 (k_2, v_2)。开始生成“机器”时，只为当前隐藏状态计算 q_3, k_3, v_3，把 (k_3, v_3) 追加到缓存，再用查询向量 q_3 读取

$$[k_1, k_2, k_3] \quad \text{和} \quad [v_1, v_2, v_3].$$

下一步生成“学习”时，同样只新算 q_4, k_4, v_4，历史 k_1 到 k_3 不再投影。q_3 用完后不会被下一步读取，所以无需缓存查询；下一步会从新的隐藏状态产生 q_4。

关键推导

键值缓存显存为

$$M_{\mathrm{KV}} = 2 \times L \times B \times T \times n_{\mathrm{kv}} \times d_h \times s.$$

其中 M_{KV} 是缓存占用的总字节数；2 表示 K, V 两份；L 是层数；B 是批次大小；T 是缓存长度；n_{kv} 是键值头数；d_h 是单头维度；s 是每个数值的存储字节数，例如 FP16/BF16 为 2。

例：$L = 32, B = 8, T = 4096, n_{\mathrm{kv}} = 8, d_h = 128, s = 2$，则

$$M_{\mathrm{KV}} = 2 \cdot 32 \cdot 8 \cdot 4096 \cdot 8 \cdot 128 \cdot 2 \approx 4.29 \text{ GB}.$$

若多头注意力有 32 个键值头，同条件约 17.18 GB；分组查询注意力的收益非常直接。

更严格地说，上例的 4,294,967,296 字节等于 4.29 GB（十进制）或 4.00 GiB（二进制）。面试估算时应先说明单位口径，避免把 GB 与 GiB 混用。

计算题：长上下文键值缓存

给定

$$L = 32, \quad B = 4, \quad T = 8192, \quad n_{\mathrm{kv}} = 8, \quad d_h = 128,$$

BF16 每个元素占 2 字节，则

$$M_{\mathrm{KV}} = 2 \times 32 \times 4 \times 8192 \times 8 \times 128 \times 2 = 4{,}294{,}967{,}296 \text{ bytes}.$$

因此缓存约为 4.29 GB 或 4.00 GiB。若改为 32 个键值头，其他条件不变，缓存约增至 17.18 GB。

7.1 面试追问链

面试深挖：为什么缓存 K/V 而通常不缓存 Q？

20 秒回答：历史 K/V 会被后续每一个新查询反复读取，缓存后可避免对历史隐藏状态重复投影；历史 Q 只在它自己生成时使用一次，下一步会从新隐藏状态产生新的 Q，因此缓存历史 Q 没有复用价值。

深入解释：不用缓存时，第 t 步会再次对 $x_{1:t}$ 计算 K/V，前 $t-1$ 个位置与上一步完全重复。使用缓存后只投影新位置，并让 q_t 与 $K_{1:t}$ 计算分数。投影重复被消除，但读取历史 K/V 和计算 $q_t K_{1:t}^\top$ 仍随 t 线性增长，所以不能说“KV Cache 让 attention 变成 $O(1)$”。

工程含义：缓存以显存容量和带宽换计算。上下文越长，单步需要读取的历史 K/V 越大；这正是 decode 常从计算瓶颈转为 memory-bound 的原因。

面试官继续追问：

1. 完全不用缓存时，重复计算具体发生在哪里？
2. 缓存后单步 attention 还剩多少与 T 有关的工作？
3. GQA 为什么同时降低缓存和 decode 带宽？
4. KV Cache 量化会引入什么误差？

面试深挖：Prefix caching 与普通 KV Cache 有什么区别？

20 秒回答：普通 KV Cache 在一次请求内部复用该请求已经计算的历史；前缀缓存（prefix caching）进一步在不同请求之间复用完全相同前缀的 K/V，例如共享系统提示词或公共文档前缀。

深入解释：跨请求复用要求前缀词元、模型版本、位置编码状态和相关推理配置兼容。命中后可跳过公共前缀的重复预填充，降低首词元延迟；但缓存索引、生命周期、租户隔离和显存淘汰都会变复杂。相似但不完全相同的自然语言前缀不能直接复用精确 K/V。

面试官继续追问：

1. prefix cache 命中主要减少 prefill 还是 decode？
2. 为什么位置偏移会影响复用？
3. 多租户系统如何避免缓存泄漏？

容易答错

错误说法： KV Cache 让自回归 attention 变成 $O(1)$。
更准确： 它让每步新增位置的 K/V 投影接近常数工作量，并消除历史投影的重复计算；但当前查询仍要读取并匹配长度为 T 的历史键值，单步读取量和点积工作仍随上下文线性增长。

7.2 对应实现：单步逐词元解码与键值缓存

下面的代码展示语义完整的单步逐词元解码。它把当前词元产生的新 K, V 追加到历史缓存，再让当前查询读取全部历史位置。为突出张量形状，代码保留了显式的 GQA 头复制；高性能计算内核通常不会真的复制这些数据。

```
import math
import torch

def decode_attention(
    x_t, k_cache, v_cache,
    q_proj, k_proj, v_proj,
    num_query_heads, num_kv_heads,
):
    # x_t: [B, 1, C], decode 每次只处理一个新 token。
    batch = x_t.size(0)
    if num_query_heads % num_kv_heads != 0:
        raise ValueError("query heads must divide into KV groups")

    head_dim = q_proj.out_features // num_query_heads

    def project_one_token(layer, tensor, num_heads):
        # [B, 1, heads*D] -> [B, heads, 1, D]
        projected = layer(tensor)
        return projected.view(
            batch, 1, num_heads, head_dim
        ).transpose(1, 2)

    q = project_one_token(q_proj, x_t, num_query_heads)
    k_new = project_one_token(k_proj, x_t, num_kv_heads)
    v_new = project_one_token(v_proj, x_t, num_kv_heads)

    # cache 的时间维是 dim=2；这里只追加新 token 的 K、V。
    if k_cache is None:
        k_cache, v_cache = k_new, v_new
    else:
        k_cache = torch.cat((k_cache, k_new), dim=2)
        v_cache = torch.cat((v_cache, v_new), dim=2)
```

```
    # GQA 中多个 query 头共享一个 KV 头。
    # repeat_interleave 便于理解，生产 kernel 会直接按组读取。
    repeats = num_query_heads // num_kv_heads
    k_for_attn = k_cache.repeat_interleave(repeats, dim=1)
    v_for_attn = v_cache.repeat_interleave(repeats, dim=1)

    # q: [B, H, 1, D]；历史 K: [B, H, T_cache, D]
    scores = torch.matmul(q, k_for_attn.transpose(-2, -1))
    scores = scores / math.sqrt(head_dim)
    probs = torch.softmax(scores.float(), dim=-1).to(q.dtype)
    output = torch.matmul(probs, v_for_attn)  # [B, H, 1, D]

    # 合并注意力头，恢复为 [B, 1, H*D]。
    output = output.transpose(1, 2).contiguous()
    output = output.view(batch, 1, num_query_heads * head_dim)
    return output, k_cache, v_cache
```

这段示例中的 `torch.cat` 是张量拼接操作，每步都会重新分配连续内存，所以只适合解释算法。真实推理系统会预分配缓存、按当前位置原地写入，或使用分页缓存管理离散块。另一个常见疑问是“为什么没有因果遮罩”：单步查询的键集合已经只包含过去和当前位置，未来词元尚不存在，因此天然满足因果约束。

第八章　FlashAttention

提出动机：算术量没变，为什么标准 Attention 仍然慢？

标准实现会在高带宽显存（High Bandwidth Memory，HBM）中写入并再次读取 $T \times T$ 的分数或概率矩阵。长序列下，数据搬运耗时和中间激活显存可能比乘法本身更突出。FlashAttention 的提出不是为了近似注意力，而是利用分块和在线 softmax 改变精确计算的输入输出（I/O）路径，让中间矩阵尽量停留在更快的片上静态随机存储器（SRAM）。

标准 attention 若显式保存 $A \in \mathbb{R}^{T\times T}$，长序列会产生大量 HBM 读写。FlashAttention 的关键不是改变数学结果，而是改变计算组织：分块读入 Q, K, V，在 SRAM 中做局部计算，用在线 softmax 累积归一化结果，避免把完整注意力矩阵写回 HBM。

关键推导

对某一行分数，处理完旧分块后保存行最大值 m、指数和 ℓ 与未归一化输出 u。新分块分数为 s 时，令

$$m' = \max(m, \max s),$$

$$\ell' = e^{m-m'}\ell + \sum_j e^{s_j - m'}, \qquad u' = e^{m-m'}u + \sum_j e^{s_j-m'}v_j.$$

这里 s_j 是新分块中的第 j 个分数，v_j 是它对应的值向量；m 是此前见过的最大分数，ℓ 是以 m 为基准的指数和，u 是尚未除以指数和的加权值向量。带撇号的 m', ℓ', u' 表示合并新分块后的统计量，最终输出为 $o = u'/\ell'$。因最大值变化而乘的 $e^{m-m'}$ 恰好把旧分块重标定到新基准，所以分块结果与一次性数值稳定 softmax 完全一致，却不必保存完整分数矩阵。

具体例子：在线 softmax 如何合并两个分块

设某个查询的三个分数为 $[1,2,3]$，对应标量值为 $[10,20,30]$。先处理前两项：

$$m = 2, \quad \ell = e^{-1} + 1 \approx 1.368, \quad u = 10e^{-1} + 20 \approx 23.679.$$

再读到分数 3 时，新最大值变为 $m' = 3$。旧统计量必须乘 $e^{2-3} = e^{-1}$ 后再合并：

$$\ell' = e^{-1}\ell + 1 \approx 1.503, \quad u' = e^{-1}u + 30 \approx 38.711.$$

最终 $o = u'/\ell' \approx 25.75$，与一次性计算 $\mathrm{softmax}([1,2,3]) \cdot [10,20,30]$ 完全相同。分块只改变计算顺序，没有近似误差。

```
# FlashAttention 概念伪代码，不是可直接运行的 kernel。
for q_block in blocks(Q):
    # 每个 query 行维护三个在线统计量：
    # m 是目前见过的最大 score，l 是指数和，out 是加权和。
    m = -inf
    l = 0
    out = 0

    for k_block, v_block in blocks(K, V):
        # 当前 Q 分块只与一小块 K 做乘法，score 留在片上内存。
        scores = q_block @ k_block.T / sqrt(d)
        m_new = maximum(m, rowmax(scores))

        # 所有指数都减去新的行最大值，避免 exp 溢出。
        p = exp(scores - m_new)

        # 若最大值发生变化，旧统计量必须重标定到新基准。
        out = out * exp(m - m_new) + p @ v_block
        l = l * exp(m - m_new) + rowsum(p)
        m = m_new

    # 直到读完所有 K/V 分块后才完成归一化。
    O_block = out / l
```

面试一句话回答

FlashAttention 面试重点是数据搬运：减少 HBM 读写，并避免显式生成（materialize）$T \times T$ 中间矩阵，而不是简单说“用了 CUDA 所以快”。

8.1 面试追问链

面试深挖：FlashAttention 为什么更快，却没有改变稠密 attention 的数学结果？

20 秒回答：它仍精确计算所有需要的 $QK^\top$ 和 AV，稠密 attention 的算术复杂度仍是 $O(T^2d)$。主要收益来自分块、在线 softmax 和算子融合：中间分数尽量留在 SRAM，不把完整 $T \times T$ 矩阵写回 HBM，从而减少显存流量和激活占用。

深入解释：在线 softmax 用每行最大值 m、指数和 ℓ 与加权和 u 合并分块，重标定保证结果等价于全行数值稳定 softmax。算法改变的是 I/O 复杂度和执行顺序，不是把注意力近似成稀疏或低秩形式。

工程含义：在长序列预填充/训练中，避免物化 $[B, H, T, T]$ 中间矩阵的收益很大。单步 decode 的查询长度通常为 1，本就不会生成同样规模的二维查询矩阵，此时历史 KV 读取、分页布局和专用 decode kernel 往往更关键；不能假设两个阶段得到相同加速比。

面试官继续追问：

1. FlashAttention 主要减少 FLOPs 吗?
2. 为什么 HBM 与 SRAM 的数据移动会支配时间?
3. online softmax 保存哪些统计量?
4. FlashAttention 和 KV Cache 解决的是同一个问题吗?

容易答错

错误说法：FlashAttention 把 attention 的时间复杂度从 $O(T^2)$ 降成了 $O(T)$。

更准确：对稠密 attention，它仍计算二次数量级的位置对；核心改进是减少 HBM 往返和完整分数矩阵物化，因此峰值显存和墙钟时间可明显下降，但算术复杂度没有变成线性。

第九章　量化、采样与动态批处理

9.1　量化

提出动机：模型权重和带宽成为推理成本上限

大模型推理会反复从显存读取权重，瓶颈可能是数据搬运而不是乘法计算。半精度权重占用的容量和带宽仍会限制模型大小、批次和吞吐。量化用更少比特表示权重或激活，以降低存储、传输和计算成本，代价是舍入、截断和尺度估计引入的误差。

量化把浮点数映射到低比特整数。对称量化常写为

$$q = \operatorname{round}\left(\frac{x}{s}\right), \qquad \hat{x} = sq.$$

其中 x 是原始浮点数，q 是量化后的整数，$s > 0$ 是量化尺度，$\hat{x}$ 是反量化得到的近似值。非对称量化再加入零点（zero point）z：

$$q = \operatorname{round}(x/s) + z, \qquad \hat{x} = s(q - z).$$

逐通道量化（per-channel）为不同通道设置不同尺度，比整个张量共用一个尺度（per-tensor）更细，误差通常更小但元数据更多。

对有符号 b 位对称量化，令 $Q_{\max} = 2^{b-1} - 1$，即正整数的最大可表示值，常取

$$s = \frac{\max_i |x_i|}{Q_{\max}}, \qquad q_i = \operatorname{clip}(\operatorname{round}(x_i/s), -Q_{\max}, Q_{\max}).$$

这里 i 是张量元素下标，clip 把结果截断到整数可表示范围。若没有发生截断，舍入到最近整数可给出误差界 $|x_i - \hat{x}_i| \le s/2$。逐通道量化为第 c 个输出通道单独选择尺度 s_c，因此能降低被极端通道放大的误差。

具体例子：把三个浮点数映射到 INT4

取 $x = [-1.0, -0.2, 0.7]$。有符号 INT4 的 $Q_{\max} = 7$，对称尺度为 $s = 1/7 \approx 0.143$。量化整数为

$$q = \mathrm{round}(x/s) = [-7, -1, 5].$$

反量化得到

$$\hat{x} = sq \approx [-1.000, -0.143, 0.714].$$

第二、三个数与原值存在误差，但每个整数只需 4 位。若同一张量中还有绝对值为 100 的离群值，统一尺度会变成 100/7，上面三个小数几乎都被量化为零；这正是逐通道尺度和异常通道保护有价值的原因。

格式	字节	优点	注意点
FP32	4	精度高	训练显存大
FP16	2	Tensor Core 友好	指数范围较小
BF16	2	指数范围接近 FP32	尾数较少
INT8	1	推理常用	激活尺度需校准
INT4	0.5	权重显存低	误差控制更难

表 9.1 常见数值格式。

直接按权重最大值做均匀舍入，会把所有权重误差视为同等重要；但某些方向上的微小误差会被层输入显著放大。**GPTQ** 因此利用近似二阶信息逐层选择和补偿量化误差，目标是减小该层输出变化，而不只是减小权重自身的逐元素误差。

AWQ 来自另一项观察：少量与大激活相乘的权重通道对输出尤其敏感。它利用校准激活识别并保护这些重要通道，通过尺度变换让其更不容易被低比特量化破坏。二者都是训练后量化（post-training quantization），不需要完整重训；它们改善权重量化误差，却不能自动解决异常激活值的动态范围问题。

9.2 温度、Top-k 与 Top-p 采样

提出动机：概率分布不等于最终生成策略

语言模型只给出下一个词元的条件分布。总选最大概率虽然确定，但容易产生重复和僵化文本；直接从完整分布采样又可能选到长尾噪声。温度（temperature）调节分布锐度，Top-k 和 Top-p 截断不可靠尾部，它们共同解决“如何把模型概率转成符合任务需求的生成行为”。

设候选词元 i 的原始未归一化分数为 z_i。温度采样使用 $p_i \propto \exp(z_i/\tau)$，其中 p_i 是归一化后的采样概率，$\tau > 0$ 是温度：$\tau < 1$ 会放大分数差异，使分布更尖锐；$\tau > 1$ 会缩小差异，增加随机性。Top-k 只保留概率最高的固定 k 个词元；Top-p 则保留累计概率达到阈值 $p \in (0, 1]$ 的最小候选集合，因此会随分布的确定程度自动改变集合大小。

完整温度公式为

$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

当 $\tau = 1$ 时保持原分布；$0 < \tau < 1$ 放大 logit 差，使分布更尖；$\tau > 1$ 压缩差异，使分布更平。当 $\tau \to 0^+$ 时，最大 logit 与其他 logit 的差被无限放大，概率质量集中到最大项，极限行为接近贪心解码。温度直接缩放 logits 等价于统一调节所有类别的相对 log-odds；对 softmax 后概率直接除以温度再归一化，没有同样清晰的指数族解释。

具体例子：Top-p 为什么必须保留越过阈值的词元

假设排序后的词元概率为

$$[0.50, 0.25, 0.15, 0.10],$$

累计概率为 $[0.50, 0.75, 0.90, 1.00]$。当 $p = 0.8$ 时，前两个词元的总概率只有 0.75，尚未达到阈值；必须把第三个词元也保留，候选集合才是前三个，总质量为 0.90。因此代码先标记累计值超过 p 的位置，再把删除遮罩右移一位，留下首次越过阈值的词元。

算法	控制参数	优点	风险
贪心解码	无	稳定快速	容易重复、缺少多样性
Top-k	k	截断低概率噪声	k 固定不适应分布形状
Top-p	p	自适应候选集合	实现需排序或近似
束搜索	束宽	适合确定性任务	开放生成可能僵硬

表 9.2 常见解码算法。

9.2.1 对应实现：Temperature、Top-k 与 Top-p

```python
import torch

def temperature_sample(logits, temperature=1.0):
    # logits: [B, V]，每行是一个样本的词表分数。
    if temperature <= 0:
        raise ValueError("temperature must be positive")

    # 除以温度后再归一化；不要对概率直接除以温度。
    probs = torch.softmax(logits / temperature, dim=-1)
    return torch.multinomial(probs, num_samples=1)  # [B, 1]

def top_k_top_p_filter(logits, top_k=None, top_p=None):
    # clone 避免原地修改调用方仍需使用的 logits。
    filtered = logits.clone()

    if top_k is not None:
        if top_k <= 0:
            raise ValueError("top_k must be positive")
        top_k = min(top_k, filtered.size(-1))
        # 第 k 大的 logit 是保留集合的门槛。
        threshold = filtered.topk(top_k, dim=-1).values[..., -1, None]
        filtered.masked_fill_(filtered < threshold, float("-inf"))

    if top_p is not None:
        if not 0.0 < top_p <= 1.0:
            raise ValueError("top_p must be in (0, 1]")

        # Top-p 必须先按概率从高到低排序。
        sorted_logits, sorted_idx = filtered.sort(
            descending=True, dim=-1
        )
        sorted_probs = torch.softmax(sorted_logits, dim=-1)
        cumulative = sorted_probs.cumsum(dim=-1)

        # 初始 mask 会标记累计概率已经超过 p 的位置。
        remove = cumulative > top_p

        # 右移一位，保留“首次使累计概率达到 p”的 token。
        # 否则候选集合的概率和可能仍小于目标阈值。
        remove[..., 1:] = remove[..., :-1].clone()
        remove[..., 0] = False  # 至少保留最高概率 token

        sorted_logits.masked_fill_(remove, float("-inf"))

        # scatter 把排序后的结果放回原词表下标。
        filtered = torch.full_like(filtered, float("-inf"))
```

```
        filtered.scatter_(-1, sorted_idx, sorted_logits)

    return filtered
```

实际生成时通常先调用过滤函数，再对过滤后的未归一化分数做 softmax 和多项式采样。Top-k 与 Top-p 可以同时启用：代码会先做 Top-k，再在剩余候选中做 Top-p。被置为 $-\infty$ 的位置经 softmax 后概率为零。

9.3 束搜索（Beam Search）

提出动机：局部最优词元不一定组成全局最优序列

贪心解码每步只保留一个词元，一旦早期选择错误，后面无法回退。束搜索同时保留多条高分前缀，用额外计算换取更广的序列搜索，因此适合答案空间较确定、序列概率与任务质量相关较强的场景。

束搜索在每一步保留累计对数概率最高的 K 条序列，其中 K 是束宽。关键不是为每条候选序列各选一个词元，而是把“父候选序列 $\times$ 词表”展平成 KV 个联合候选，再全局取前 K 个。父序列下标决定新序列从哪条旧序列复制，词元下标决定追加什么。

具体例子：为什么要在所有父 beam 中全局选 Top-k

设两条旧 beam 的累计分数为 $A=-0.1$、$B=-0.4$。下一 token 只有 x,y，其条件 log-prob 分别为

$$A:[-0.2,-1.0],\qquad B:[-0.05,-0.2].$$

四个联合候选的累计分数是 $A+x=-0.3$、$A+y=-1.1$、$B+x=-0.45$、$B+y=-0.6$。若 beam size 为 2，全局保留 $A+x$ 与 $B+x$。这说明不能简单地让每条父 beam 各自保留一个 token；所有父子组合必须一起竞争。

9.3.1 对应实现：单步 Beam 扩展

```python
def simple_beam_step(log_probs, beam_scores, beam_size):
    # log_probs: [B, beam, V]，当前步每条 beam 的词表分数。
    # beam_scores: [B, beam]，生成到上一步的累计对数概率。
    batch, old_beam, vocab_size = log_probs.shape

    # 广播累计分数，得到所有“父 beam + 新 token”组合。
    scores = log_probs + beam_scores[..., None]
    # scores: [B, old_beam, V]

    # 展平后，每个下标都唯一对应一组 (parent, token)。
    flat_scores = scores.view(batch, old_beam * vocab_size)
    new_scores, flat_idx = flat_scores.topk(beam_size, dim=-1)

    # 整除恢复父 beam，下标取模恢复新 token。
    parent_beam = flat_idx // vocab_size
    next_token = flat_idx % vocab_size
    return parent_beam, next_token, new_scores
```

完整实现还要处理 EOS、长度惩罚、重复惩罚和回溯序列。长度惩罚尤其重要，因为对数概率通常为负，单纯累加会天然偏爱较短序列。开放式对话往往更适合采样；翻译、语音识别等目标较确定的任务更常使用 Beam Search。

面试深挖：Top-k、Top-p 与束搜索分别改变什么？

20 秒回答： Top-k 固定候选数量，Top-p 固定累计概率质量、候选数量动态变化；两者通常用于随机采样。束搜索则并行保留若干条累计分数最高的序列前缀，目标更接近搜索高概率完整序列，开放式对话中可能显得保守和模板化。

深入解释： 分布很尖时，Top-p 可能只保留少数候选；分布平时会保留更多。束搜索累加负对数概率，序列越长通常累加越多负数，因此需要长度归一化或长度惩罚。它擅长答案空间较确定的任务，但最大似然序列不一定等于人类认为最自然、多样的回答。

面试官继续追问：

1. $\tau \to 0$ 为什么接近 greedy?
2. Top-p 为什么候选数量不是常数?
3. beam search 为什么偏好短序列?
4. 为什么开放式对话常使用 sampling?

9.4 推测式解码（Speculative Decoding）

提出动机：大模型逐词元解码串行且每步利用率低

目标模型必须逐词元生成，单步矩阵规模小、带宽占比高，而且下一步依赖上一步结果，难以直接并行。推测式解码让计算较便宜的草稿模型（draft model）先连续提出多个候选，再由目标模型（target model）一次并行验证，把“多次小前向”尽量合并成“一次较大的前向”。

推测式解码使用小草稿模型先提出多个词元，大目标模型再并行验证。若接受规则按概率比修正，最终分布仍可与目标模型的采样分布一致。收益来自把多个候选词元的验证合并成一次目标模型前向。

设候选词元为 x，草稿模型与目标模型在当前步对它给出的概率分别为 $q(x)$ 与 $p(x)$。草稿候选 x 的接受概率为

$$\alpha(x) = \min\left(1, \frac{p(x)}{q(x)}\right).$$

若拒绝，则从残差分布 $p'(x) \propto [p(x) - q(x)]_+$ 重新采样，其中 $[a]_+ = \max(a, 0)$ 表示只保留正数部分。接受分支贡献 $\min(p(x), q(x))$，拒绝分支补齐其余概率，因此校正后的边缘分布仍是目标分布 $p(x)$。

具体例子：草稿候选何时会被接受

假设草稿模型对候选词元“学习”给出 $q(x) = 0.5$，目标模型只给 $p(x) = 0.3$，则接受概率

$$\alpha(x) = \min(1, 0.3/0.5) = 0.6.$$

也就是说，草稿模型对该词元过于自信，只有 60% 概率直接接受，其余情况要按残差分布校正。若 $q(x) = 0.2$、$p(x) = 0.3$，则 $\alpha = 1$，候选总被接受。草稿模型越接近目标模型，连续接受的词元越多，加速收益越高；候选质量太差时频繁拒绝，额外草稿计算反而可能得不偿失。

面试深挖：推测式解码为什么能加速又保持目标分布？

20 秒回答：更快的草稿模型先提出多个候选，目标模型用一次并行前向验证；基于 $\min(1, p/q)$ 的接受规则和残差分布校正，使最终边缘分布仍等于目标模型分布。收益取决于一次验证能接受多少连续候选。

深入解释：草稿模型太弱会使接受率低，白做许多草稿计算；太大又会让草稿本身接近目标模型成本。理想草稿模型既快又与目标分布接近。该方法针对串行 decode，而不是已经能够大矩阵并行的 prefill。

面试官继续追问：

1. acceptance rate 决定什么?
2. draft model 太弱和太大分别会怎样?
3. 为什么目标模型能一次验证多个候选?
4. 贪心推理与随机采样下的验证规则有何不同?

9.5 连续批处理与分页注意力

提出动机：真实请求长度不同，静态批处理会浪费算力和显存

在线服务中的提示词和输出长度各不相同。静态批处理必须等待最慢请求结束，已完成请求留下空槽；按最大长度预留连续键值空间又会产生碎片和浪费。连续批处理（continuous batching）解决调度空洞，分页注意力（paged attention）解决键值内存必须连续分配的问题。

静态批处理要等同一批请求全部结束，容易被长请求拖慢。连续批处理在每个解码步动态加入新请求、移除完成请求，提高 GPU 利用率。分页注意力把键值缓存分成块，像虚拟内存一样管理，减少大块连续显存需求和碎片。

分页注意力解决的核心是**内存管理和访问布局**，不是改变 attention 公式。逻辑上连续的词元可以映射到物理上不连续的缓存块；调度器按需分配、复用和回收页面，避免为每个请求按最大可能长度预留整块显存。代价是页表管理、间接寻址和内核实现更复杂。

键值缓存量化则从另一维度压缩每个缓存元素的位宽。若从 BF16 降到 INT8，理想存储量可减半，decode 读取带宽也可能下降；但 K/V 的量化误差会直接扰动注意力分数与加权内容，长上下文、异常通道和逐层误差积累都可能影响质量，还要付出量化元数据和反量化计算。

面试深挖：PagedAttention 解决的是计算问题还是内存问题？

20 秒回答：主要是键值缓存的分配、碎片与地址管理问题。它把逻辑连续缓存映射到固定大小物理页，按需分配并复用；attention 的数学计算仍然存在，只是内核能够从分页布局中读取历史键值。

深入解释：连续批处理解决“请求何时进入和退出批次”，分页缓存解决“每个请求的 K/V 放在哪里”。两者经常配合，但一个是调度问题，一个是内存布局问题。它们都不等价于减少模型参数或改变注意力概率。

面试官继续追问：

1. 页太大或太小分别有什么代价?

2. 分页如何减少外部碎片?
3. continuous batching 与 static batching 的延迟权衡是什么?
4. KV Cache quantization 主要收益和风险是什么?

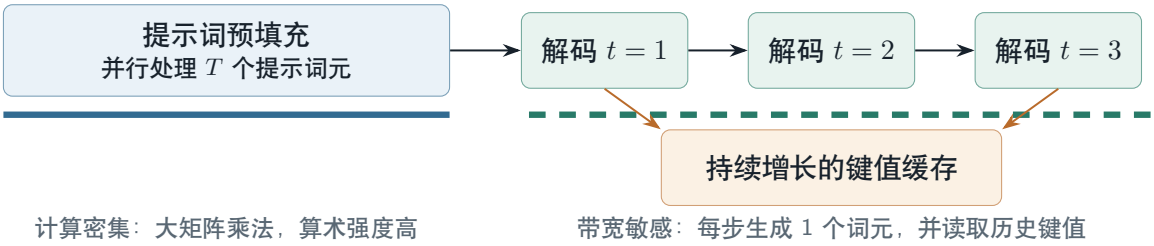


图 9.1 LLM 推理时间线。提示词预填充能并行处理输入；逐词元解码必须串行生成，并随上下文增长读取更大的键值缓存。

从这一章可以看到，推理优化不是单一“加速技巧”：键值缓存消除重复计算，FlashAttention 减少数据搬运，量化减少表示位宽，采样控制输出行为，推测式解码增加单次验证并行度，连续批处理与分页注意力提高多请求资源利用率。接下来后训练要解决的是另一类问题：预训练模型会续写文本，但未必会遵循指令或符合人类偏好。

面试高频问题

Q: 逐词元解码为什么常被显存带宽限制？

20 秒回答：每步只生成一个词元，矩阵计算规模小，但必须读取所有层、所有历史词元的键值缓存，算术强度低。

Q: vLLM 的分页键值缓存解决什么问题？

20 秒回答：它把键值缓存切成页，按需分配和复用，降低长短请求混合时的显存碎片和预留浪费。

第五部分

推理模型与测试时计算

第十章　Thinking 与 Non-Thinking 模式

提出动机：同一组参数之外，推理阶段还能增加什么？

传统语言模型常按固定解码策略直接生成答案；但数学、代码和规划任务需要探索中间状态、比较候选并检查约束。只增加训练参数并不是唯一提升路径：推理阶段也可以投入更多词元预算、采样次数、搜索与验证计算。推理模型关注的正是如何把这些额外计算转化为更高的问题求解质量。

推理模型（reasoning model）不是由单一网络结构定义的严格类别。面试中更稳妥的说法是：它经过数据、训练目标或推理策略优化，能够在复杂任务上有效利用较多的中间计算、搜索或验证，再输出最终答案。推理能力可能同时来自预训练中的结构学习、监督微调的示范、可验证奖励强化学习，以及推理时搜索；不能把能力归因于某一个唯一阶段。

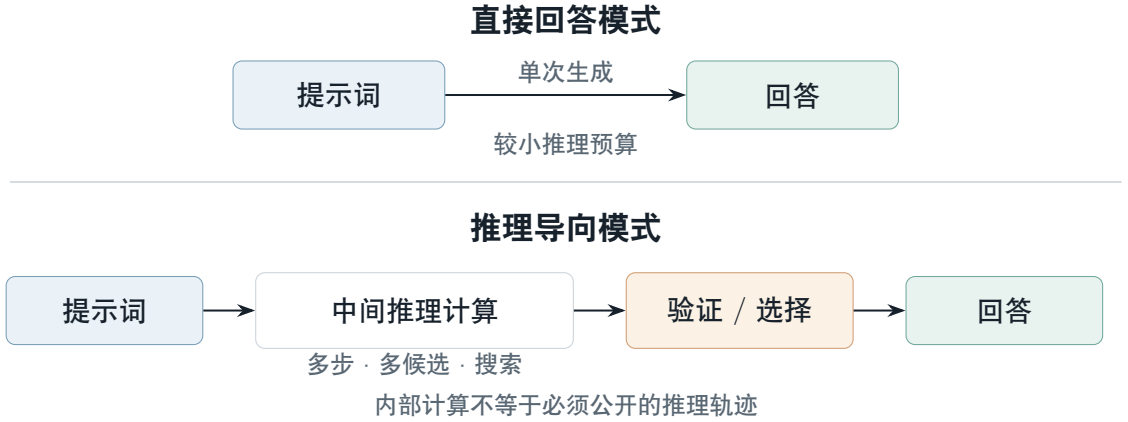


图 10.1　直接回答与推理导向推理的执行差异。后者增加推理预算、候选或验证步骤，但不意味着必须公开内部推理轨迹。

10.1　六个容易混淆的概念

Thinking 与 Non-Thinking 可以是同一模型的两种推理行为，也可以来自不同模型。若系统只改变提示模板、最大推理词元、采样数量或验证流程，参数并未改变；若两个模式对应不同的后训练检查点、路由分支或专用模型，参数或激活路径也可能不同。因此“thinking 是否改变参数”的正确答案是：**单次模式切换不必修改参数，但产品中的两个模式不保证共享完全相同的模型实现**。

内部推理轨迹和可见解释也不能画等号。系统可以在内部进行隐藏状态计算、搜索或工具调用，只输出简洁结论；也可以额外生成一段面向用户的验证

概念	准确定义
直接回答模式	提示词后立即生成面向用户的答案，通常使用较小推理预算
推理导向模式	在最终答案前投入更多内部计算、候选搜索或验证
思维链提示（CoT prompting）	用提示或示例引导模型生成分步文本；它是一种交互方法，不等同于全部推理能力
内部推理	模型或系统内部用于求解的中间计算，不应假定会完整暴露给用户
可见推理说明	面向用户生成、可审核的步骤或解释；可能是内部计算的筛选、重写或单独生成结果
推理预算	对中间词元、采样次数、搜索宽度、验证调用或时间的限制

表 10.1 Reasoning 讨论中必须区分的层次。

说明。可见步骤可能帮助审计，但并不保证忠实复现全部内部计算，更长的可见文本也不自动代表更强的推理。

面试深挖：Thinking 模式和普通生成模式的本质区别是什么？

20 秒回答：区别不一定在参数，而在推理阶段分配的计算与控制策略。Thinking 模式通常允许更多中间词元、多候选、搜索或验证，再产生答案；普通模式更接近单次直接生成。前者用更高延迟和成本换取复杂任务上潜在更高的正确率。

深入解释：语言模型的基本操作仍是条件生成。系统可以改变停止条件、推理预算、采样次数、验证器和工具闭环，使同一参数模型表现出不同推理行为；也可以通过 SFT/RL 训练它更有效地利用这些预算。模式名称本身不能说明具体训练配方。

面试官继续追问：

1. thinking 是否意味着参数发生改变?
2. reasoning 能力来自预训练、SFT、RL 还是搜索?
3. CoT 与真正的问题求解能力是一回事吗?
4. 为什么内部推理不一定对用户可见?

10.2 推理词元、延迟与适用任务

设直接回答生成 T_a 个词元，推理导向模式额外使用 T_r 个中间词元。忽略缓存常数后，单请求解码工作随总生成长度 $T_r + T_a$ 增长；而且上下文越长，后续每步读取的键值缓存越大。若还采样 N 条轨迹，粗略计算预算可写成

$$C_{\text{test}} \propto \sum_{n=1}^{N} \left(T_r^{(n)} + T_a^{(n)} \right) + C_{\text{verify}} + C_{\text{search}}.$$

这里 C_{verify} 和 C_{search} 分别表示验证与搜索成本。真实延迟还取决于并行批处理、接受率、硬件带宽和服务调度，不能只按词元数线性外推。

数学和代码任务通常收益更明显，是因为它们有明确约束、可探索的中间结构和较可靠的验证器：答案可代入方程，代码可编译并运行测试。开放式写作没有唯一正确目标，额外搜索容易优化到狭窄评分器偏好；简单事实题的解空间又很小，长推理可能只增加延迟、引入无关分支甚至把正确直觉改错。

面试深挖：为什么更长的 reasoning 不一定更正确？

20 秒回答：更多计算只增加探索机会，不保证探索方向和验证器正确。长轨迹会累积错误、重复和无关分支；如果没有可靠反馈，模型可能用更多词元强化错误假设。收益通常先上升后饱和，甚至下降。

深入解释：可以把推理看成在候选解空间中分配预算。质量取决于候选覆盖、搜索策略和选择器准确率，而不是轨迹长度本身。简单问题的最优策略可能就是直接回答；复杂问题才可能值得多路径搜索和验证。系统应根据难度动态分配预算，而不是所有请求固定长思考。

面试官继续追问：

1. reasoning token 增加会影响哪些成本?
2. 为什么数学和代码更适合测试时扩展?
3. 如何为简单问题设置提前停止?
4. 长推理会怎样影响 KV Cache 和吞吐?

容易答错

错误说法：能输出很长思维链，就证明模型具有更强的推理能力。

更准确：可见文本长度只是计算轨迹的一个表面指标。应使用最终正确率、验证通过率、鲁棒性和单位计算收益评估推理能力；可见解释还可能是对内部过程的重新组织，而非完整记录。

第十一章　Test-Time Scaling 与验证

提出动机：不增加参数，也能否用更多推理计算换质量？

训练完成后，模型参数固定，但系统仍可增加单轨迹长度、候选数量、搜索宽度和验证次数。测试时扩展（test-time scaling）研究如何分配这些推理预算，并关注质量提升是否足以覆盖延迟和成本。

测试时计算的主要形式包括：

方法	增加的计算	有效前提
延长单轨迹	更多中间步骤	模型能利用长轨迹且会自我修正
多次采样	生成 N 个候选	候选错误不完全相关
自洽性投票	对答案聚合或多数投票	正确答案较稳定、可规范化
Best-of-N	用验证器选最佳候选	验证器排序与真实质量一致
搜索	扩展并剪枝中间状态	状态可比较，分支与预算可控
工具验证	执行代码、计算或查询	工具反馈可靠且接口安全

表 11.1　常见测试时扩展方式及其前提。

11.1　多次采样、自洽性与 Best-of-N

若单次采样正确率为 p，并且 N 次采样近似独立，则“至少出现一个正确候选”的概率为

$$P(\text{至少一个正确}) = 1 - (1-p)^N.$$

这个公式只描述**候选覆盖率**，不等于系统最终正确率。系统还必须识别哪个候选正确；若验证器不可靠，更多候选反而增加被错误高分样本欺骗的机会。实际样本也高度相关，因此独立假设常高估收益。

自洽性（self-consistency）让模型用不同采样路径得到多个答案，再对规范化后的最终答案投票。它可能有效，是因为正确推理往往汇聚到同一结果，而错误路径更分散；但若模型存在系统性偏差，多数路径会一起犯错，投票无法纠正。

Best-of-N 生成 N 个候选，再用奖励模型、规则或测试用例选择最高分。其前提不是“N 越大越好”，而是候选分布有足够多样性、验证器与真实质量对齐、额外成本可接受。固定验证器下无限增加 N 还可能产生验证器过优化：模型偶然生成专门钻评分漏洞的候选。

计算题：多采样提高的是候选覆盖率

若单次正确率 $p = 0.30$，采样 $N = 8$ 次，在独立假设下至少包含一个正确答案的概率为

$$1 - (1 - 0.30)^8 = 1 - 0.7^8 \approx 0.942.$$

这不意味着最终正确率就是 94.2%。若验证器只能以 80% 的概率从候选中选对，或八个样本高度相关，最终收益会明显更低。

11.1.1 对应实现：Best-of-N 的系统边界

```python
def best_of_n(prompt, model, verifier, n=8):
    # 多次 rollout 可以并行，但总生成 token 和显存占用会增加。
    candidates = [model.generate(prompt) for _ in range(n)]

    # verifier 可以是测试用例、规则或奖励模型。
    # 它必须只读取允许的信息，并返回可比较分数。
    scores = [verifier(prompt, answer) for answer in candidates]

    # 选中的是 verifier 最高分，而不必是真实质量最高的候选。
    # 因此验证器偏差会直接成为系统上限。
    best_index = max(range(n), key=lambda i: scores[i])
    return candidates[best_index], scores[best_index]
```

面试深挖：为什么 Best-of-N 可能提高性能？

20 秒回答：多次采样扩大候选覆盖，验证器再从中选择更优答案。它有效的前提是候选具有多样性，而且验证器的排序与真实质量相关；否则只是在增加重复样本或放大评分器偏差。

深入解释：$1 - (1 - p)^N$ 说明候选覆盖可随 N 增长，但边际收益递减。最终性能还要乘上选择器识别正确候选的能力。验证器过弱时，增加 N 可能出现“选择者诅咒”：极端高分更可能来自评分噪声或 reward hacking。

面试官继续追问：

1. self-consistency 与 Best-of-N 的聚合方式有何区别?
2. 样本相关性为什么会降低理论收益?
3. verifier 不可靠会发生什么?
4. 如何比较增加模型参数与增加测试时计算?

容易答错

错误说法：测试时计算越多，质量就会单调且无限提高。

更准确：更多预算只提供更大的搜索机会。收益受到模型探索能力、候选相关性、验证器准确率和任务可验证性的共同限制，通常边际递减，并可能因过度思考或验证器漏洞而下降。

第十二章　RLVR：可验证奖励强化学习

提出动机：开放式偏好难以精确，但有些任务可以自动判对错

人类偏好奖励主观且昂贵，而数学答案、代码测试、定理检查和结构化约束常有确定性验证器。可验证奖励强化学习（RL with Verifiable Rewards，RLVR）使用这些检查结果为策略提供反馈，使模型可以通过大量在线采样探索更好的求解行为。

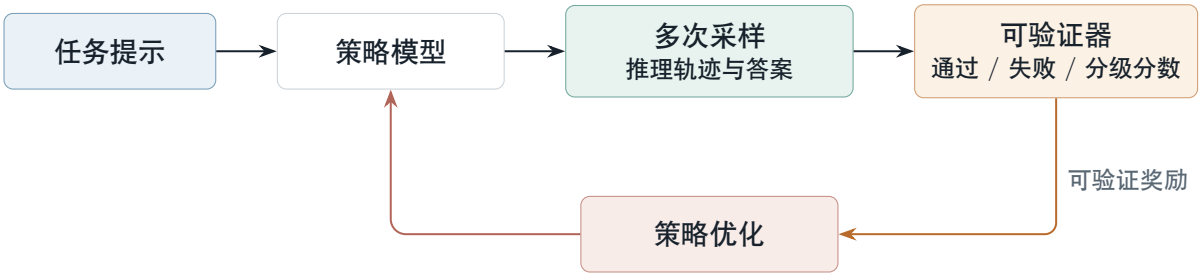


图 12.1　RLVR 的基本闭环。验证器评价最终结果或过程步骤，多次 rollout 为策略优化提供可比较的探索样本。

对提示词 x，策略采样推理与答案 $y \sim \pi_\theta(\cdot \mid x)$，验证器给出

$$r(x, y) \in \{0, 1\} \quad \text{或更细的分级分数。}$$

数学题可验证最终数值或符号等价性，代码题可运行隔离测试，定理证明可交给形式化检查器，结构化任务可检查语法与约束。开放式写作没有唯一正确输出，很难仅用确定性检查器评价事实性、风格、创造性和帮助程度，因此通常仍需要人类偏好、奖励模型或多维规则。

12.1　Outcome、Process 与稀疏奖励

结果监督（outcome supervision）只检查最终答案，成本低且目标清楚，但无法区分“过程大体正确、最后算错”和“过程完全错误、偶然猜对”；模型还可能找到绕过验证器的捷径。**过程监督（process supervision）**为中间步骤提供反馈，信用分配更细，却需要可靠的步骤定义与标注，并可能把模型束缚在特定解题风格上。

若长轨迹只有最终通过时得到 1，绝大多数探索都得到 0，就形成稀疏奖励。策略很难判断哪一步值得保留，梯度估计方差也高。常见缓解思路包括：从较容易任务开始的课程学习、一次提示采样多个 rollout、提供可验证的部分分、保持足够探索熵，以及使用组内相对基线降低方差。

多 rollout 对 reasoning RL 有两层价值：一是提高偶然发现正确解的概率，二是为同一提示词提供可比较的好坏样本。GRPO 用同组奖励均值和标准差构造相对优势，正适合“一题多答”的数据形态；但若整组奖励相同，组内优势仍为零，不能凭空产生学习信号。

监督方式	优点	风险
最终结果奖励	自动、便宜、目标明确	稀疏、信用分配差、可能偶然猜对
过程奖励	步骤级反馈更密集	标注昂贵、步骤规范可能有偏差
奖励模型	可覆盖开放任务	评分偏差与 reward hacking
确定性验证器	可复现、难以主观漂移	只覆盖可形式化检查的维度

表 12.1 不同推理监督信号的取舍。

面试深挖：为什么数学和代码特别适合 RLVR？

20 秒回答：它们常有低成本、可重复的外部验证器：数学答案可代入或做符号检查，代码可编译并跑测试。模型能大规模采样，再从明确通过/失败反馈中优化策略，而不必为每条轨迹请人打分。

深入解释：可靠验证器降低了奖励模型主观偏差，但最终奖励仍可能稀疏。通过测试也不一定代表实现优雅、安全或没有测试外漏洞，因此验证器覆盖面决定可学目标。开放式写作缺少唯一判据，不适合只用单一确定性奖励。

面试官继续追问：

1. outcome reward 的信用分配问题是什么？
2. process reward 有哪些新风险？
3. sparse reward 为什么让探索困难？
4. 多 rollout 与 GRPO 有什么关系？

面试深挖：验证器可靠是否就不会 reward hacking？

20 秒回答：不会。验证器只检查被编码的条件，模型可能找到未覆盖的捷径，例如硬编码样例、利用测试漏洞或输出满足格式但语义错误的结果。可验证不等于验证完备。

深入解释：奖励优化会把策略推向验证器的边界。需要隐藏测试、对抗样例、沙箱、规则组合和人工抽检；还应监控训练奖励与独立评测是否分离。过程奖励也会被利用，只是漏洞从最终检查转移到步骤评分。

面试官继续追问：

1. 如何设计代码任务的防泄漏测试？

2. curriculum 为什么有助于稀疏奖励?

3. 奖励越来越高但独立正确率下降说明什么?

第十三章　推理蒸馏

提出动机：昂贵推理能否转移到更小、更快的模型？

大型教师模型可以用长轨迹、多采样和验证生成高质量训练数据，但在线部署每次都承担同样预算过于昂贵。推理蒸馏尝试把教师发现的解题模式压缩到学生参数中，使学生用更少测试时计算复现部分能力。

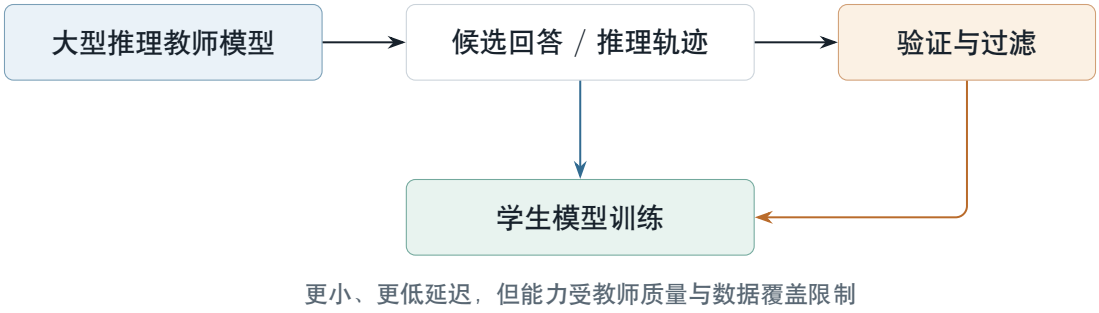


图 13.1　推理蒸馏的概念流程。可训练数据必须经过质量检查；教师的错误轨迹也可能被学生继承。

普通 logits 蒸馏让学生匹配教师的软类别分布；回答蒸馏用教师生成的输入–输出对训练学生；推理蒸馏还可能使用经过过滤的中间步骤或结构化求解轨迹。只蒸馏最终答案数据简单、暴露错误过程较少，但学生看不到如何到达答案；蒸馏轨迹能提供更密集监督，却会把教师的冗余、风格偏差和错误推理一起传递。

推理蒸馏能让小模型变快，是因为部分搜索工作被“摊销”进训练：教师离线探索许多路径，学生在参数中学习常见映射，在线时不必重复同样规模的搜索。但学生容量、训练分布和验证过滤决定上限；蒸馏不能保证小模型在分布外任务上复现教师的全部探索能力。

面试深挖：为什么 reasoning 能力可以蒸馏给小模型？

20 秒回答：教师用高预算产生并验证求解示范，学生通过监督学习把这些常见推理模式压缩进参数。在线时学生可用更短轨迹直接复现部分解法，因此延迟更低；但它继承教师和过滤器的偏差，也受学生容量限制。

深入解释：最终答案蒸馏提供硬目标，轨迹蒸馏提供步骤级条件分布，logits 蒸馏还保留候选间相对信息。高质量过滤十分关键：错误但流畅的轨迹会成为强监督噪声，甚至让学生学会合理化错误答案。

面试官继续追问：

1. 推理蒸馏与普通 logits 蒸馏有什么区别?
2. 只蒸馏答案和蒸馏轨迹各有什么取舍?
3. 为什么蒸馏模型可能更短、更快?
4. teacher 的错误 reasoning 会发生什么?

常见误区

不要把“教师生成了长解释”直接等同于“高质量推理数据”。商业训练流水线还需要答案验证、轨迹去重、难度配比、污染检测和安全过滤；具体私有模型的训练配方不能从公开输出反推。

第六部分

后训练

第十四章　监督微调与奖励模型

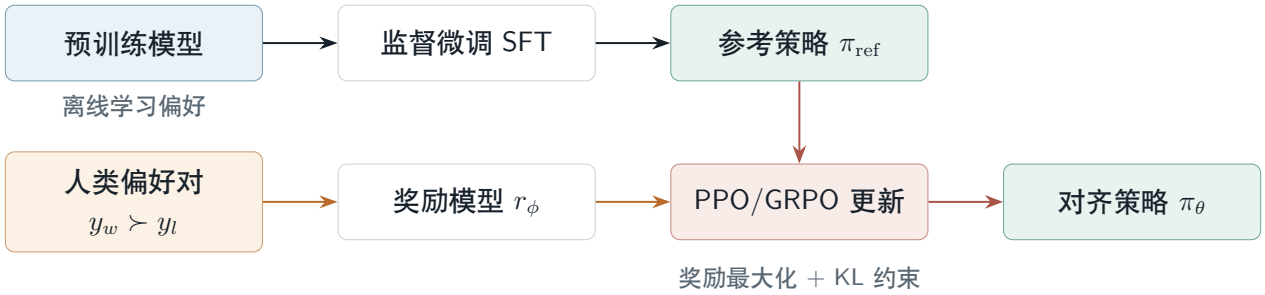


图 14.1　经典 RLHF 流程。SFT 提供参考策略，偏好对训练奖励模型，强化学习在奖励与 KL 约束之间更新策略。

14.1　监督微调（SFT）

提出动机：预训练会续写文本，但不会天然遵循指令

预测下一个词元的预训练学到的是互联网文本分布。面对用户问题，模型可能继续补写问题、模仿网页格式，或生成与任务无关的延续，而不是稳定地扮演助手。监督微调（Supervised Fine-Tuning，SFT）用“指令–回答”示范重新塑造条件分布，使模型学会任务格式、回答风格和基本指令遵循。

SFT 使用“指令–回答”数据继续计算下一个词元损失。常见做法是只在回答词元上计算损失，提示词词元作为条件但不计入监督，避免模型学习“复述用户问题”。

```
import torch.nn.functional as F

# input_ids: [B, T]，包含 prompt 与 response。
# labels: [B, T]；prompt、padding 位置已填成 -100。
logits = model(input_ids)  # [B, T, V]

# 与预训练一样做 next-token shift。
shift_logits = logits[:, :-1, :].contiguous()
shift_labels = labels[:, 1:].contiguous()

loss = F.cross_entropy(
    shift_logits.view(-1, shift_logits.size(-1)),
    shift_labels.view(-1),
    # 被设为 -100 的 prompt 与 padding 不贡献 loss。
    ignore_index=-100,
)
```

“提示词不计入损失”不等于“模型看不到提示词”。提示词词元仍参与前向并生成 K, V，回答位置也仍可注意它们；只是提示词位置对应的预测误差不参与梯度平均。这样训练目标集中在学习如何回答，而不是要求模型复现用户输入。

具体例子：SFT 中哪些词元参与损失

把样本简化为

$$[\text{用户:}, \text{法国首都?}, \text{助手:}, \text{巴黎}, \text{EOS}].$$

输入序列保持完整，因此生成“巴黎”时仍能注意“法国首都?”。但 labels 可设为

$$[-100, -100, -100, \text{巴黎}, \text{EOS}],$$

其中 -100 的位置被交叉熵忽略。这样模型只因回答“巴黎”和正确结束而获得监督，不会被要求预测用户问题本身。若连回答也误设为 -100，该样本虽然完成前向，却不会产生有效训练梯度。

面试深挖：为什么提示词可以 mask loss，却仍参与 attention？

20 秒回答：提示词词元仍在输入中，产生隐藏状态和 K/V，回答位置可以读取它们；mask 只把提示词位置对应的预测标签设为忽略值，不让这些位置的交叉熵进入损失。条件信息保留了，只是不训练模型复述用户输入。

深入解释：若把提示词从输入中删掉，回答位置失去任务条件，训练目标变成无条件生成。assistant-only loss 聚焦回答行为；full-sequence loss 也训练提示词本身的语言建模，可能浪费容量或学习复现模板，但在某些持续预训练混合目标中仍有用途。

面试官继续追问：

1. mask label 与 attention mask 有什么区别?
2. prompt K/V 会不会收到来自回答损失的梯度?
3. assistant-only loss 与 full-sequence loss 如何选择?
4. SFT 能否学到示范中完全没有的信息?

SFT 的风险是灾难性遗忘（catastrophic forgetting）：过窄的指令数据可能损伤预训练能力。因此通常混入少量通用数据、使用较小学习率，或采用低秩适配（Low-Rank Adaptation，LoRA）等参数高效微调方法。

全量微调还要求保存全部参数对应的梯度和优化器状态，在多任务或多客户场景中每个版本都复制完整模型。LoRA 因此把原权重矩阵 W 的更新写成低秩矩阵 $\Delta W = BA$，其中 A 和 B 是新增的可训练小矩阵。它只训练少量参数，

解决训练和存储成本；但秩（rank）太小会限制可表达的更新方向，也不能保证消除遗忘。

方式	优点	风险
全量微调	容量最大，行为改变充分	显存高，容易遗忘
LoRA	训练参数少，部署可合并	秩太小会欠拟合
适配器（Adapter）	模块化强	推理结构更复杂

表 14.1 LoRA 与全量微调的核心权衡。

14.2 奖励建模（Reward Modeling）

提出动机：示范数据难以写出，但偏好比较更容易获得

对开放问题，唯一“标准答案”往往不存在，人工持续编写高质量完整回答成本很高；相比之下，让标注者在两个候选中选择更好的一个更容易且一致性更高。奖励模型把这种成对偏好压缩为可比较的标量，为后续策略优化提供自动反馈。

偏好数据写作 (x, y_w, y_l)，其中 x 是提示词，y_w 是被选中的较优回答，y_l 是被拒绝的较差回答。参数为 ϕ 的奖励模型对回答 y 输出标量 $r_\phi(x, y)$。Bradley–Terry 模型假设偏好概率为：

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right).$$

最大化该概率等价于最小化：

$$\mathcal{L}_{\mathrm{RM}} = -\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right).$$

这里 $\sigma(a) = 1/(1 + e^{-a})$ 是逻辑函数。若较优回答的奖励高于较差回答，差值为正，损失变小；反之损失增大并推动模型拉开奖励差。

令 $\Delta r = r_\phi(x, y_w) - r_\phi(x, y_l)$，则

$$\frac{\partial \mathcal{L}_{\mathrm{RM}}}{\partial \Delta r} = \sigma(\Delta r) - 1 = -\sigma(-\Delta r).$$

其中 Δr 是两条回答的奖励差。当排序错误且 $\Delta r \ll 0$ 时，梯度绝对值接近 1；排序已正确且间隔（margin）足够大时，梯度逐渐趋近 0。这说明 Bradley–Terry 损失学的是相对顺序，而不是奖励的绝对标尺。

具体例子：奖励差如何变成偏好概率

若较优回答的奖励为 $r_w = 2.0$，较差回答为 $r_l = 0.5$，则奖励间隔为 1.5，模型预测较优回答获胜的概率是

$$\sigma(1.5) \approx 0.818, \qquad \mathcal{L} = -\log 0.818 \approx 0.201.$$

若排序反了，$r_w = 0.5, r_l = 2.0$，概率变为 $\sigma(-1.5) \approx 0.182$，loss 增大到约 1.701。训练因此只关心两者差值；把两个奖励同时加 100，概率完全不变。

14.2.1 对应实现：奖励模型成对损失

```python
import torch.nn.functional as F

def pairwise_reward_loss(chosen_reward, rejected_reward):
    # 两个输入均为 [B]，每个回答最终只对应一个标量奖励。
    reward_margin = chosen_reward - rejected_reward

    # -log(sigmoid(margin)) 的稳定写法。
    # 不要先手动 sigmoid 再取 log，否则大幅值时更易下溢。
    per_pair_loss = -F.logsigmoid(reward_margin)
    return per_pair_loss.mean()
```

奖励值整体加上同一个常数不会改变差值，也不会改变损失，所以奖励模型没有天然的绝对零点。真正参与偏好判断的是奖励间隔；部署时若把奖励绝对值当成跨模型、跨数据集可直接比较的分数，往往会得出错误结论。

奖励模型只学会“评价”，还不会直接改变生成策略。由于采样出的词元是离散动作，奖励也通常在整段回答结束后才能得到，不能像普通监督学习那样直接对文本求导；这就引出了策略梯度与 PPO。

面试深挖：奖励模型为什么输出一个标量？

20 秒回答：后续策略优化需要对完整回答做可比较的偏好排序，标量奖励把多维偏好压缩成统一优化信号。成对 Bradley–Terry 损失只要求较优回答的标量更高，不要求奖励具有绝对零点。

深入解释：标量便于排序和策略梯度，但也会丢失帮助性、事实性、安全性等维度之间的结构。成对比较通常比让标注者给绝对分数更一致，因为人更擅长判断 A/B 哪个好；模型仍可能学习标注偏差，并被策略通过分布外输出利用。

面试官继续追问：

1. reward 的绝对值重要吗?
2. pairwise ranking 为什么往往比直接打分稳定?
3. Bradley–Terry 假设了什么?
4. reward hacking 与过拟合奖励模型有什么区别?

第十五章　PPO、DPO 与 GRPO



图 15.1　PPO 与 GRPO 的优势估计差异。两者都可使用概率比裁剪，但 GRPO 用组均值替代单独训练的价值基线。

15.1　近端策略优化（PPO）与人类反馈强化学习

提出动机：把不可微的整段奖励传回生成策略

奖励模型给出的是回答级标量，而策略需要决定每个词元的概率。策略梯度可以用采样估计把奖励传回动作概率，但直接更新方差大，而且一次过强更新可能让新策略远离收集数据的旧策略，导致训练失稳或钻奖励模型漏洞。近端策略优化（Proximal Policy Optimization，PPO）用优势基线降低方差，用概率比裁剪限制单轮策略变化，并用参考模型的 KL 散度保持语言质量。

在人类反馈强化学习（Reinforcement Learning from Human Feedback，RLHF）中，参数为 θ 的策略 π_θ 生成回答，奖励模型给出回答级奖励，参考策略 π_{ref} 用 KL 约束防止策略偏离 SFT 模型过远。对第 t 个生成位置，状态 s_t 是“提示词加上已经生成的回答前缀”，动作 a_t 是当前选择的词元。PPO 使用旧策略 $\pi_{\theta_{\text{old}}}$ 收集样本，并定义概率比：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

这里 $r_t(\theta)$ 表示新旧策略对同一动作的概率之比，**不是奖励**；$r_t > 1$ 表示新策略提高了该动作概率。若优势 $A_t > 0$，希望提高该动作概率；若 $A_t < 0$，希望降低。直接优化 $r_t A_t$ 可能一步更新过大，因此裁剪代理目标为：

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon)A_t\right)\right].$$

其中 $\epsilon > 0$ 是允许概率比变化的裁剪宽度，期望 $\mathbb{E}$ 对采样到的有效动作取平均；训练希望最大化 L^{CLIP}。裁剪形成近似的信赖域（trust region），限制单轮策略变化。

具体例子：PPO clip 如何阻止一次更新过大

某词元在旧策略中的概率为 0.20，新策略提高到 0.30，所以概率比为 $r = 1.5$。若优势 $A = 2 > 0$、$\epsilon = 0.2$，未裁剪收益为 $rA = 3.0$，裁剪后为

$$\text{clip}(1.5, 0.8, 1.2) \times 2 = 2.4.$$

目标取二者较小值 2.4，因此继续把概率从 0.20 大幅推高不会再获得额外收益。若 $A < 0$ 且概率朝错误方向变化，min 仍会保留惩罚，这就是 PPO 不是简单“把所有概率比截断”的原因。

15.1.1 对应实现：PPO 裁剪策略损失

```python
import torch

def ppo_policy_loss(
    new_logp, old_logp, advantages, valid_mask, clip_eps=0.2
):
    # new_logp/old_logp: [B, T]，动作 token 的对数概率。
    # advantages: [B, T]；实际使用前通常已经 detach。
    # valid_mask: [B, T]，排除 prompt 与 padding token。

    # 概率相除在 log 空间中写成先相减再 exp，更稳定。
    ratio = torch.exp(new_logp - old_logp)

    unclipped = ratio * advantages
    clipped_ratio = ratio.clamp(1.0 - clip_eps, 1.0 + clip_eps)
    clipped = clipped_ratio * advantages

    # PPO 最大化两者较小值；训练代码用负号改成最小化。
    token_loss = -torch.minimum(unclipped, clipped)

    # 只平均有效 response token，避免 padding 改变 loss 尺度。
    token_loss = token_loss * valid_mask
    return token_loss.sum() / valid_mask.sum().clamp_min(1)
```

裁剪不是简单地把所有概率比截到区间内。目标取 min，只在更新会让代理目标“过度变好”时截断收益；如果策略朝错误方向移动，目标仍保留惩罚梯度。因此裁剪是保守更新机制，而不是把梯度普遍裁小。

15.1.2 旧策略与参考策略不是同一个概念

PPO/RLHF 中常同时出现两份“固定模型”，但作用不同：

- **旧策略** $\pi_{\theta_{\text{old}}}$**：**生成当前 rollout，并作为 PPO 重要性采样概率比的分母；每轮优化后会周期性更新为新的策略快照。
- **参考策略** π_{ref}**：**通常是冻结的 SFT 基线，用 KL 惩罚限制长期行为漂移；它不会因为每轮 PPO 更新而同步前进。

训练开始时二者可能来自同一检查点，但语义和生命周期不同。若把 reference 也随策略每轮更新，KL 锚点会移动，无法约束模型长期偏离原有语言分布；若用一个很旧的策略做 PPO ratio 分母，重要性权重会极端，代理目标变得不可靠。

面试深挖：PPO 为什么同时需要 old policy 和 reference policy？

20 秒回答：old policy 是短期优化基线，负责说明当前数据由什么策略采样，并进入 probability ratio；reference policy 是长期行为锚点，通过 KL 防止策略远离 SFT 模型。二者初始可能相同，但更新频率和数学职责不同。

深入解释：$\pi_{\theta_{\text{old}}}$ 解决 on-policy 数据在多步更新中的分布校正，PPO clip 限制相对它的单轮变化；π_{ref} 解决奖励最大化导致的长期漂移和语言质量退化。PPO clipping 也不是严格 KL 约束，它只裁剪采样动作上的代理收益，仍需监控实际 KL。

面试官继续追问：

1. probability ratio 为什么不是 reward?
2. clipping 限制的是参数距离还是动作概率变化?
3. reference model 可以多久不更新?
4. old policy 与当前 policy 相差太大会怎样?

概念	含义
奖励（reward）	环境或奖励模型给出的即时标量
回报（return）	从当前步开始的累计折扣奖励
状态价值 $V(s)$	状态 s 下未来回报的期望
动作价值 $Q(s,a)$	执行动作 a 后未来回报的期望
优势 A	动作比平均行为好多少，常为 $Q(s,a)-V(s)$

表 15.1 RLHF 中容易混淆的概念。

15.2 直接偏好优化（DPO）

提出动机：能否绕开昂贵且脆弱的在线强化学习？

PPO 通常要同时维护策略、参考模型、奖励模型和价值模型，还要在线采样并反复协调多种损失，显存、系统和调参成本都很高。直接偏好优化（Direct Preference Optimization，DPO）从 KL 正则化最优策略中反解隐式奖励，把偏好学习直接化为“较优回答/较差回答”的分类式目标，从而保留参考策略约束并省去显式奖励模型与在线强化学习环路。

DPO 的目标是直接用偏好数据优化策略，而不显式训练奖励模型。关键直觉是：在 KL 约束下，最优策略相对参考策略的对数概率比可解释为隐式奖励：

$$r(x,y)=\beta\log\frac{\pi_\theta(y\mid x)}{\pi_{\text{ref}}(y\mid x)}+C(x).$$

其中 x 是提示词，y 是一条回答，π_θ 是当前策略，π_{ref} 是冻结的参考策略，β 控制偏离参考策略的代价，$C(x)$ 是只与提示词有关的常数。代入偏好概率，常用 DPO 损失为：

$$\mathcal{L}_{\text{DPO}}=-\log\sigma\left[\beta\left(\log\frac{\pi_\theta(y_w\mid x)}{\pi_{\text{ref}}(y_w\mid x)}-\log\frac{\pi_\theta(y_l\mid x)}{\pi_{\text{ref}}(y_l\mid x)}\right)\right].$$

具体例子：DPO 比较的是相对参考模型的改变量

设当前策略对较优/较差回答的序列对数概率分别为 $-2.0,-3.0$，策略间隔为 1.0；参考模型对应值为 $-2.4,-2.8$，参考间隔为 0.4。当前策略相对参考模型多偏向较优回答的程度为

$$1.0-0.4=0.6.$$

若 $\beta=0.1$，DPO 的分类分数为 0.06，损失为 $-\log\sigma(0.06)\approx0.664$。若只看当前策略间隔而不减参考间隔，就无法区分“策略新学到的偏好”和“参考模型原本已有的偏好”。

关键推导

从单个提示词上的 KL 正则化目标出发：

$$\max_{\pi} \sum_{y} \pi(y \mid x) r(x, y) - \beta \sum_{y} \pi(y \mid x) \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)}, \quad \text{s.t.} \quad \sum_{y} \pi(y \mid x) = 1.$$

加入拉格朗日乘子并对 $\pi(y \mid x)$ 求导，可得

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) e^{r(x,y)/\beta}.$$

反解奖励即

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

这里 $\pi(y \mid x)$ 是待优化的回答分布，$r(x, y)$ 是回答奖励，约束保证所有回答概率之和为 1；$Z(x)$ 是归一化常数，π^* 是该目标的最优策略。同一提示词下对较优/较差回答做差时，未知的 $\log Z(x)$ 正好抵消，从而得到上面的 DPO 损失。

β 是原 KL 正则目标中的惩罚系数：较大时更强地约束策略靠近参考模型，较小时允许更大偏移。实现中它也会缩放 DPO 的分类分数，因此应结合学习率、数据噪声与实际 KL 一起调节。

15.2.1 对应实现：DPO 成对偏好损失

```python
import torch.nn.functional as F

def dpo_loss(
    policy_chosen_logp,
    policy_rejected_logp,
    ref_chosen_logp,
    ref_rejected_logp,
    beta=0.1,
):
    # 四个输入均为 [B]。
    # 每个值是 response 所有有效 token 的 log-prob 之和。

    # 当前策略更偏爱 chosen 的程度。
    policy_margin = policy_chosen_logp - policy_rejected_logp

    # 参考策略原本就可能更偏爱 chosen，必须减掉这部分基线。
    reference_margin = ref_chosen_logp - ref_rejected_logp

    # 正值表示当前策略相对参考策略更偏向 chosen。
```

```
    preference_logit = beta * (policy_margin - reference_margin)

    # 最大化 sigmoid(preference_logit) 的稳定负对数形式。
    return -F.logsigmoid(preference_logit).mean()
```

这里的对数概率必须只汇总回答中的有效词元，不能把提示词或补齐位置算进去。参考模型也必须使用同一分词器、同一截断和同一遮罩，否则两组对数概率比不可比。训练时参考模型参数冻结，但其对数概率仍是 DPO 基线的一部分。

DPO 简化了系统，但只能从固定偏好对学习，难以在训练过程中主动探索新回答；当任务具有可自动验证的奖励，例如数学答案或代码测试时，在线采样仍然有价值。GRPO 正是在保留在线策略优化的同时，尝试去掉昂贵价值模型。

面试深挖：DPO 为什么不训练显式奖励模型，却仍需要参考策略？

20 秒回答： DPO 把 KL 正则化最优策略中的隐式奖励写成当前策略相对参考策略的对数概率比，再直接对偏好对做分类式优化。参考策略提供“原本有多偏好”的基线并约束漂移；没有它，目标会退化成只拉大当前策略的 chosen/rejected 间隔。

深入解释： β 同时缩放相对参考策略的偏好 logit。过大时目标更保守、梯度尺度也会变化；过小时允许更大偏移，但更容易过拟合偏好数据。DPO 源自 KL 正则化 RL 目标，但训练过程是离线可微的成对似然优化，没有在线环境交互；是否把它简称为“RL”应先说明所指的是目标来源还是优化过程。

面试官继续追问：

1. reference policy 去掉会怎样？
2. β 太大或太小分别意味着什么？
3. DPO 与普通 pairwise classification 的区别在哪里？
4. DPO 为什么难以主动探索新回答？

15.3 组相对策略优化（GRPO）

提出动机：在线 RL 中价值模型成本高且未必容易学准

PPO 的价值模型要为开放式语言状态估计未来回报，需要额外参数、激活和训练目标；在稀疏的回答级奖励下，这个估计本身也可能不稳定。组相对策略优化（Group Relative Policy Optimization，GRPO）对同一提示词采样多个候选，用组内相对奖励充当基线（baseline），借候选之间的比较降低方差，从而省去独立价值模型。

GRPO 对同一提示词采样 G 个回答并得到奖励 $r_1, \dots, r_G$。这里 G 是组内回答数，r_i 是第 i 个回答的奖励。它用组内归一化构造相对优势：

$$\bar{r} = \frac{1}{G}\sum_{i=1}^{G} r_i, \qquad A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}.$$

具体例子：三个回答怎样产生组内优势

同一提示词的三个回答奖励为 $[1, 3, 2]$。组均值 $\bar{r} = 2$，总体标准差

$$\sigma_r = \sqrt{\frac{(1-2)^2 + (3-2)^2 + (2-2)^2}{3}} \approx 0.816.$$

忽略很小的 ϵ，三个优势约为

$$[-1.225,\ 1.225,\ 0].$$

第二个回答高于同组平均，因此其词元概率会被提高；第一个被降低；第三个接近基线。奖励的绝对值若整体加 10，优势不变，这正是“组内相对”二字的含义。

其中 $\bar{r}$ 是组内平均奖励，σ_r 是组内奖励标准差，A_i 是第 i 个回答的标准化优势，ϵ 是防止除零的小常数。这样可以不训练显式价值模型，节省显存和训练复杂度。风险是组内奖励方差过小、奖励稀疏或噪声大时，优势估计会不稳定，因此仍需要 KL 控制、足够采样数和奖励归一化。

对第 i 个回答的第 t 个词元，记新旧策略概率比

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}.$$

GRPO 可使用与 PPO 类似的裁剪代理目标

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t}\min(\rho_{i,t}A_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)A_i) + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}).$$

式中，$y_{i,t}$ 是第 i 个回答的第 t 个词元，$y_{i,<t}$ 是它之前的回答前缀，$|y_i|$ 是该回答的有效长度；$\rho_{i,t}$ 是新旧策略概率比，ϵ 是裁剪宽度，β 是参考策略 KL 惩罚系数。组内标准化解决的是基线估计，不会自动限制策略漂移；最后的 KL 项仍然有独立作用。

15.3.1 对应实现：GRPO 组内优势

```python
import torch

def group_relative_advantages(rewards, eps=1e-6):
    # rewards: [B, G]
    # B 个 prompt，每个 prompt 有 G 个候选回答。
    group_mean = rewards.mean(dim=1, keepdim=True)

    # unbiased=False 使用总体标准差；G 较小时更适合此处归一化。
    group_std = rewards.std(
        dim=1, keepdim=True, unbiased=False
    )

    # 同一 prompt 内高于均值为正优势，低于均值为负优势。
    advantages = (rewards - group_mean) / (group_std + eps)
    return advantages  # [B, G]
```

优势的比较范围只能是同一提示词内部，因为不同提示词的奖励难度和标尺可能不同。当一组回答奖励完全相同时，分子为零，优势也全部为零；ϵ 只能防止除零，不能凭空创造学习信号。因此 GRPO 依赖组内存在可辨别的质量差异。

面试深挖：GRPO 为什么可以不训练独立价值模型？

20 秒回答：它对同一提示词采样多个回答，用组内奖励均值和标准差构造相对优势，把同组候选当作彼此的基线。因此不需要另一个网络预测 $V(s)$，但仍然存在 baseline，并不是“没有 critic 就没有基线”。

深入解释：组内中心化降低不同题目奖励尺度带来的方差。组大小增加通常改善均值和方差估计，也提高探索覆盖，但 rollout 成本线性上升；候选高度相关时边际收益有限。若奖励标准差很小，除法会放大噪声；若全相等，优势全为零。稀疏奖励下，一整组都失败是常见挑战。

面试官继续追问：

1. 为什么每个 prompt 要生成多个 responses?
2. group size 增大会怎样?
3. reward std 很小时如何处理?
4. GRPO 与 PPO 最本质的区别是什么?
5. GRPO 是否完全不需要 critic-like signal?

容易答错

错误说法： GRPO 没有价值模型，所以没有 baseline。

更准确： GRPO 没有独立训练的 value model，但组均值 $\bar{r}$ 就承担了 baseline 的角色。它把回答奖励变成同题候选之间的相对优势，仍需要 KL、裁剪或其他机制限制策略漂移。

SFT、PPO、DPO 与 GRPO 不是简单的新旧替代关系：SFT 提供稳定行为起点，PPO 适合需要在线探索且能承担复杂系统的场景，DPO 适合高质量离线偏好对，GRPO 则适合能批量采样并获得可靠可验证奖励的任务。选择方法前应先判断反馈从哪里来、是否需要探索，以及能否承担额外模型和采样成本。

方法	数据	是否需奖励模型	是否需价值模型	典型特点
SFT	指令回答	否	否	简单稳定，受数据质量限制
PPO	在线采样 + 奖励	是	通常是	强但复杂，显存和调参成本高
DPO	偏好对	否	否	离线稳定，依赖参考模型对数概率
GRPO	多回答奖励	可用规则或奖励模型	否	省价值模型，适合可验证奖励

表 15.2 SFT、PPO、DPO 与 GRPO 对比。

面试高频问题

Q: DPO 和 PPO 最大区别是什么？

20 秒回答：PPO 把偏好转成奖励再做强化学习；DPO 直接把偏好对转成策略相对参考策略的对数概率优化目标，省掉在线强化学习和价值模型。

Q: GRPO 为什么能省显存？

20 秒回答：它用同一提示词多个回答的组内奖励构造优势，不需要额外价值模型，少存一套模型与相应激活。

第七部分

参数高效微调与蒸馏

第十六章　LoRA 与 QLoRA

提出动机：每个任务都复制并更新完整模型过于昂贵

全量微调需要为全部权重保存梯度和优化器状态，并为每个任务维护完整检查点。参数高效微调（Parameter-Efficient Fine-Tuning，PEFT）希望冻结大部分基础模型，只训练少量任务相关参数。LoRA 的关键假设是：预训练模型适配到某个下游任务时，所需权重更新常集中在较低维子空间，可以用低秩矩阵近似。

采用列向量约定，原线性层权重为

$$W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \qquad y = Wx.$$

低秩适配（Low-Rank Adaptation，LoRA）冻结 W，只学习

$$W' = W + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA,$$

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad r \ll \min(d_{\text{in}}, d_{\text{out}}).$$

这里 r 是低秩维度，α 是缩放超参数，α/r 让更新幅度与 rank 的变化更容易解耦；A 先把输入压到 r 维，B 再映射回输出维。不同实现的矩阵命名或转置约定可能相反，但参数量始终是

$$r(d_{\text{in}} + d_{\text{out}}),$$

而不是 $d_{\text{in}} d_{\text{out}}$。

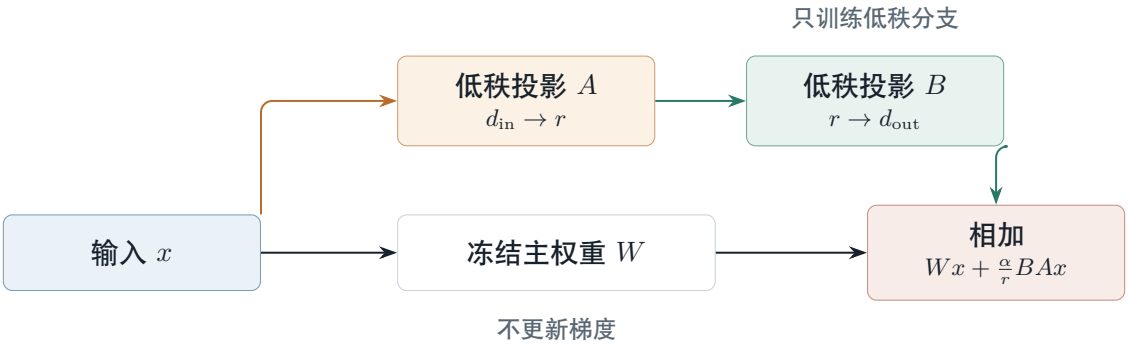


图 16.1　LoRA 在冻结主权重旁增加低秩更新分支。部署时可把 $\frac{\alpha}{r} BA$ 合并进原权重。

LoRA 常让一个低秩矩阵随机初始化、另一个初始化为零，使训练开始时 $\Delta W = 0$，模型初始行为与基础模型完全一致，同时非零分支保证后续梯度能传播。若 A 与 B 都初始化为零，第一步两边梯度也可能同时为零，无法启动学习。

计算题：4096 维方阵的 LoRA 参数量

给定

$$W \in \mathbb{R}^{4096\times 4096}, \qquad r = 16.$$

全量矩阵有

$$4096^2 = 16{,}777{,}216$$

个参数；LoRA 低秩分支有

$$16(4096 + 4096) = 131{,}072$$

个参数，仅为完整矩阵的 1/128。这个比例描述该层的可训练权重，不代表整次训练显存也精确下降 128 倍，因为基础权重、激活和临时缓冲仍然存在。

16.1 对应实现：带缩放与零更新初始化的 LoRA 线性层

```
import math
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, rank=16, alpha=32.0):
        super().__init__()
        self.base = base
        self.rank = rank
        self.scale = alpha / rank

        # 冻结基础权重；训练时只更新 A、B。
        for parameter in self.base.parameters():
            parameter.requires_grad = False

        # PyTorch Linear 使用 y = x @ W.T。
        # A: [r, d_in], B: [d_out, r]。
        self.lora_a = nn.Parameter(
            torch.empty(rank, base.in_features)
        )
        self.lora_b = nn.Parameter(
            torch.zeros(base.out_features, rank)
        )
        nn.init.kaiming_uniform_(self.lora_a, a=math.sqrt(5))

    def forward(self, x):
        base_output = self.base(x)

        # x @ A.T: [..., r]；再乘 B.T 回到 [..., d_out]。
        low_rank = (x @ self.lora_a.T) @ self.lora_b.T
```

```
        return base_output + self.scale * low_rank

    @torch.no_grad()
    def merged_weight(self):
        # 合并后恢复普通 Linear 权重，不再需要额外 LoRA 分支。
        delta = self.scale * (self.lora_b @ self.lora_a)
        return self.base.weight + delta
```

16.2 LoRA 到底省什么？

LoRA 最直接减少的是可训练参数、对应梯度和优化器状态；它也让每个任务只保存小型 adapter。基础模型前向与激活仍要计算，因此训练 FLOPs 和峰值显存不会按可训练参数比例等比下降。推理时可以把 ΔW 合并进 W，消除低秩分支的额外算子，但合并后的模型通常只是回到基础线性层速度，**LoRA 不自动让推理比原模型更快**。

Q、K、V、输出投影与 FFN 都可成为 LoRA 目标。Q/V 常被选中，是因为它们直接影响“如何查询”和“传递什么内容”，且参数预算较小；但最佳目标依任务而异，复杂领域适配可能需要 K、O 或 FFN。不能把“Q/V 最常见”误说成理论上唯一正确。

面试深挖：为什么 LoRA 可以用低秩更新？

20 秒回答： LoRA 假设下游适配不需要在完整权重空间任意移动，关键更新集中在较低维子空间，因此用 BA 近似 ΔW。这是一种经验归纳偏置，不是所有任务都保证成立；rank 太小会欠拟合，太大则逐渐失去参数效率。

深入解释： 秩 r 决定可表达更新子空间的维度，α/r 控制实际更新尺度。LoRA 主要节省可训练状态，而非消除基础模型计算。合并是把低秩乘积提前加到 W，使部署图恢复成普通线性层，并不改变合并后矩阵大小。

面试官继续追问：

1. rank 越大一定越好吗?
2. 为什么初始化时希望 $\Delta W = 0$?
3. α/r 的作用是什么?
4. LoRA 与 Adapter 有什么区别?
5. 哪些投影适合插 LoRA?

容易答错

错误说法： LoRA 参数少，所以训练和推理都会按相同比例变快。
更准确： 它显著减少梯度、优化器状态和任务检查点；基础模型矩阵乘法与激活仍存在。训练速度取决于激活反向、内核和数据流水线；推理合并后通常是不增加额外延迟，而不是天然快于基础模型。

16.3 QLoRA

QLoRA 的核心组合是：**冻结量化后的基础模型，只训练高精度 LoRA adapter**。基础权重以低比特形式存储，前向时由量化计算内核使用或按需反量化；梯度不更新量化基础权重，而是流向 LoRA 参数。这样同时减少基础权重存储和可训练优化器状态，显著降低微调显存。

它与普通 INT4 推理量化不是同一件事。后者的目标是用低比特权重完成部署推理；QLoRA 的目标是在低比特冻结底座上进行参数高效训练。二者都涉及低比特基础权重，但训练图、梯度和 adapter 生命周期不同。

面试深挖：基础权重量化后，为什么 LoRA 仍能训练？

20 秒回答： 基础权重被冻结，不需要对离散量化值做参数更新；前向仍能通过量化权重计算输出，损失梯度沿激活传播到旁路 LoRA 参数。真正更新的是通常以 BF16/FP16 保存的 A, B，不是 INT4 基础权重。
深入解释： 量化误差改变了 LoRA 所适配的底座，因此量化格式、尺度和异常值处理仍会影响最终质量。QLoRA 节省的是底座存储和训练状态，不意味着所有激活都变成 4 bit，也不意味着训练过程与 INT4 推理完全相同。
面试官继续追问：

1. QLoRA 真正训练哪些参数?
2. 量化底座是否需要梯度?
3. 为什么激活显存仍可能很大?
4. QLoRA 与普通 INT4 inference quantization 有何区别?

第十七章　知识蒸馏

提出动机：one-hot 标签没有表达教师知道的候选结构

硬标签只告诉学生哪个类别被选中，不告诉它其他候选之间的相似性。大型教师模型的输出分布、生成回答和求解轨迹包含更丰富的结构。知识蒸馏用这些软信息训练更小学生模型，以容量和训练数据换取部署效率。

17.1　Logits、回答与推理蒸馏

设教师与学生 logits 分别为 z_t, z_s，温度为 $\tau > 0$：

$$p_t^{(\tau)} = \mathrm{softmax}(z_t/\tau), \qquad p_s^{(\tau)} = \mathrm{softmax}(z_s/\tau).$$

一个常见概念目标是

$$\mathcal{L} = (1-\lambda)\mathcal{L}_{\mathrm{hard}} + \lambda\tau^2 D_{\mathrm{KL}}\left(p_t^{(\tau)} \| p_s^{(\tau)}\right),$$

其中 $\mathcal{L}_{\mathrm{hard}}$ 是真实标签交叉熵，$\lambda \in [0,1]$ 平衡硬标签与教师软分布，τ 把分布变平，使低概率候选之间的相对关系更可见；τ^2 常用于补偿温度导致的梯度尺度变化。

Logits 蒸馏匹配同一输入上的教师分布；**回答蒸馏**用大模型生成的数据训练小模型；**推理蒸馏**还使用经过验证的步骤或轨迹。大模型生成数据再训练小模型属于回答或序列级蒸馏，即使没有直接访问教师 logits，也仍是知识转移。

方式	传递的信息	主要风险
硬标签	最终正确类别	信息少，忽略候选关系
Logits 蒸馏	完整软类别分布	需要教师分数，词表与接口要对齐
回答蒸馏	教师生成的输入–输出对	继承风格与事实错误
推理蒸馏	答案及经过筛选的求解步骤	错误合理化、轨迹冗余、验证成本

表 17.1　不同蒸馏信号的比较。

面试深挖：教师 logits 比 one-hot 标签多提供了什么？

20 秒回答：one-hot 只标出正确类，教师软分布还表达其他类别的相对相似度和不确定性。例如教师可能认为两个同义词都合理，只是一个概率更高；学生可以学习这种“暗知识”，而不是把所有非标签类别都同等视为错误。

深入解释：较高温度减小 logit 差，使长尾候选概率更可见，但温度过高会把有用排序抹平。蒸馏有效还依赖教师确实优于学生、训练输入覆盖部署分布，以及学生容量足以吸收目标；它不是无损压缩。
面试官继续追问：

1. temperature 为什么用于 distillation?
2. 回答蒸馏与 logits 蒸馏的接口要求有何不同?
3. teacher 比 student 差时会怎样?
4. reasoning distillation 有什么额外风险?

推理蒸馏的更完整讨论见第72页。关键原则不变：教师轨迹必须经过答案验证和质量过滤，不能因为文本更长就视为更高质量监督。

第八部分

混合专家模型

第十八章　稀疏专家模型

提出动机：增加模型容量而不等比例增加每词元计算

稠密 Transformer 扩大前馈网络参数时，每个词元都必须经过全部新增参数，模型容量、训练浮点运算量和推理成本同步增长。混合专家模型（Mixture of Experts，MoE）引入条件计算：准备多组专家参数，但让路由器（router）为每个词元只选择少数专家。这样总参数量可以显著增加，而激活参数量主要由 Top-k 决定。

稠密 Transformer 的前馈网络对每个词元都使用同一组参数。MoE 把前馈网络替换为 N 个专家函数 $E_1, \dots, E_N$，每个词元只激活概率最高的 k 个专家。

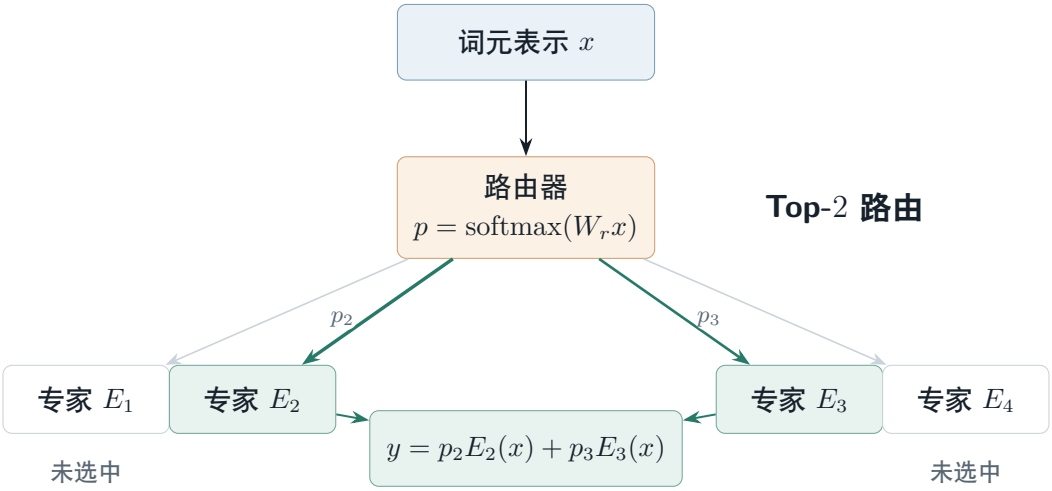


图 18.1　Top-k MoE 路由。路由器只激活少数专家，并按门控概率加权合并输出；线宽表示本例中的路由权重。

对某个词元的隐藏状态 $x \in \mathbb{R}^d$，路由器先计算：

$$p = \mathrm{softmax}(W_r x),$$

其中 $W_r \in \mathbb{R}^{N \times d}$ 是可学习的路由矩阵，$p \in \mathbb{R}^N$ 是专家概率向量，p_i 是选择第 i 个专家的概率。Top-k 输出为：

$$y = \sum_{i \in \mathrm{TopK}(p)} p_i E_i(x).$$

这里 $\mathrm{TopK}(p)$ 是概率最大的 k 个专家下标集合，$E_i(x)$ 是专家 i 的输出，y 是按路由概率加权后的最终输出。由于每个词元只运行少数专家，MoE 可以增加总参数量而不让每词元浮点运算量等比例增加。

Top-k 以后通常会对选中的 k 个概率重新归一化，使它们的和恢复为 1。需要区分“总参数”和“激活参数”：所有专家参数都占模型存储，但一次词元前向只读取被选专家，因此计算量主要随 k 而不是专家总数 N 增长。

具体例子：一个词元如何经过 Top-2 专家

假设路由器对四个专家给出的概率为

$$p = [0.10, 0.60, 0.25, 0.05].$$

Top-2 选择 E_2, E_3。在入选集合内重新归一化后，权重为

$$\tilde{p}_2 = 0.60/0.85 \approx 0.706, \qquad \tilde{p}_3 = 0.25/0.85 \approx 0.294.$$

若两个专家输出 $E_2(x) = (2, 0)$、$E_3(x) = (0, 4)$，最终结果为

$$y = 0.706(2, 0) + 0.294(0, 4) \approx (1.412, 1.176).$$

E_1 和 E_4 本次不执行，因此不产生这个词元的主要前馈网络计算。

18.1 为什么专家通常替换 FFN？

FFN 对每个词元独立执行，且通常占单层的大部分参数与相当比例的 FLOPs，因此天然适合“先按词元路由，再按专家分桶计算”。attention 的独特作用则是跨词元通信：一个查询的输出依赖多个位置。虽然 Q/K/V/O 投影也会变换特征通道，但若直接把注意力头当作互斥专家，仍会改变哪些词元可以互相通信，并引入更复杂的路由与跨设备依赖。

Top-1 路由每个词元只执行一个专家，激活计算和通信较低，但路由错误没有第二专家补偿，训练更依赖负载均衡；Top-2 执行两个专家并加权，通常表达更稳健，但计算、通信与容量需求更高。两者没有脱离硬件与质量目标的绝对优劣。

面试深挖：为什么 MoE 通常替换 FFN，而不是 attention？

20 秒回答： FFN 是逐词元独立的大参数模块，路由后可以把词元分发给不同专家而不改变其可见上下文；attention 承担跨词元通信，稀疏路由会同时改变信息连接和通信。替换 FFN 更容易获得“总参数变大、每词元只激活少量参数”的条件计算收益。

深入解释： MoE 增大的是总容量，不是每个词元的激活容量。所有专家都占存储，但每个词元只运行 Top-k 个。attention 的线性投影当然也会混合特

征，但 attention 头首先共同实现跨位置的信息交换，并不是按输入条件选择的独立大网络；直接把头当专家也无法获得同样的 FFN 参数扩展比例。

面试官继续追问：

1. Top-1 与 Top-2 routing 有什么权衡?
2. 总参数和激活参数有什么区别?
3. 路由器的离散 Top-k 如何训练?
4. 为什么 MoE FLOPs 不高但延迟可能很高?

18.2 对应实现：朴素 Top-k MoE

下面的实现用显式循环展示路由语义，便于理解，但不适合高性能训练。真实系统会按专家对词元排序和分桶，再用分组矩阵乘法（grouped GEMM）或专家并行的全交换通信批量计算。

```python
import torch
import torch.nn as nn

class NaiveTopKMoE(nn.Module):
    def __init__(self, dim, experts, top_k=2):
        super().__init__()
        if not 1 <= top_k <= len(experts):
            raise ValueError("top_k must be within expert count")

        self.experts = nn.ModuleList(experts)
        self.top_k = top_k

        # Router 为每个 token 产生 num_experts 个打分。
        self.router = nn.Linear(dim, len(experts), bias=False)

    def forward(self, x):
        # x: [B, T, C]
        batch, seq_len, dim = x.shape
        tokens = x.reshape(batch * seq_len, dim)  # [N_token, C]

        router_logits = self.router(tokens)
        router_probs = torch.softmax(router_logits, dim=-1)
        # router_probs: [N_token, num_experts]

        # 每个 token 只保留概率最大的 top_k 个专家。
        top_prob, top_idx = router_probs.topk(self.top_k, dim=-1)

        # 只在入选专家之间重新归一化门控权重。
        top_prob = top_prob / top_prob.sum(dim=-1, keepdim=True)
```

```
        output = torch.zeros_like(tokens)
        for expert_id, expert in enumerate(self.experts):
            for slot in range(self.top_k):
                # 找出在当前 slot 选择了该专家的 token。
                token_mask = top_idx[:, slot] == expert_id
                if not token_mask.any():
                    continue

                expert_out = expert(tokens[token_mask])
                weight = top_prob[token_mask, slot, None]

                # 同一 token 的多个专家结果按路由概率加权相加。
                output[token_mask] += weight * expert_out

        return output.view(batch, seq_len, dim), router_probs
```

`topk` 返回的专家下标是离散选择，不能像普通连续函数那样对“下标变化”求导；路由器主要通过入选专家的门控权重和额外的负载均衡损失学习。若长期只有少数专家入选，未选专家几乎收不到主任务梯度，这就是专家坍缩（expert collapse）难以自行恢复的原因。

18.3 容量与负载均衡

提出动机：条件路由会自然产生热点专家

路由器若只追求主任务损失，可能发现少数专家在早期略占优势，随后把更多词元送给它们，使这些专家获得更多训练并进一步垄断流量。设备容量有限，热点专家既会溢出，也会让其他设备空闲。专家容量（expert capacity）和负载均衡损失就是为打破这种正反馈而提出的。

专家容量限制每个专家在一个批次中最多处理多少词元。容量太小会丢弃词元或转给备选专家，容量太大则会因负载不均让慢专家拖住整体。

若一个批次共有 N_{tok} 个词元、N 个专家且每个词元路由到 k 个专家，平均每个专家接收 kN_{tok}/N 个词元。引入容量因子（capacity factor）$c \geq 1$ 后，常设

$$C = \left\lceil c \frac{kN_{\text{tok}}}{N} \right\rceil .$$

其中 C 是每个专家允许处理的最大词元数，$\lceil \cdot \rceil$ 表示向上取整。c 越大，溢出概率越低，但预留缓冲与无效填充越多；c 太小则高负载专家超过 C 后必须丢弃或改路由。

具体例子：容量因子如何决定专家上限

一个批次有 $N_{\text{tok}} = 1000$ 个词元、$N = 4$ 个专家，每个词元选择 $k = 2$ 个专家。平均每个专家会收到 $2 \times 1000/4 = 500$ 次路由。若容量因子 $c = 1.25$，则每个专家最多接收

$$C = \lceil 1.25 \times 500 \rceil = 625$$

个词元。若热点专家实际收到 800 个，就有 175 个超出容量，需要丢弃、转给备选专家或延后处理；与此同时其他专家可能仍未满载。这就是负载不均会同时伤害质量和吞吐的具体方式。

负载均衡的直觉是希望路由器分配概率和实际词元计数都接近均匀。设 f_i 为分到专家 i 的词元比例，P_i 为所有词元对专家 i 的平均路由概率，可加入辅助损失：

$$\mathcal{L}_{\text{aux}} = N \sum_{i=1}^{N} f_i P_i.$$

其中 N 是专家总数。当少数专家吸走大部分词元时，该项会变大，从而惩罚专家坍缩。

18.4 并行与工程影响

提出动机：专家总参数放不下一张卡

MoE 虽然每个词元只激活少数专家，但所有专家参数仍要存储。专家数量增大后，单卡无法容纳完整专家集合，于是需要把不同专家放到不同设备。代价是词元必须跨设备送往被选专家，计算结束后再把结果送回原序列位置。

专家并行（expert parallelism）把不同专家放到不同设备。通信通常包含词元分发（dispatch）与结果合并（combine），并形成全交换（All-to-All）模式。MoE 的瓶颈不只是浮点运算量，还包括网络通信、负载不均和小批次下专家利用率；因此“激活计算量较低”并不自动等于端到端延迟较低。

全交换通信不同于 AllReduce：每张设备持有不同专家，需要把各词元发送到其目标专家所在设备，计算后再按原位置送回。若某个专家成为热点，其他设备即使完成计算也必须等待最慢设备；小批次还可能让每个专家只收到很少词元，矩阵乘法无法达到高利用率。

面试深挖：Router collapse 为什么会发生，负载均衡损失在优化什么？

20 秒回答：早期少数专家略优时，路由器会给它们更多词元，它们得到更多主任务梯度后进一步变强，形成正反馈。负载均衡损失同时约束实际分配比例和平均路由概率，鼓励流量分散，避免热点溢出与闲置专家。

深入解释：capacity factor 只提供容量缓冲，不能从根本上消除偏斜；过大浪费内存和填充，过小会丢词元或改路由。辅助损失过强又可能为了均匀牺牲专家专业化，因此主任务质量、负载和通信要联合调节。

面试官继续追问：

1. capacity factor 如何影响溢出概率?
2. expert parallel 为什么使用 All-to-All?
3. 小 batch 为什么对 MoE 不友好?
4. 辅助损失权重过大会怎样?

MoE 把模型扩展问题从纯计算转变为路由、容量和通信的联合问题。下一部分的分布式训练会把视角进一步推广：当 dense 参数、激活或优化器状态也放不下单卡时，应当切分哪一种资源。

常见误区

MoE 的“参数更多”不等于“推理一定更慢”。真正要看每词元激活参数、专家并行通信和负载均衡。

面试高频问题

Q: MoE 为什么能提升容量但控制计算量？

20 秒回答：每个词元只激活 Top-k 个专家，所以总参数可以很大，但每个词元只使用其中一小部分。

Q: 专家坍缩是什么？

少数专家被大量词元选择，其他专家几乎不用。结果是容量浪费、负载不均和训练不稳定，需要辅助负载均衡损失或路由约束。

第九部分

分布式训练

第十九章　并行策略

提出动机：单卡容量和单卡吞吐都无法继续扩展

模型变大后会依次遇到两类限制：模型虽然能放入单卡，但训练速度太慢；或者参数、优化器状态与激活根本放不下。分布式训练不是单一算法，而是根据瓶颈选择切分轴：切批次提高吞吐，切矩阵或层容纳模型，切序列降低长上下文激活，切专家承载 MoE 参数。

大模型训练的核心问题是：参数、梯度、优化器状态、激活和批次都可能放不下单卡。不同并行策略切分的对象不同。

方法	切分对象	通信	显存收益	典型用途
数据并行（DDP）	批次	梯度全归约	参数仍复制	中小模型多卡训练
张量并行	矩阵维度	全归约/全收集	切参数与激活	单层太大
流水线并行	层	发送/接收激活	切模型层	层数很多
序列并行	序列维度	全收集/归约散射	降低激活	长上下文
专家并行	专家	全交换	分散专家参数	MoE
ZeRO-1	优化器状态	归约与收集	节省优化器状态	Adam 状态大
ZeRO-2	再切梯度	归约散射	进一步节省	大规模稠密模型
ZeRO-3/FSDP	再切参数	全收集参数	最大节省	超大模型

表 19.1　常见分布式训练方法。

19.1　为什么需要不同的并行维度

数据并行（Data Parallel）与分布式数据并行（DDP）适用于完整模型能放入单卡，但希望用更多样本并行提高吞吐的情况。每张卡计算不同数据，代价是反向后必须同步梯度。它不切模型参数，因此不能解决“单卡放不下模型”。

张量并行（Tensor Parallel，TP）针对单个 Transformer 层或大矩阵本身放不下单卡的问题，沿隐藏维、注意力头或前馈网络中间维切分矩阵乘法。每层前向和反向都可能需要全归约（AllReduce）或全收集（AllGather），通信频繁，所以通常优先放在节点内高速互联的 GPU 之间。

流水线并行（Pipeline Parallel，PP）针对模型层数多、整体参数放不下的问题，把连续层分配到不同阶段（stage）。它只在阶段边界传激活，但后一个阶段必须等待前一个阶段的输出。微批处理用流水线重叠不同样本来减少空闲，仍会留下流水线气泡，也会增加调度复杂度。

序列并行（Sequence Parallel）针对长上下文下激活随 T 增长的问题，沿序列维分散层归一化、随机失活或部分注意力/前馈网络激活。它节省的是激活而非模型总参数，并需要在进入依赖完整隐藏维或序列信息的算子前做集合通信。

专家并行（Expert Parallel）针对 MoE 的专家总参数过大，把不同专家放到不同设备。路由器决定词元去向后执行全交换分发，再将专家输出合并回原位置。其效率高度依赖负载均衡和网络带宽。

实际大模型常组合数据并行、张量并行、流水线并行与 ZeRO/FSDP，形成多维并行。选择顺序应从瓶颈出发：先问模型是否放得下，再问激活是否放得下，最后评估哪种通信能被当前硬件拓扑承受。

方法	切了什么	没切什么	主要通信	主要节省
DDP	批次	参数、梯度状态副本	梯度 AllReduce	提高吞吐，不省模型状态
TP	单层矩阵	通常不切层数	层内 AllReduce/AllGather	参数与部分激活
PP	连续层	单层内部矩阵	阶段间激活发送/接收	每阶段模型层
SP	序列维激活	通常不切全部参数	AllGather / ReduceScatter	长序列激活
EP	专家集合	非专家主干	词元 All-to-All	专家参数
ZeRO/FSDP	模型状态	视阶段而定的激活	参数收集与梯度归约	参数/梯度/优化器状态

表 19.2 从“切了什么、没切什么”理解分布式策略。

面试深挖：为什么数据并行与张量并行经常组合？

20 秒回答：张量并行解决单个模型副本或单层放不下，把一层矩阵切到设备组内；数据并行再复制这个 TP 设备组，处理不同批次提高吞吐。它们切的是不同轴，因此可以组成二维并行。

深入解释：TP 每层通信频繁，通常放在节点内高速互联；DP 的梯度同步粒度更粗，可以跨节点。只用 DP 时每卡仍需完整模型，只用 TP 时虽然模型放得下，但全局 batch 吞吐未必足够。组合方式应匹配网络拓扑，而不是只追求更多并行维度。

面试官继续追问：

1. TP 为什么几乎每层都有通信?
2. PP 的 bubble 从哪里产生?
3. micro-batch 为什么能减少 bubble?

4. EP 为什么与 MoE 强绑定?

19.2 分布式数据并行与全归约

提出动机：多卡计算必须保持同一份模型参数

把不同小批次（mini-batch）分给多张卡可以近似线性增加样本吞吐，但各卡会得到不同局部梯度。若直接更新，模型副本从第一步起就会分叉。DDP 通过全归约求全局平均梯度，使多卡结果在数学上等价于一个更大的批次。

DDP 中每张卡持有完整模型，处理不同小批次分片。反向传播后，第 i 张卡得到局部梯度 g_i。设设备总数为 N，为了等价于大批次训练，需要同步平均：

$$g = \frac{1}{N}\sum_{i=1}^{N} g_i.$$

其中 g 是所有设备的平均梯度。全归约完成“所有设备求和并把结果发回所有设备”；如果不同步梯度，各卡参数会向不同方向更新。

这里不能用普通 Gather 代替：Gather 只是把各卡张量拼到某个目标设备，既没有求和，也不会把结果自动发回所有设备；AllReduce 同时执行归约和广播语义，使每张卡都获得同一全局梯度。实现上可以分解为 ReduceScatter 与 AllGather，但语义仍是“全局归约后每卡得到结果”。

具体例子：两张卡为什么必须平均梯度

同一参数在卡 1 的局部梯度为 $g_1 = (2, 0)$，卡 2 为 $g_2 = (0, 4)$。AllReduce 求和后两张卡都得到 $(2, 4)$，再除以设备数得到全局平均

$$g = (1, 2).$$

若学习率为 0.1，两张卡都会执行同样的参数更新 $-0.1(1, 2)$。若不同步，卡 1 更新 $(-0.2, 0)$、卡 2 更新 $(0, -0.4)$，下一步两份模型已不再相同，也就不能继续表示同一个大批次训练。

对大小为 S 字节的梯度，环形全归约（ring AllReduce）可分解为归约散射（ReduceScatter）与全收集。每阶段通信 $(N-1)S/N$ 字节，因此每张卡总通信量近似为

$$V_{\text{ring}} = 2\frac{N-1}{N}S.$$

其中 V_{ring} 是每张卡发送和接收的数据量，N 是设备数。卡数增大时它趋近 $2S$，所以 DDP 扩展效率取决于反向计算能否覆盖这部分通信，而不只是 GPU 数量。

19.3 ZeRO 与 FSDP 的内存推理

提出动机：DDP 的每张卡为什么仍然很快耗尽显存？

DDP 只切数据，每张卡仍复制完整参数、梯度和 Adam 状态。模型参数规模增长时，增加 GPU 数量并不会降低单卡模型状态显存。ZeRO/FSDP 把这些原本重复的状态分片存放，在计算需要时临时聚合，用更多通信换取近似 $1/N$ 的状态显存。

Adam 训练常存参数、梯度、一阶矩、二阶矩。若每项都按 FP32 估算，优化器状态可比参数本身还大。ZeRO/FSDP 的思路是把这些状态分片（shard）到不同设备，需要时再收集（gather）。代价是通信增加，收益是单卡显存显著下降。

关键推导

以混合精度 Adam 为例，每个参数常近似需要：BF16 参数 2 字节、BF16 梯度 2 字节、FP32 主参数副本 4 字节、两个 FP32 动量状态 8 字节，合计约 16 字节。若模型有 P 个参数，设备数为 N，忽略激活与临时缓冲区：

$$M_{\mathrm{DDP}} \approx 16P,$$

$$M_{\mathrm{ZeRO1}} \approx 4P + \frac{12P}{N}, \qquad M_{\mathrm{ZeRO2}} \approx 2P + \frac{14P}{N}, \qquad M_{\mathrm{ZeRO3}} \approx \frac{16P}{N}.$$

这些公式中的 M 表示单卡持久模型状态的字节数。它们是用于面试估算的理想值；真实峰值还包含激活、通信桶（bucket）、参数全收集和显存碎片。

具体例子：10 亿参数在 4 张卡上的理想状态显存

令 $P = 10^9$，按每参数约 16 字节估算。DDP 在每张卡都保存完整状态，因此约需 16 GB。理想 ZeRO-3/FSDP 将参数、梯度和优化器状态都均分到 $N = 4$ 张卡：

$$M_{\mathrm{ZeRO3}} \approx \frac{16P}{4} = 4 \text{ GB/卡}.$$

节省的 12 GB 并没有消失，而是分散到了其他设备；某层计算前仍要临时全收集它的参数。所以 4 GB 是持久模型状态估算，不等于训练峰值显存。

ZeRO 的三个阶段可以理解为逐步消除 DDP 的重复副本：阶段 1 切优化器状态，阶段 2 再切梯度，阶段 3 连参数也切。完全分片数据并行（Fully Sharded Data Parallel，FSDP）与 ZeRO-3 的核心目标相近，都是在层计算前全收集所需参数、计算后再释放或重新分片。分片越彻底，显存越省，但参数通信和调度越敏感。

19.3.1 训练显存的完整核算框架

面试中先写出

$$M_{\text{train}} = M_{\text{param}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}} + M_{\text{temp}} + M_{\text{comm}}.$$

六项分别是参数、梯度、优化器状态、前向激活、算子临时缓冲和通信缓冲。模型权重只是第一项；序列越长、微批次越大或未使用高效 attention 时，激活与临时缓冲也可能成为峰值主导项。

推理通常不需要参数梯度、Adam 一二阶动量和反向激活，因此比训练少很多状态；但长上下文服务会新增键值缓存。梯度累积只降低单次微批次激活，不会减少参数、梯度或优化器状态。激活检查点用重复前向计算换激活显存；ZeRO/FSDP 主要分片模型状态，二者解决的不是同一项，常组合使用。

计算题：为什么 7B BF16 模型不只用 14 GB 就能训练？

7B 参数的 BF16 权重约为

$$7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}.$$

若采用前述每参数约 16 字节的混合精度 Adam 简化模型，单卡持久模型状态约为

$$7 \times 10^9 \times 16 \text{ bytes} = 112 \text{ GB}.$$

这还没有包含激活、临时张量、通信桶和碎片。若理想 ZeRO-3 分到 8 张卡，持久状态约为 $112/8 = 14$ GB/卡，但层参数 AllGather 与激活会使真实峰值高于 14 GB。

面试深挖：为什么训练显存远大于推理显存？

20 秒回答： 训练除权重外，还要保存梯度、Adam 状态和用于反向的中间激活，并存在临时与通信缓冲；推理通常只保留权重、当前激活和 KV Cache。因此“7B BF16 权重 14 GB”只是推理权重下界，不是训练显存。

深入解释： 混合精度 Adam 常保留 BF16 权重/梯度、FP32 主权重和两个 FP32 动量，约 16 字节/参数。ZeRO 分片模型状态，checkpointing 压缩激活，gradient accumulation 只改变单步激活与逻辑 batch；三者不能互相替代。

面试官继续追问：

1. gradient accumulation 是否降低 parameter memory?
2. activation checkpointing 用什么换显存?

3. ZeRO-2 与 ZeRO-3 多切了什么？
4. inference 中什么状态可能随上下文增长？

面试深挖：FSDP forward 前为什么需要 AllGather？

20 秒回答：持久状态中每张卡只保存当前层参数的一个分片，但完整矩阵乘法需要该计算组内的完整参数。因此进入层前临时 AllGather 参数，计算后再释放或重新分片，以通信换单卡持久显存。

深入解释：ZeRO-1 只分片优化器状态，ZeRO-2 再分片梯度，ZeRO-3/FSDP 连参数也分片。分片越彻底，持久显存越低，但参数预取、通信重叠和峰值临时内存越关键；理想 $1/N$ 不能直接当作真实峰值。

面试官继续追问：

1. 参数 AllGather 何时可以与计算重叠？
2. 为什么 bucket 太大或太小都可能不好？
3. FSDP 与 TP 都切参数，区别是什么？

19.4 激活检查点（Activation Checkpointing）

提出动机：模型状态能放下，前向激活仍可能放不下

反向传播需要中间激活来计算梯度，深层网络和长序列会保存大量 $[B, T, C]$ 张量。这里 B 是批次大小，T 是序列长度，C 是隐藏维度。ZeRO/FSDP 主要切模型状态，并不能消除这些激活。激活检查点只保存少数边界激活，反向时重新执行部分前向，以额外计算换显存。

若普通训练保存 L 层全部激活，激活显存大致随层数 L 线性增长。分段检查点可把若干连续层视为一段，只保存段输入；反向到该段时重算内部激活。分段数量越少，显存越省但重算越多。它不减少参数、梯度或优化器状态，也不会直接减少通信，因此通常与 FSDP、混合精度和序列并行组合使用。

具体例子：用重算换掉哪些激活

假设网络有 12 层。普通训练保存每层输出 $h_1, \ldots, h_{12}$，其中 h_ℓ 表示第 ℓ 层的输出；若每份激活占 1 GB，仅这些边界张量就约 12 GB。现在每 4 层设一个检查点，只长期保存输入 h_0, h_4, h_8 和最终输出。反向计算第 9–12 层时，先从 h_8 重新前向得到内部激活，再立即用于反向并释放。

实际峰值还包含当前分段的临时激活，因此不会精确降到 3 GB；但例子说明了交换关系：保存的长期激活减少，部分前向计算会在反向阶段重复执行。

分布式训练解决的是“如何把固定训练任务放到多设备”。部署后的知识时效性与外部操作能力仍不是扩大并行度能解决的问题，这分别引出检索增强生成和智能体。

面试高频问题

Q: 张量并行和流水线并行的区别？

20 秒回答：TP 切单层矩阵，解决一层放不下；PP 切模型层，解决整体层数太多。TP 通信更频繁，PP 要处理流水线气泡。

Q: ZeRO-3 为什么比 ZeRO-2 更省显存？

ZeRO-2 切优化器状态和梯度；ZeRO-3 还切参数本身，因此参数也不再每卡完整复制。

第十部分

模型评测

第二十章　LLM Evaluation

提出动机：为什么不能只看 validation loss？

验证损失衡量模型对某个数据分布的下一个词元预测，却没有直接测量指令遵循、事实忠实性、代码可执行性、安全性和用户偏好。后训练还可能主动改变回答风格，使通用语料上的困惑度变差，但任务成功率更高。因此商业系统需要把“概率拟合得怎样”拆成“任务是否完成、用户是否认可、风险是否可控”。

20.1　指标分别回答什么问题

设有效验证词元的平均负对数似然为 $\mathcal{L}_{\text{token}}$，困惑度为

$$\text{PPL} = \exp(\mathcal{L}_{\text{token}}).$$

它可以直观理解为模型在每一步面对的“有效候选不确定度”，但这一解释只是直觉。不同分词器会把同一字符串切成不同数量的词元，损失的计量单位随之改变，所以不能直接用 token-level PPL 公平比较采用不同分词器的模型。

指标	适合回答	主要盲区
Validation loss / PPL	是否拟合验证语料的词元分布	不直接测任务体验；受分词器影响
Accuracy / Exact Match	分类或唯一答案是否正确	忽略等价表述和部分正确
pass@k	k 次代码采样中至少一次通过测试的能力	强依赖测试覆盖率与采样设置
Pairwise win rate	两个回答中哪一个更受偏好	顺序、长度、评审模型会引入偏差
Human evaluation	人类对有用性、正确性和风格的判断	成本高、标注一致性有限
LLM-as-a-judge	低成本、可扩展的成对或量表评审	自偏好、位置偏差、verbosity 偏差
Online metrics	真实用户任务成功率、留存或投诉	有延迟、混杂因素和实验风险

若对同一道代码题独立采样 n 个候选，其中 c 个通过单元测试，从中不放回选 k 个时，常用无偏估计为

$$\widehat{\text{pass@}k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}.$$

当 $n - c < k$ 时，至少会选中一个已知通过样本，估计值为 1。pass@1 关注单次可靠性，pass@10 还混入了多样采样和选择预算；后者高而前者低，说明模型能够偶尔找到答案，但默认一次生成还不稳定。

```python
from math import comb


def pass_at_k(num_samples: int, num_correct: int, k: int) -> float:
    """Estimate the probability that at least one of k samples passes."""
    if not 0 <= num_correct <= num_samples:
        raise ValueError("num_correct must be in [0, num_samples]")
    if not 1 <= k <= num_samples:
        raise ValueError("k must be in [1, num_samples]")

    # Fewer than k failed samples means every size-k subset contains a
   pass.
    if num_samples - num_correct < k:
        return 1.0

    # Probability of at least one pass = 1 - probability of all failures.
    return 1.0 - comb(num_samples - num_correct, k) / comb(num_samples,
   k)
```

计算题：pass@k 的直觉

某模型为一道代码题生成 $n = 20$ 个候选，其中 $c = 4$ 个通过测试。则

$$\widehat{\text{pass@}}1 = 1 - \frac{16}{20} = 0.2, \qquad \widehat{\text{pass@}}5 = 1 - \frac{\binom{16}{5}}{\binom{20}{5}} \approx 0.718.$$

这不表示线上单次调用有 71.8% 正确率，而是表示在同一采样协议下，五个候选中至少一个通过测试的估计概率。

20.2 污染、评审偏差与线上闭环

评测集污染（benchmark contamination）指题目、答案或高度相似变体进入训练数据。模型可能依靠记忆获得高分，使离线榜单不再代表泛化能力。仅检查逐字重复不够，还要关注释义、翻译、题解和生成数据回流。可靠评测应保留时间切分或私有集合，记录提示模板、采样参数、模型版本和评审版本。

LLM-as-a-judge 可以快速评价开放式回答，但评审模型可能偏好更长的答案、特定位置、与自己相似的表达或熟悉的模型家族。缓解方式包括交换候选顺序、隐藏模型身份、使用明确量表、对关键样本做人类复核，并报告一致性而不是只报告一个总分。

离线评测适合快速迭代和故障回归，线上评测则观察真实分布中的任务完成率、用户修正率、延迟与成本。二者不能互相替代：离线分数改善可能在真实输入上无效；线上总体指标也可能掩盖某类高风险用户的退化。

面试深挖：validation loss 下降为什么不代表模型一定更好用？

20 秒回答：它只测给定验证语料上的词元概率，不直接测指令遵循、事实正确性、安全性或用户偏好；数据分布和分词器还会影响数值。后训练可能牺牲通用语料似然，换取更符合任务目标的行为。

深入解释：交叉熵是 proper scoring rule，适合比较同一 tokenization 和分布下的概率预测；产品体验却是多目标函数。评测应把基础能力、目标任务、行为约束、延迟和成本分开测量，并检查平均值是否掩盖长尾退化。

面试官继续追问： 1. 为什么不同 tokenizer 的 PPL 不宜直接比较？2. SFT 后 PPL 变差一定是灾难性遗忘吗？3. 什么时候应使用 exact match，什么时候需要语义评审？

面试深挖：pass@1、pass@10 与 Best-of-N 有什么关系？

20 秒回答：它们都衡量多次采样带来的覆盖收益，但 pass@k 用可执行测试定义“通过”，Best-of-N 还需要 verifier 从候选中选出一个。候选里存在正确答案，不等于选择器一定能找到它。

深入解释：提高 k 通常会提高“至少一个通过”的概率，同时增加推理成本。若候选高度相关、测试不完整或 verifier 不可靠，收益会饱和甚至产生错误选择。因此必须同时报告采样温度、候选数、测试集和最终选择策略。

面试官继续追问： 1. pass@10 高但 pass@1 低说明什么？2. 测试用例覆盖不足会造成什么假阳性？3. 为什么不能只报最优一次运行？

面试深挖：LLM-as-a-judge 可以代替人类评测吗？

20 秒回答：它适合低成本扩大覆盖和做回归筛查，但不能无条件替代人类。评审模型存在位置、长度、自偏好和领域知识偏差，高风险或主观任务仍需校准与人工复核。

深入解释：应把 judge 看作一个有误差的测量工具：先用人工标注集估计一致性，再固定提示和版本，交换答案顺序，保留分项理由，并对分歧样本复核。judge score 与 reward model score 都是代理指标，不是真实质量本身。

面试官继续追问： 1. 如何检测位置偏差？2. pairwise win rate 为什么不具备绝对标尺？3. 离线 judge 胜率怎样与线上 A/B 结果联动？

容易答错

容易答错：“后训练后困惑度变差，说明模型整体退化。”

更准确：需要先确认分词器、数据和计算口径一致，再区分通用语言建模似然与目标行为。模型可能因学习简洁拒答、指令格式或偏好而降低某类语料似然，却在目标任务上更可靠；也可能真的发生遗忘，必须用分层评测判断。

第十一部分

检索增强生成与智能体

第二十一章　检索增强生成（RAG）

提出动机：参数记忆无法及时更新，也难以给出可核验来源

预训练知识被压缩在参数中，更新一次通常需要重新训练或微调；面对企业私有数据、刚发生的事件和要求精确引用的任务，单靠参数记忆既可能过时，也难以追溯证据。检索增强生成（Retrieval-Augmented Generation，RAG）在生成前动态检索外部资料，把“记住所有事实”改成“需要时找到事实并放入上下文”。

RAG 用外部检索补充模型上下文，常见流程是：文档切块、计算向量表示（embedding）、写入向量库、查询召回、重新排序、拼接上下文、生成答案。

21.1　为什么需要切块、召回与重排

整篇文档通常长于向量模型或生成模型的有效输入，也很难用一个向量表达所有局部主题，因此首先需要**文档切块（chunking）**。块太大时，向量语义会被多个主题稀释，命中后还会带入大量噪声；块太小时，句子和表格上下文会被切断。重叠窗口、按标题分段和保留父文档元数据，都是为了缓解边界信息丢失。

向量召回的目标是从海量文档中低成本取得较高**召回率（recall）**，即尽量不要漏掉相关材料，因此常使用近似最近邻，并容忍候选中混入噪声。仅靠向量相似度又难以精细判断复杂查询与段落的逐词对应，于是增加**重排器（reranker）**，用计算更贵的交叉编码或模型评分，在小候选集上提高**精确率（precision）**，也就是减少无关材料。两阶段设计本质上是“便宜地广搜，再昂贵地精排”。

最终只把有限数量的高价值片段放入上下文，是因为上下文窗口有限，提示词预填充（prefill）的计算成本也会随输入变长，而且更多材料不一定更好。噪声证据会分散注意力，互相矛盾的片段还可能诱导错误回答。因此 RAG 的关键权衡是召回率、精确率、上下文预算与延迟，而不是简单增加返回候选数 Top-k。

稠密向量检索擅长语义近似，BM25 等词法检索更擅长精确名称、编号和罕见词；**混合检索（hybrid search）**把二者候选或分数融合。元数据过滤则在检索前限制租户、权限、语言、产品版本和生效日期。它不是“锦上添花”：当旧版制度与新版制度语义高度相似时，单靠 embedding 很难可靠判断哪个版本有效。

长上下文还存在 **Lost in the Middle** 风险：相关证据虽然被放入窗口，却位于大量文本中间而没有被模型稳定利用。改善方式包括重排、去重、压缩、把高置信证据放在更显著的位置，以及要求引用与证据核验；简单扩大 Top-k 往往同时增加预填充成本与干扰。

RAG 不能从根本上消除幻觉，因为模型仍可能误读、忽略或编造上下文。要提高可信度，还需要要求答案引用检索片段、检查引用是否真正支持结论，并在证据不足时允许拒答。RAG 解决知识访问问题，不自动解决推理正确性和生成忠实性。

具体例子：查询一条刚更新的公司制度

用户询问："2026 年正式员工有多少天年假?" 知识库中有三个候选块：

- A：2024 年旧制度，年假 10 天；
- B：2026 年新制度，年假调整为 15 天；
- C：2026 年差旅报销标准。

向量召回可能因 "年假" 相似度返回 $[A, B, C]$。重排器同时阅读问题与段落后，应把包含正确年份和主题的 B 排在最前。生成上下文只放 B，并保留文档日期与来源，模型才能回答 "15 天" 并引用新制度。
若系统只按语义相似度、不检查版本元数据，旧制度 A 也很可能排名很高；若 B 根本没有被召回，生成模型再强也无法从上下文恢复缺失事实。这个例子分别对应检索的召回率、重排的精确率和版本过滤问题。

面试一句话回答

RAG 面试不要只说 "接向量数据库"。要能解释文档切块、向量质量、召回评估、重新排序、上下文窗口和引用可信度。

面试深挖：为什么高召回之后仍需要 reranker ?

20 秒回答： 第一阶段检索要在大库中便宜地减少漏召，通常允许较多噪声；reranker 在小候选集上做更细的查询–文档联合判断，提高精确率。二者分别优化候选覆盖和最终上下文质量。
深入解释： 双编码器可以预先计算文档向量，检索成本低，但查询与文档只在最终向量上交互；交叉编码器能逐词联合建模，判断更精细，却不适合扫描全库。若正确片段没有进入第一阶段候选，reranker 无法恢复它；若把所有高召回候选直接塞给模型，噪声和上下文成本又会上升。

面试官继续追问： 1. chunk 太大和太小分别损失什么？2. metadata filtering 应发生在召回前还是召回后？3. hybrid search 怎样合并不同分数尺度？4. 如何分别评估 retrieval 与 generation？

面试深挖：上下文越多为什么不一定越好？

20 秒回答： 更多上下文增加预填充成本，也会引入重复、过时或冲突证据；模型还可能忽略长上下文中间的信息。目标应是证据覆盖率与信噪比，而不是最大化输入长度。

深入解释： 版本冲突需要以来源、生效时间和权限等元数据解决，不能让生成模型自行“投票”。系统还应在文档更新时保存版本标识，在答案中引用实际使用的片段，并对无充分证据的查询允许拒答。

面试官继续追问： 1. Lost in the Middle 是检索失败还是利用失败？2. 文档版本冲突如何构造回归测试？3. 为什么引用存在不等于引用支持结论？

第二十二章　智能体（Agent）

提出动机：一次模型调用无法完成需要外部反馈的多步任务

普通 LLM 前向只能根据当前上下文生成文本，无法知道数据库查询是否成功、代码执行结果是什么，也不能在看到新结果后自动调整计划。智能体（Agent）把模型置于“观察–行动”（observation–action）闭环中：模型选择工具，环境返回真实观察，模型再据此继续决策，从而处理需要外部状态和多步操作的任务。

智能体通常包含模型、工具、状态/记忆、规划和观察–行动循环。一个简化循环是：

$$\text{observe} \to \text{plan} \to \text{act(tool)} \to \text{observe}.$$

22.1　为什么需要参数规范、状态与安全边界

工具参数规范（schema）把自然语言意图约束为结构化参数，解决模型输出难以被程序可靠解析的问题。参数规范越清楚，参数类型、必填项和错误反馈越明确，模型越容易选择正确工具；但它不能代替服务端校验，模型仍可能提供不存在的标识符（ID）或越权参数。

状态与记忆用于保存跨步骤事实、计划进度和工具结果，解决有限上下文下的信息持续性问题。原样保留全部历史会快速耗尽窗口并增加噪声，因此需要摘要、结构化状态或按需检索。压缩又可能丢失关键细节，所以重要状态应有明确数据结构，而不只依赖自然语言摘要。

权限边界与确认是因为智能体的错误不再只是一段错误文本，还可能变成真实写操作。读取、写入、付款、删除等工具应使用最小权限，对高影响或不可逆动作增加参数校验、幂等设计和人工确认。错误恢复则要区分可重试的临时失败与逻辑错误，避免模型无限循环调用。

可观测性记录每一步输入、工具选择、参数、响应和延迟，用于定位长链路中的首个错误。智能体的主要风险正是误差累积：一步错误观察或工具调用会污染后续计划；如果只看最终回答，通常无法判断问题来自模型、工具、数据还是权限层。

幂等性（idempotency）指同一操作重复执行时，最终效果与执行一次一致。网络超时后客户端未必知道服务端是否已经完成写入；若没有幂等键就直

接重试，可能重复付款或创建两场会议。对无法天然幂等的动作，应使用唯一请求标识、先查询后重试或事务状态机。

MCP (Model Context Protocol) 在面试层面可以理解为标准化的上下文、工具和资源接口。模型应用负责推理与编排，外部能力提供方按统一协议暴露可发现的工具或资源，从而降低每个应用重复编写私有适配器的成本。MCP 解决接口与集成问题，不自动解决权限、正确性、工具选择和业务安全。

RAG 和智能体的关系也应区分：RAG 通常是一次生成前的知识检索流程，智能体则允许模型根据观察反复决定下一步动作。检索可以成为智能体的一个工具，但使用了向量数据库并不意味着系统已经具备规划和反馈闭环。

具体例子：安排一次不能冲突的会议

用户要求“给项目组安排明天下午 30 分钟会议”。一个可靠智能体的执行链可以是：

1. 从用户请求中提取参会人、日期范围和时长；
2. 调用日历读取工具，观察每个人的忙闲状态；
3. 发现 14:00 有人冲突后，重新规划并选择共同空闲的 15:30；
4. 调用会议室查询工具，得到可用房间 R2；
5. 在真正创建事件前向用户确认“15:30–16:00，R2”；
6. 用户确认后调用写入工具，并读取返回的事件编号验证成功。

普通一次函数调用通常只能完成第 6 步的参数提交；智能体的关键是前面的多次观察、根据冲突修改计划，以及对写操作设置确认边界。若创建接口超时，还应先用事件编号或幂等键查询结果，而不是盲目重试并创建重复会议。

面试深挖：Agent 和 workflow 的本质区别是什么？

20 秒回答： workflow 的步骤和分支主要由程序预先定义；Agent 把下一步动作选择部分交给模型，并根据环境观察动态循环。实际系统常采用受约束的混合模式，而不是让模型无限自由规划。

深入解释： 固定流程更可预测、易测试；开放式 Agent 能处理未预先枚举的状态，但增加循环、权限和错误累积风险。规划也不一定要输出可见的长文本，可以体现为结构化状态更新、策略选择或下一工具调用。

面试官继续追问： 1. Agent 为什么需要显式状态？2. 哪些任务更适合固定 workflow？3. 如何设置最大步数和停止条件？4. tool error 应该重试、换工具还是交给用户？

面试深挖：为什么 tool call 需要 schema、确认与幂等性？

20 秒回答： schema 让参数可解析和校验；确认把高影响决策留给用户；幂等性避免超时重试产生重复副作用。三者分别控制输入格式、授权边界和执行语义。

深入解释： schema 不能代替服务端鉴权，确认也不能修复错误参数。可靠执行还需要最小权限、超时分类、幂等键、审计日志和补偿机制。MCP 可以统一能力接口，但这些业务约束仍由应用与服务端落实。

面试官继续追问： 1. 为什么 destructive action 必须二次确认？2. Agent loop 为什么可能无限循环？3. MCP 与具体工具实现之间如何解耦？4. 什么情况下重试反而危险？

面试高频问题

Q: RAG 的召回率和精确率如何影响答案？

召回率低会漏掉关键事实；精确率低会把噪声塞进上下文，导致模型被干扰或引用错误材料。

Q: 智能体和普通函数调用有什么不同？

智能体会根据观察迭代规划和调用工具；普通函数调用通常是一次性参数填充，不一定有多步状态和反馈循环。

第十二部分

高频面试问题与速查

第二十三章　高频公式速查 I

面试一句话回答

架构与推理（Architecture & Inference）：公式要沿“张量形状 → 信息流 → 系统成本”复习。面试时不仅写出公式，还要说明归一化维度，以及它影响计算、显存还是带宽。

主题	公式与面试解释
Q/K/V 投影	$Q = XW_Q,\ K = XW_K,\ V = XW_V$。三种投影分离查询条件、匹配索引和被聚合内容；投影本身也会混合特征通道。
缩放点积注意力	$A = \mathrm{softmax}\big(QK^\top/\sqrt{d_k} + M\big),\ O = AV$。softmax 对每个 query 沿 key 位置归一化；M 控制可见性。
多头输出	$O_{\mathrm{mha}} = \mathrm{Concat}(O_1, \ldots, O_H)W_O$。$W_O$ 重新组合各头，并把输出投影回固定残差宽度。
Residual stream	$X_{\ell+1} = X_\ell + F_\ell(X_\ell)$。逐元素相加要求同维，因此 attention 与 FFN 最终都回到 d_{model}。
RMSNorm	$\mathrm{RMSNorm}(x) = g \odot x/\sqrt{d^{-1}\sum_i x_i^2 + \epsilon}$。沿 hidden dimension 控制尺度，不保证输出均值为零。
RoPE	$q_m = R_m q,\ k_n = R_n k,\ q_m^\top k_n = q^\top R_{n-m} k$。对 Q/K 旋转，使点积显式依赖相对位置。
SwiGLU	$\mathrm{SwiGLU}(x) = (\mathrm{SiLU}(xW_g) \odot xW_u)\, W_d$。门控与内容在宽空间逐元素结合，再下投影。
GQA 缓存比例	$M_{\mathrm{GQA}}/M_{\mathrm{MHA}} = n_{kv}/H$。保持 query head 数 H，减少 KV head 数 n_{kv}，可按比例压缩 KV Cache。
KV Cache	$M_{\mathrm{KV}} = 2LBTn_{kv}d_h s$ bytes。2 对应 K/V，L, B, T, d_h, s 分别是层数、批量、长度、头维和每元素字节。
温度采样	$p_i(T) = \exp(z_i/T)/\sum_j \exp(z_j/T)$。$T < 1$ 使分布更尖锐；$T \to 0^+$ 时趋近 greedy。

第二十四章　高频公式速查 II

面试一句话回答

训练与后训练（Training & Post-Training）：公式要能指出“优化谁、基线是谁、归一化在哪里”。尤其不要混淆 PPO 的旧策略与参考策略，也不要把 GRPO 的无独立 value model 误解为没有基线。

主题	公式与面试解释
自回归似然	$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t})$。链式法则把序列学习变为每个位置的下一词元预测。
交叉熵	$\mathcal{L} = -\sum_t \log p_\theta(x_t \mid x_{<t})$，且 $\partial \ell / \partial z_i = p_i - \mathbf{1}[i = y]$。错误类别概率越高，梯度修正越大。
困惑度	$\mathrm{PPL} = \exp(\mathcal{L}_{\text{token}})$。它是平均负对数似然的指数；不同 tokenizer 的 token-level PPL 不宜直接比较。
AdamW	$\theta_{t+1} = \theta_t - \eta_t \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) - \eta_t \lambda \theta_t$。权重衰减与自适应梯度更新解耦。
LoRA	$W' = W + \Delta W,\ \Delta W = (\alpha/r)BA$。若 $W \in \mathbb{R}^{d_o \times d_i}$，可训练参数为 $r(d_i + d_o)$。
PPO ratio	$r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\text{old}}(a_t \mid s_t)$。它比较新旧策略对已采样动作的概率，不是 reward。
PPO 裁剪目标	$\mathbb{E}[\min(r_t \hat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)]$。clip 限制代理目标从过大策略变化中获益。
DPO	$-\log \sigma\left(\beta[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}]\right)$。学习偏好差，同时用参考策略锚定变化。
GRPO	$\hat{A}_i = (r_i - \bar{r}) / (\mathrm{std}(r) + \epsilon)$。同一 prompt 的组内奖励统计形成相对基线，不需单独训练 value model。
MoE	$y = \sum_{i \in \mathrm{TopK}(g(x))} p_i(x) E_i(x)$。总参数可很大，但每个 token 只激活少数专家。
pass@k	$1 - \binom{n-c}{k} / \binom{n}{k}$。从 n 个样本中选 k 个时，至少一个通过测试的无偏估计形式。

第二十五章　Interview Last-Day Checklist

- ☐ 能手推 attention 的完整 tensor shape
- ☐ 能解释为什么除以 $\sqrt{d_k}$
- ☐ 能解释 Q/K/V 为什么独立投影
- ☐ 能解释 residual stream 固定宽度
- ☐ 能解释 FFN 为什么升维再降维
- ☐ 能证明无激活的两层 linear 仍是 linear
- ☐ 能准确解释跨 token 通信与逐位置非线性变换
- ☐ 能解释 RoPE 为什么作用于 Q/K
- ☐ 能计算 MHA/GQA 的 KV Cache
- ☐ 能区分 prefill 与 decode 的瓶颈
- ☐ 能解释 FlashAttention 的 IO 优化
- ☐ 能解释 KV Cache 不缓存 Q
- ☐ 能推导 temperature 的极限行为
- ☐ 能区分量化、LoRA 与 QLoRA
- ☐ 能估算训练 model-state memory
- ☐ 能解释 compute-optimal 的参数/数据权衡
- ☐ 能区分 thinking 与 direct mode
- ☐ 能解释 test-time scaling 的上限
- ☐ 能解释 RLVR 的 verifier 风险
- ☐ 能区分 PPO 的旧策略与参考策略
- ☐ 能区分 PPO、DPO 与 GRPO
- ☐ 能解释 MoE 为什么替换 FFN
- ☐ 能说明 MoE 的 All-to-All 瓶颈
- ☐ 能逐项说明 DDP/TP/PP/FSDP 切什么
- ☐ 能解释 PPL 与任务体验的差异
- ☐ 能拆解 RAG 的召回与重排指标
- ☐ 能说明 Agent 的权限与幂等边界

25.1　不同岗位怎么刷这本书？

目标岗位	优先路径
基础模型 / Pretraining	Transformer → 预训练 → 现代架构 → 分布式训练 → 评测
Post-Training / Alignment	SFT → Reward Model → PPO → DPO → GRPO → Reasoning / RLVR
推理优化	Attention → GQA/MLA → KV Cache → FlashAttention → 量化 → 连续批处理 → 推测式解码
LLM 应用 / Agent	Transformer 基础 → 推理基础 → RAG → Agent → 评测

面试一句话回答

刷题时先遮住“短答”，用 20 秒给出结论；再补公式、形状或系统瓶颈；最后沿正文中的“面试官继续追问”自测。页码由 LaTeX 自动引用详细章节，不需要手工维护。

第二十六章　面试问答索引：210 题

26.1　Tokenization（8 题）

1. Tokenizer 解决的根本问题是什么？ → 第2页

短答：把字符串映射为有限词表中的离散编号，同时权衡序列长度、词表规模和鲁棒性。

2. BPE 为什么从字符或字节开始反复合并？ → 第2页

短答：它用频繁共现压缩常见片段，同时保留拆解稀有词的退路。

3. Byte-level BPE 为什么几乎没有 OOV？ → 第2页

短答：任意文本最终都能退化为有限的字节序列。

4. 词表越大一定越好吗？ → 第2页

短答：不一定；序列可能变短，但 embedding/LM head 参数、softmax 成本和长尾词元都会增加。

5. 字符级 tokenizer 的主要系统代价是什么？ → 第2页

短答：词表小但序列更长，会放大 attention、KV Cache 和生成步数。

6. 同一句话为什么在不同模型里 token 数不同？ → 第2页

短答：分词算法、训练语料、预分词规则和特殊词元定义不同。

7. 中文为什么不能简单按“一个字一个 token”理解？ → 第2页

短答：真实 tokenizer 可能组合多字词、字节与标点，边界由词表学习和规则共同决定。

8. Tokenizer 怎样影响推理成本？ → 第2页

短答：它改变输入长度和输出步数，进而影响 prefill FLOPs、KV Cache 与 decode latency。

26.2　Transformer Fundamentals（27 题）

9. Token embedding 的矩阵 shape 是什么？ → 第5页

短答：$E \in \mathbb{R}^{V \times d_{\text{model}}}$，每个 token id 选择其中一行。

10. 为什么 hidden state 不是概率？ → 第5页

短答：它是任意实数特征向量，须经 LM head 和 softmax 才成为词表分布。

11. LM head 为什么投影到 vocabulary size？ → 第5页

短答： 下一词元是 V 类分类问题，每个词元都需要一个 logit。

12. Embedding 与 LM head 为什么可以 weight tying？ → 第5页

短答： 输入和输出共享同一词元语义坐标，可省去一份 Vd_{model} 参数。

13. Softmax 为什么沿 vocabulary dimension 做？ → 第5页

短答： 每个位置要在全部候选词元之间形成归一化分布。

14. Temperature 为什么作用于 logits 更自然？ → 第5页

短答： 缩放 logits 等价于统一调节相对 log-odds，再由 softmax 正确归一化。

15. 为什么 Q、K、V 使用不同投影矩阵？ → 第7页

短答： 它们分别学习查询条件、匹配索引和被聚合内容，独立投影允许职责分化。

16. 若直接令 $Q = K = V = X$ 会损失什么？ → 第7页

短答： 匹配空间与内容空间被强制绑定，表达能力受限。

17. 为什么 Q/K 匹配而 V 传递内容？ → 第7页

短答： $QK^\top$ 只产生权重，真正被权重加权求和的是 V。

18. Attention 为什么除以 $\sqrt{d_k}$？ → 第7页

短答： 点积方差随 d_k 增长；缩放防止 softmax 过早饱和和梯度变小。

19. Softmax 在 attention 的哪个维度归一化？ → 第7页

短答： 对每个 query，沿可见 key 的序列维归一化。

20. 因果遮罩屏蔽哪一侧？ → 第7页

短答： 位置 t 屏蔽未来 key，即通常屏蔽注意力矩阵上三角。

21. d_k 必须等于 d_v 吗？ → 第7页

短答： 数学上不必；前者决定匹配维度，后者决定每头输出宽度。

22. 多头 attention 为什么不是重复计算？ → 第7页

短答： 不同头有独立投影，可学习不同关系子空间与注意模式。

23. 固定 d_{model} 时增加 head 数会怎样？ → 第7页

短答： 每头维度通常变小，粒度增加但单头容量和硬件效率可能下降。

24. 为什么 concat 后还需要 W_O？ → 第7页

短答： 它混合各头信息并映射回 residual stream 的固定宽度。

25. Residual stream 是什么？ → 第12页

短答： 贯穿各层的固定宽度状态，子层只向其写入增量。

26. 为什么 residual stream 通常保持 d_{model}？ → 第12页

短答： 固定接口使任意深度的残差相加和层堆叠都无需反复改维。

27. 残差连接的作用只是防梯度消失吗？ → 第12页

短答： 不是；它还提供 identity 信息路径并让子层学习增量修正。

28. Pre-Norm 为什么更接近 identity path？ → 第12页

短答： 归一化位于分支内，残差主路径可不经过归一化直接跨层传播。

29. 子层输出维度与 residual 不同怎么办？ → 第12页

短答： 必须投影到同维，或对 shortcut 做匹配投影后才能相加。

30. Attention 与 FFN 的职责有何不同？ → 第14页

短答： Attention 的独特作用是跨 token 通信，但其 Q/K/V/O 投影也会混合特征；FFN 不跨 token，而是逐位置执行更强的非线性特征变换。

31. FFN 为什么先升维再降维？ → 第14页

短答： 在更宽中间基上做非线性变换，再回到固定 residual 宽度控制全网成本。

32. 为什么不只做 $d_{\text{model}} \to d_{\text{model}}$？ → 第14页

短答： 可以但中间非线性特征基更窄，通常限制逐 token 变换容量。

33. 若 FFN 没有激活函数会怎样？ → 第14页

短答： 两层线性可合并为一个矩阵，仍然只是线性变换。

34. d_{ff} 越大一定越好吗？ → 第14页

短答： 不一定；容量增加同时抬高参数、FLOPs、激活和带宽成本。

35. 为什么 MoE 通常替换 FFN 而不是 attention？ → 第14页

短答： FFN 天然逐 token 独立，易稀疏路由；attention 承担跨 token 通信，稀疏化更改交互结构。

26.3 Pretraining + Scaling Laws（25 题）

36. 自回归似然如何分解？ → 第21页

短答： $p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$。

37. 为什么预测下一个词元能产生监督？ → 第21页

短答： 序列右移一位即可得到每个位置的真实标签，无需人工标注。

38. 为什么下一词元目标可能学到通用能力？ → 第21页

短答：降低条件熵需要建模语法、事实、语义和数据中的推理结构。

39. Logits 与 labels 怎样错位对齐？ → 第21页

短答：位置 $0 \ldots T-2$ 的 logits 预测位置 $1 \ldots T-1$ 的 token。

40. 交叉熵与 KL 散度是什么关系？ → 第21页

短答：数据熵固定时，最小化交叉熵等价于最小化 $D_{KL}(p_{data} \| p_\theta)$。

41. 教师强制为什么能并行？ → 第21页

短答：真实序列一次输入，因果遮罩让每个位置只读取其真实前缀。

42. 暴露偏差来自哪里？ → 第21页

短答：训练读取真实前缀，推理读取模型自身生成前缀，两种状态分布不同。

43. Padding token 为什么要 mask loss？ → 第21页

短答：补齐不是学习目标，否则模型会把批处理格式当作数据。

44. Sequence packing 解决什么问题？ → 第26页

短答：把短样本装入固定长度块，减少 padding 的无效计算。

45. Packing 最容易引入什么错误？ → 第26页

短答：未隔离样本边界会造成跨样本 attention 和标签泄漏。

46. 数据去重为什么重要？ → 第26页

短答：减少重复记忆、算力浪费和 benchmark contamination。

47. AdamW 与 Adam 加 L2 的关键区别是什么？ → 第26页

短答：AdamW 将参数衰减从自适应梯度预条件中解耦。

48. Warmup 解决什么问题？ → 第26页

短答：训练初期统计量与表示不稳定时，限制过大的更新步长。

49. 梯度累积降低哪些显存？ → 第26页

短答：主要降低单个微批次的激活峰值，不减少参数与优化器状态。

50. 梯度裁剪与 warmup 能互相替代吗？ → 第26页

短答：不能；前者限制当前异常梯度，后者调节训练初期学习率日程。

51. 什么是 Scaling Law？ → 第28页

短答：从受控实验中拟合的经验关系，用来描述损失如何随参数量、训练 token 和计算量变化。

52. Scaling Law 是经验规律还是数学定理？ → 第28页

短答：是依赖训练条件的经验规律；特定系数和指数不能当作普适定理。

53. 为什么参数量越大不一定越好？ → 第28页

短答：参数会挤占固定预算下可训练的 token 数；模型过大但数据不足可能欠训练，部署成本也更高。

54. 什么叫 under-trained model？ → 第28页

短答：模型容量相对训练信号过大，许多参数尚未被足量数据和优化步骤有效塑造。

55. 什么叫 compute-optimal training？ → 第28页

短答：在给定计算预算下协调模型规模与训练 token，使预算带来的质量收益尽可能大。

56. 固定 FLOPs 时为什么要平衡模型大小和训练 token 数？ → 第28页

短答：粗略地 $C \propto ND$；增大参数会压缩可训练数据量，反之亦然。

57. 模型很大但训练 token 太少会怎样？ → 第28页

短答：可能欠训练；同等计算下，更小但训练更充分的模型可能达到更低损失。

58. 数据很多但模型太小会怎样？ → 第28页

短答：容量可能成为瓶颈，新增 token 的边际收益下降，计算未必被高效利用。

59. Scaling Law 对训练预算规划有什么价值？ → 第28页

短答：可用小规模试验拟合趋势，比较等计算量方案并估计扩展收益与风险。

60. 为什么不能机械套用固定参数/token 比例？ → 第28页

短答：最优点依赖架构、数据质量、训练流程、计算范围和部署目标，论文比例只适用于其研究条件。

26.4 Modern Architecture（20 题）

61. LayerNorm 为什么要减均值？ → 第33页

短答：它提供平移与缩放归一化，使特征中心和尺度受控。

62. RMSNorm 为什么可以不减均值？ → 第33页

短答：它只追求重缩放不变性，经验上可省去重定中心步骤。

63. RMSNorm 保证输出均值为零吗？ → 第33页

短答：不保证；它只控制均方根。

64. Transformer 的 norm 沿哪个维度做？ → 第33页

短答： 通常对每个 token 的 hidden dimension 独立归一化。

65. 为什么 Transformer 不主要用 BatchNorm？ → 第33页

短答： 序列长度、batch 统计和自回归场景变化大，跨样本统计会引入不稳定依赖。

66. RMSNorm 的 ϵ 是正则化吗？ → 第33页

短答： 不是；它主要防止分母过小导致数值不稳定。

67. SwiGLU 为什么有 gate 与 up 两条投影？ → 第14页

短答： 一条产生数据相关门控，另一条提供被调制的内容特征。

68. SwiGLU 的 gate 为什么不是简单 sigmoid？ → 第14页

短答： SiLU 同时提供平滑非线性和非严格 $[0, 1]$ 的幅度调制。

69. SwiGLU 为什么常重新调整 hidden size？ → 第14页

短答： 它有三块大矩阵，为与普通 FFN 参数预算接近需缩小中间宽度。

70. FFN 中真正的非线性在哪里？ → 第14页

短答： 在激活函数以及 SwiGLU 的逐元素门控乘法。

71. 为什么绝对位置 embedding 不总是够用？ → 第35页

短答： attention 更关心 token 间相对距离，且固定表在训练长度外缺少自然定义。

72. RoPE 为什么作用于 Q/K 而不是 V？ → 第35页

短答： 位置信息需要改变匹配分数；V 负责传递被选中的内容。

73. RoPE 怎样在点积中产生相对位置？ → 第35页

短答： 相同旋转族满足 $R_m^T R_n = R_{n-m}$。

74. RoPE 会增加参数量或改变 shape 吗？ → 第35页

短答： 不会；它是无参数、保持维度的二维旋转。

75. RoPE 为什么成对处理 head dimension？ → 第35页

短答： 每两个坐标构成一个二维平面，才能应用旋转矩阵。

76. RoPE 长上下文外推为何会退化？ → 第35页

短答： 测试位置产生训练中未覆盖的旋转相位和距离分布。

77. Position interpolation 的基本思想是什么？ → 第35页

短答： 把更长位置压缩回训练时熟悉的坐标范围，但牺牲位置分辨率。

78. MHA、MQA、GQA 的 KV head 数怎样变化？ → 第39页

短答： MHA 每个 Q head 一组 KV，MQA 仅一组，GQA 介于二者之间。

79. $H = 32, n_{kv} = 8$ 时每个 KV head 服务几个 Q head？ → 第39页

短答： $32/8 = 4$ 个，理想 KV Cache 为 MHA 的四分之一。

80. MLA 与 GQA 压缩 KV 的思路有何不同？ → 第41页

短答： GQA 减少 KV head 数；MLA 缓存更低维的潜在表示。

26.5 Inference / KV / Flash（27 题）

81. Prefill 是什么？ → 第45页

短答： 提示词预填充：一次并行处理完整输入并建立每层 KV Cache。

82. Decode 是什么？ → 第45页

短答： 逐词元解码：每一步基于历史 cache 生成一个新 token。

83. KV Cache 缓存什么？ → 第45页

短答： 每层历史 token 的 K 与 V 投影结果。

84. 为什么通常不缓存 Q？ → 第45页

短答： 旧 query 在其生成步使用后不会再参与未来 token 的注意力计算。

85. 不用 KV Cache 的重复计算在哪里？ → 第45页

短答： 每个新步骤都会重新投影并处理全部历史 token 的 K/V。

86. KV Cache 显存为什么随上下文线性增长？ → 第45页

短答： 每新增 token 都需在每层追加一份 K 与一份 V。

87. KV Cache 的通用显存公式是什么？ → 第45页

短答： $2LBTn_{kv}d_h s$ bytes。

88. KV Cache 会让 attention 变成 $O(1)$ 吗？ → 第45页

短答： 不会；历史 K/V 无需重算，但新 query 仍需扫描不断增长的历史。

89. GQA 省的是 FLOPs、显存还是带宽？ → 第39页

短答： 主要省 KV 参数/缓存与 decode 读取带宽；prefill 主体 FLOPs 不按同倍数下降。

90. 为什么 decode 比 prefill 更容易受 KV bandwidth 限制？ → 第39页

短答： 单步矩阵较小、复用低，却反复读取长历史 cache，算术强度较低。

91. 为什么所有模型不直接使用 MQA？ → 第39页

短答： 最大共享可能损失 KV 表达多样性和模型质量。

92. Prefix caching 与普通 KV Cache 有何区别？ → 第45页

短答： 前者在请求之间复用相同前缀，后者主要在单次请求的解码步之间复用。

93. KV Cache quantization 的收益与风险是什么？ → 第60页

短答： 降低容量和读取带宽，但量化误差会持续影响后续全部解码步。

94. PagedAttention 主要解决计算还是内存管理？ → 第60页

短答： 主要解决 KV Cache 的分页分配、碎片和请求动态增长。

95. Continuous batching 解决什么？ → 第60页

短答： 请求到达和结束时动态重组批次，减少设备空闲。

96. FlashAttention 会改变 attention 数学结果吗？ → 第50页

短答： 在相同精度和实现约定下是精确分块计算，不是近似 attention。

97. FlashAttention 主要减少 FLOPs 吗？ → 第50页

短答： 主要减少 HBM 与片上存储之间的数据移动和中间矩阵物化。

98. 为什么不能保存完整 $T \times T$ attention matrix？ → 第50页

短答： 长序列下其二次显存与读写成本很高。

99. Online softmax 为什么允许分块 attention？ → 第50页

短答： 维护逐行最大值、归一化和与加权和即可精确合并各块。

100. FlashAttention 与 KV Cache 解决同一问题吗？ → 第50页

短答： 前者优化一次 attention 内部 IO，后者避免跨解码步重算历史 K/V。

101. 量化中的 scale 和 zero point 分别做什么？ → 第53页

短答： scale 映射实数步长，zero point 表示实数零在整数域的位置。

102. Per-channel 为什么常比 per-tensor 精确？ → 第53页

短答： 不同通道可使用各自动态范围，减少被极端值共同拉伸。

103. 当 temperature $T \to 0$ 时为何接近 greedy？ → 第54页

短答： 最大 logit 的指数相对其他项占据全部概率质量。

104. Top-k 与 Top-p 的根本区别是什么？ → 第54页

短答： 前者固定候选数量，后者按累计概率动态决定集合大小。

105. Beam search 为什么容易偏好短序列？ → 第54页

短答：序列对数概率是负项累加，未校正时长序列得分天然更低。

106. 推测式解码为什么能保持目标分布？ → 第59页

短答：目标模型并行验证草稿，并用接受/拒绝校正保证最终采样分布。

107. Draft model 太弱或太大会怎样？ → 第59页

短答：太弱接受率低；太大自身成本高，二者都会吞噬加速收益。

26.6 Reasoning / Thinking（23 题）

108. 什么是 reasoning model？ → 第63页

短答：在多步问题上被训练或配置为投入额外推理计算、搜索或验证的模型。

109. Thinking 与普通生成的本质区别是什么？ → 第63页

短答：前者在最终回答前分配更多中间计算预算；后者偏向直接低延迟输出。

110. Thinking 是否意味着参数发生改变？ → 第63页

短答：不一定；同一权重可由模式、预算、提示或解码策略产生不同推理行为。

111. 同一模型为何可能有 thinking/non-thinking 两种行为？ → 第63页

短答：服务端可改变推理预算、控制 token、搜索和验证路径。

112. Chain-of-Thought 与 reasoning capability 是一回事吗？ → 第63页

短答：不是；CoT 是提示或可见表达形式，能力还取决于模型与验证过程。

113. Hidden reasoning 与可见 reasoning trace 有何区别？ → 第63页

短答：内部计算不必原样暴露；可见解释通常是面向用户的受控输出。

114. Reasoning 能力可能来自哪些阶段？ → 第63页

短答：预训练、SFT、RL、蒸馏与 inference-time search 都可能贡献。

115. 为什么 reasoning 越长不一定越正确？ → 第63页

短答：错误前提会被延长，冗余步骤增加偏航机会，且缺少可靠验证。

116. Reasoning token 增加意味着什么成本？ → 第63页

短答：更高 decode latency、算力、能耗和服务费用。

117. 为什么数学和代码更受益于 reasoning？ → 第63页

短答：任务可分解且常有可检查中间约束或最终验证器。

118. 什么叫 test-time scaling？ → 第66页

短答：不改参数，在推理时增加轨迹长度、候选、搜索或验证预算。

119. Test-time compute 与 training-time compute 有何区别？ → 第66页

短答： 前者为具体请求临时付费，后者把能力固化进权重供多次复用。

120. 增加采样次数为什么可能提高正确率？ → 第66页

短答： 多个候选提高覆盖到正确解的概率，但需要可靠选择器。

121. Self-consistency 为什么可能有效？ → 第66页

短答： 不同推理路径若汇聚到同一答案，可用一致性抵消部分偶发错误。

122. Best-of-N 的前提是什么？ → 第66页

短答： 候选需有足够多样性，且 verifier 排序与真实质量相关。

123. Verifier 不可靠会发生什么？ → 第66页

短答： 系统可能稳定选中表面高分但错误或投机的候选。

124. 什么是 RLVR？ → 第69页

短答： 使用可自动验证的结果奖励优化策略，常见于数学、代码和形式化任务。

125. 为什么数学题适合 RLVR？ → 第69页

短答： 答案常能由规则或证明检查器给出低歧义反馈。

126. Outcome reward 的主要问题是什么？ → 第69页

短答： 只看终点，难以给长轨迹中的中间决策分配信用。

127. Process reward 的优势与风险是什么？ → 第69页

短答： 反馈更密集，但标注昂贵且错误过程标准可能限制有效新路径。

128. Sparse reward 为什么让 RL 困难？ → 第69页

短答： 大多数 rollout 同分且无梯度方向，探索很难碰到成功样本。

129. 多 rollout 为什么有助于 reasoning RL？ → 第69页

短答： 增加探索覆盖，并为组内相对基线或候选选择提供样本。

130. Reasoning distillation 的额外风险是什么？ → 第72页

短答： 教师的冗长、捷径和错误轨迹会被学生一并模仿。

26.7 Post-Training / PEFT / Distillation（32 题）

131. SFT 为什么可 mask prompt loss 但 prompt 仍参与 attention？ → 第75页

短答： loss mask 只取消标签监督；prompt 仍是生成回答所需条件。

132. 若把 prompt token 从输入删除会怎样？ → 第75页

短答： 回答失去任务条件，模型无法知道应回答什么。

133. Assistant-only loss 与 full-sequence loss 有何区别？ → 第75页

短答： 前者只监督回答，后者还拟合用户提示和模板 token。

134. SFT 能否学到 demonstration 中没有的能力？ → 第75页

短答： 可组合和迁移预训练能力，但不能假设凭少量示例创造任意新知识。

135. Reward model 为什么输出 scalar？ → 第77页

短答： 策略优化需要把整段回答映射为可比较的单一偏好信号。

136. Reward 的绝对值重要吗？ → 第77页

短答： 排序训练主要依赖差值；尺度却会影响策略更新强度与稳定性。

137. Pairwise ranking 为什么常比直接打分稳定？ → 第77页

短答： 人更容易判断两者谁好，减少个人评分尺度不一致。

138. Bradley–Terry 模型假设什么？ → 第77页

短答： 偏好概率由两个候选潜在标量分数之差的 logistic 函数决定。

139. Reward hacking 是什么？ → 第77页

短答： 策略利用代理奖励漏洞获得高分，却没有提高真实目标质量。

140. PPO 中 reward、return 与 value 分别是什么？ → 第80页

短答： reward 是即时反馈，return 是累计回报，value 是状态期望回报估计。

141. Q value 与 advantage 分别是什么？ → 第80页

短答： $Q(s,a)$ 评估动作后回报，$A = Q - V$ 衡量相对基线好多少。

142. PPO probability ratio 是 reward 吗？ → 第80页

短答： 不是；它是新旧策略对已采样动作的概率比。

143. PPO 为什么需要 old policy？ → 第80页

短答： 它定义采样分布与更新前基线，使 importance ratio 和 clip 可计算。

144. PPO clipping 限制的是什么？ → 第80页

短答： 限制概率比继续推动代理目标的收益，不是直接裁剪模型参数。

145. PPO clipping 等于严格 KL constraint 吗？ → 第80页

短答： 不等于；它是逐样本代理目标截断，不能保证全局 KL 上界。

146. Old policy 与 reference policy 是同一个模型吗？ → 第80页

短答：不是；old 是近期行为策略快照，reference 通常是固定锚点以限制分布漂移。

147. DPO 为什么不需要显式 reward model？ → 第83页

短答：它把 KL 正则化偏好目标重参数化为 policy 与 reference 的对数概率差。

148. DPO 为什么仍需要 reference policy？ → 第83页

短答：reference 提供相对锚点，决定偏好提升是相对哪个基础分布。

149. DPO 的 β 控制什么？ → 第83页

短答：控制相对 reference 偏移与偏好间隔的尺度。

150. DPO 是不是 RL？ → 第83页

短答：它来源于 KL 正则 RL 目标，但训练常是固定偏好数据上的离线监督式优化。

151. DPO 与普通 pairwise classification 的区别是什么？ → 第83页

短答：分类 logit 由 policy 相对 reference 的序列对数概率差构造。

152. GRPO 为什么不训练独立 value model？ → 第85页

短答：它用同一 prompt 多个回答的组内奖励统计构造相对优势。

153. GRPO 没有 critic 是否等于没有 baseline？ → 第85页

短答：不是；组均值或标准化奖励就是样本级 baseline。

154. GRPO 中所有 group rewards 相同会怎样？ → 第85页

短答：相对优势接近零，该组几乎不提供区分学习信号。

155. Reward std 很小时为什么不稳定？ → 第85页

短答：除以很小标准差会放大噪声，因此需要 ϵ 或其他稳健处理。

156. Group size 增大有什么取舍？ → 第85页

短答：基线更稳定、探索更多，但 rollout 成本和延迟上升。

157. LoRA 为什么假设更新可以低秩？ → 第90页

短答：许多任务适配所需变化集中在较低维子空间，这是经验假设而非普遍定理。

158. LoRA 的 α/r 做什么？ → 第90页

短答：控制低秩分支的有效更新尺度，便于不同 rank 下调节。

159. LoRA 为什么常让初始 $\Delta W = 0$？ → 第90页

短答： 训练从原模型函数出发，避免随机适配分支先破坏输出。

160. LoRA merge 是什么意思？ → 第90页

短答： 部署前把 $(\alpha/r)BA$ 加回冻结权重，消除独立分支。

161. QLoRA 与普通 INT4 推理量化一样吗？ → 第93页

短答： 不一样；QLoRA 通过量化冻结基座节省微调显存，只训练适配器。

162. Soft-target distillation 比 hard label 多提供什么？ → 第94页

短答： 提供类别间相似性和教师不确定性，而不只有唯一正确类别。

26.8 MoE（11 题）

163. MoE 为什么通常把 FFN 换成 experts？ → 第97页

短答： FFN 逐 token 独立，适合条件路由而不破坏 attention 的跨 token 交互。

164. 为什么不直接把 attention heads 当 experts？ → 第97页

短答： head 是同一次跨 token 聚合的组成部分，路由会改变通信和注意结构。

165. Top-1 与 Top-2 routing 有何区别？ → 第97页

短答： Top-2 冗余和容量更强，但专家计算与通信更高。

166. MoE 为什么总参数大而每 token FLOPs 不同比增长？ → 第97页

短答： 每个 token 只激活 Top-k 个专家。

167. Activated parameters 与 total parameters 有何区别？ → 第97页

短答： 前者是单 token 实际参与计算的子集，后者是全部专家容量。

168. Router collapse 为什么发生？ → 第97页

短答： 早期微小偏好形成正反馈，使少数专家持续获得更多 token 和训练信号。

169. Load balancing loss 优化什么？ → 第97页

短答： 鼓励 token 负载和路由概率更均匀地分布到专家。

170. Capacity factor 是什么？ → 第97页

短答： 为每个专家预留容量相对平均负载的倍率。

171. Expert Parallel 为什么需要 All-to-All？ → 第97页

短答： token 必须发送到专家所在设备，再把输出送回原并行布局。

172. MoE FLOPs 不大为什么 latency 仍可能高？ → 第97页

短答： 路由、All-to-All、小矩阵和负载不均会降低硬件利用率。

173. 专家数越多一定越好吗？ → 第97页

短答： 不一定；容量增加同时加剧路由、通信、存储和训练均衡难题。

26.9 Distributed Training（16 题）

174. DDP 切了什么、没切什么？ → 第104页

短答： 切数据批次；每卡仍保存完整参数、梯度和优化器状态。

175. DDP 为什么每张 GPU 存完整参数？ → 第104页

短答： 各卡需独立完成相同模型的前后向，只在梯度阶段同步。

176. Gradient 为什么用 AllReduce 而不是 Gather？ → 第104页

短答： 每卡都需要聚合后的完整梯度更新其本地参数副本。

177. Tensor Parallel 切什么？ → 第104页

短答： 切单层矩阵或 attention/FFN 的张量维度。

178. Tensor Parallel 为什么每层常有通信？ → 第104页

短答： 被切开的局部矩阵结果必须规约或拼接，才能进入下一算子。

179. Pipeline Parallel 切什么？ → 第104页

短答： 沿网络深度把不同层分配到不同设备。

180. Pipeline bubble 怎样产生？ → 第104页

短答： 流水线填充和排空阶段部分 stage 没有可处理的 micro-batch。

181. Micro-batch 为什么能减小 bubble？ → 第104页

短答： 更多流水片段提高不同 stage 同时忙碌的比例。

182. Sequence Parallel 切什么？ → 第104页

短答： 切序列维的激活或相关计算，以降低长序列激活内存。

183. Expert Parallel 为什么与 MoE 强绑定？ → 第104页

短答： 它把不同专家放到不同设备，并按路由交换 token。

184. ZeRO-1 shard 什么？ → 第104页

短答： 优化器状态。

185. ZeRO-2 比 ZeRO-1 多 shard 什么？ → 第104页

短答： 梯度。

186. ZeRO-3 再多 shard 什么？ → 第104页

短答： 模型参数。

187. FSDP forward 前为什么可能 AllGather？ → 第104页

短答：计算某个模块前需临时重建其完整参数。

188. 7B BF16 为什么不只需 14 GB 就能训练？ → 第107页

短答：还要梯度、FP32 主权重、Adam 状态、激活与临时/通信缓冲。

189. Activation checkpointing 与 ZeRO 解决同一种显存吗？ → 第107页

短答：不是；前者压缩激活，后者分片模型状态。

26.10 Evaluation（9 题）

190. Validation loss 下降为什么不代表一定更好用？ → 第112页

短答：它测词元似然，不直接测指令遵循、事实、安全和用户偏好。

191. 不同 tokenizer 的模型能直接比较 PPL 吗？ → 第112页

短答：不宜；token 计量单位和序列切分不同。

192. Exact Match 的主要局限是什么？ → 第112页

短答：可能把语义等价的不同表述判错，也忽略部分正确。

193. pass@k 衡量什么？ → 第112页

短答：从 k 个候选中至少有一个通过测试的概率估计。

194. pass@1 与 pass@10 分别说明什么？ → 第112页

短答：前者看单次可靠性，后者还看多样采样覆盖能力。

195. LLM-as-a-judge 有哪些常见 bias？ → 第112页

短答：位置、长度、自偏好、风格和领域知识偏差。

196. Benchmark contamination 为什么危险？ → 第112页

短答：记忆泄漏会抬高分数，使评测不再代表泛化能力。

197. Reward model score 能直接当真实质量吗？ → 第112页

短答：不能；它是可能被利用且分布外失真的代理指标。

198. Post-training 后 PPL 变差一定是模型变差吗？ → 第112页

短答：不一定；可能是目标行为改变，也可能是遗忘，需用分层任务评测区分。

26.11 RAG + Agents（12 题）

199. Chunk 太大和太小分别有什么问题？ → 第117页

短答：太大主题稀释且噪声多；太小破坏语义、表格和上下文边界。

200. Embedding retrieval 与 reranker 分别优化什么？ → 第117页

短答： 前者低成本保召回，后者在小候选集提高精确率。

201. 高 recall 后为什么还要 reranking？ → 第117页

短答： 高召回候选含噪声，直接全部放入上下文会干扰生成并增加成本。

202. Metadata filtering 为什么重要？ → 第117页

短答： 它在语义相似候选中强制权限、租户、语言和版本约束。

203. Hybrid search 是什么？ → 第117页

短答： 融合向量语义检索与 BM25 等词法检索。

204. RAG 怎样处理文档版本冲突？ → 第117页

短答： 记录版本和生效时间，先过滤再重排，并在答案中引用实际证据。

205. Lost in the Middle 是什么？ → 第117页

短答： 模型对长上下文中部相关信息的利用可能显著弱于开头或结尾。

206. Agent 和 workflow 的区别是什么？ → 第120页

短答： Agent 动态选择下一动作；workflow 的主要步骤与分支由程序预定义。

207. Agent 为什么需要状态？ → 第120页

短答： 保存跨步骤事实、工具结果、计划进度与恢复信息。

208. Tool call 为什么需要 schema？ → 第120页

短答： 把自然语言意图约束成可解析、可验证的结构化参数。

209. 什么是 idempotency？ → 第120页

短答： 重复执行同一请求不会产生额外副作用，是安全重试的基础。

210. MCP 在 Agent 系统中做什么？ → 第120页

短答： 标准化上下文、工具和资源接口，解耦模型应用与外部能力提供方。

参考文献与进一步阅读

面试一句话回答

以下优先列出原始论文或官方技术报告。面试中不要求背诵完整作者列表，但应知道方法解决的问题、适用边界，以及原始工作是否声称“精确”等价或只是经验近似。

1. Vaswani et al. (2017). Attention Is All You Need. NeurIPS 2017.
2. Kaplan et al. (2020). Scaling Laws for Neural Language Models. 研究语言模型损失随参数量、数据量和训练计算变化的经验幂律。
3. Hoffmann et al. (2022). Training Compute-Optimal Large Language Models. 在其受控实验范围内研究固定计算预算下模型规模与训练 token 的平衡。
4. Zhang and Sennrich (2019). Root Mean Square Layer Normalization. NeurIPS 2019.
5. Su et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding.
6. Shazeer (2019). Fast Transformer Decoding: One Write-Head is All You Need.
7. Ainslie et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP 2023.
8. DeepSeek-AI et al. (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. MLA 的原始技术说明来源。
9. Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.
10. Leviathan, Kalman, and Matias (2023). Fast Inference from Transformers via Speculative Decoding. ICML 2023.
11. Hu et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
12. Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.
13. Hinton, Vinyals, and Dean (2015). Distilling the Knowledge in a Neural Network.
14. Schulman et al. (2017). Proximal Policy Optimization Algorithms.
15. Rafailov et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.
16. Shao et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. 文中提出 GRPO。
17. Wang et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models.
18. Snell, Lee, Xu, and Kumar (2024). Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. 支撑按题目难度分配推理时计算及 verifier-guided search 的讨论。
19. Cobbe et al. (2021). Training Verifiers to Solve Math Word Problems. 支撑多候选生成、验证器排序与可检验数学任务的讨论。

20. Lightman et al. (2023). Let's Verify Step by Step. 对比 outcome supervision 与 process supervision，并发布步骤级反馈数据 PRM800K。

21. Zelikman, Wu, Mu, and Goodman (2022). STaR: Bootstrapping Reasoning With Reasoning. 支撑用筛选后的推理过程迭代自训练的公开方法。

22. Hsieh et al. (2023). Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. 支撑将教师 rationale 作为额外监督的蒸馏思路。

23. DeepSeek-AI et al. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. 支撑该报告公开披露的强化学习、多阶段训练与蒸馏模型信息。

24. Shazeer et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR 2017.

25. Rajbhandari et al. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. SC 2020.

26. Zhao et al. (2023). PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. VLDB 2023.

27. Chen et al. (2021). Evaluating Large Language Models Trained on Code. HumanEval 与 pass@k 估计。

28. Liu et al. (2023). Lost in the Middle: How Language Models Use Long Contexts.

阅读建议：先根据正文的问题定位原论文，再重点阅读 problem setup、method、ablation 和 limitations。技术报告中的具体模型数字具有实现依赖性，本书只把它们作为方法来源，不把单一模型结果外推为普遍规律。公开技术报告只能支持公开披露的训练流程，不能据此推断未公开的内部训练配方。